

Уроки HTML

СОДЕРЖАНИЕ

1. Структура HTML-кода	5
1.1 Параграф	5
1.2 Заголовок	6
1.3 Базовая структура	7
1.4 Семантические теги	9
1.5 Шапка	11
1.6 Меню	13
1.7 Элемент для основного содержания страницы	15
1.8 Секция	17
1.9 Самостоятельная секция	20
1.10 Боковая панель	23
1.11 Подвал	25
1.12 Метатеги	26
1.12.1 Атрибут charset	28
1.12.2 Атрибуты name и content	29
1.12.3 Атрибуты http-equiv и content	34
1.13 Фавиконки	35
1.14 Семантический WEB	38
1.15 Микроразметка	41
1.15.1 Микроразметка контактов	42
2. Текст	44
2.1 Форматирование текста	44
2.2 Перенос строки	46
2.3 Сохранения оригинального форматирования текста	47
2.4 Спец. Символы	48
2.5 Всплывающая подсказка	50
2.6 Вставка кода в HTML	52
3. Ссылки	53
3.1 Относительные ссылки	55
3.2 Абсолютные ссылки	56
3.3 Ссылки-якоря	57
3.4 Ссылки на файл	58
3.5 Ссылки на электронную почту	59

3.6	Ссылки на телефон	61
3.7	Пустые ссылки	62
4.	Изображения.....	64
4.1	Атрибуты изображений	64
4.1.1	Атрибут src.....	64
4.1.2	Атрибут alt	65
4.2	Размеры изображений	66
5.	Аудио.....	67
5.1	Атрибуты аудио	69
6.	Видео	70
6.1	Атрибуты видео	72
6.2	Fallback-контент.....	73
6.3	Атрибут aria-label.....	74
6.4	Субтитры	75
7.	Списки.....	78
7.1	Ненумерованный список.....	78
7.2	Нумерованный список	81
7.3	Горизонтальный список	83
7.4	Вложенные списки	84
8.	Таблицы	86
8.1	Атрибуты таблиц	88
8.1.1	Атрибуты тега <table>	88
8.1.2	Атрибуты тега <td>	94
9.	Формы	100
9.1	Поле для ввода текста	101
9.2	Поле для пароля.....	103
9.3	Многострочный текст	104
9.4	Кнопки	106
9.5	Переключатели	110
9.6	Флажки.....	115
9.7	Поле со списком.....	117
9.8	Скрытое поле.....	123
9.9	Поле с изображением	125
9.10	Загрузка файлов	127

9.11	Адрес электронной почты	131
9.12	Веб-адрес	133
9.13	Выбор цвета.....	134
9.14	Ввод чисел.....	136
9.15	Ползунок.....	138
9.16	Дата и время	140
9.17	Поле для поиска.....	146
9.18	Поле для телефона.....	148
9.19	Группирование элементов форм.....	149
9.20	Базовые атрибуты форм.....	151
9.20.1	Переход между полями с помощью табуляции	151
9.20.2	Блокирование элементов форм.....	154
9.20.3	Поле только для чтения	157
9.20.4	Автофокус	159
9.20.5	Подсказывающий текст	161
9.20.6	Шаблон для ввода данных.....	162
9.21	Атрибуты для защита от дурака.....	165
9.21.1	Обязательное поле.....	166
9.21.2	Корректность данных	167
9.21.3	Шаблон ввода	169
9.21.4	Отмена валидации	171
	Заключение	173

Уроки HTML

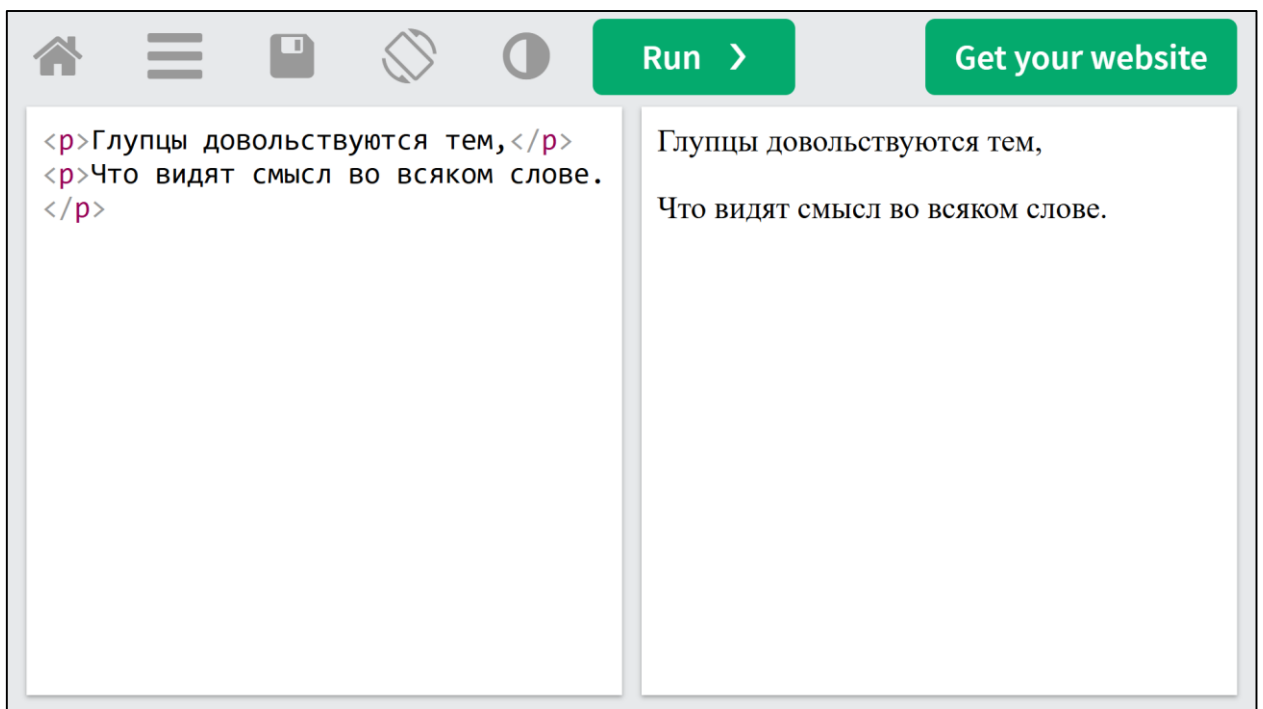
1. Структура HTML-кода

1.1 Параграф

Параграфы создаются с помощью тегов `<p>` и `</p>`, которые заключают некоторое содержимое. Каждый новый параграф располагается на новой строке.

Пример 1:

```
<p>Глупцы довольствуются тем,</p>  
<p>Что видят смысл во всяком слове.</p>
```

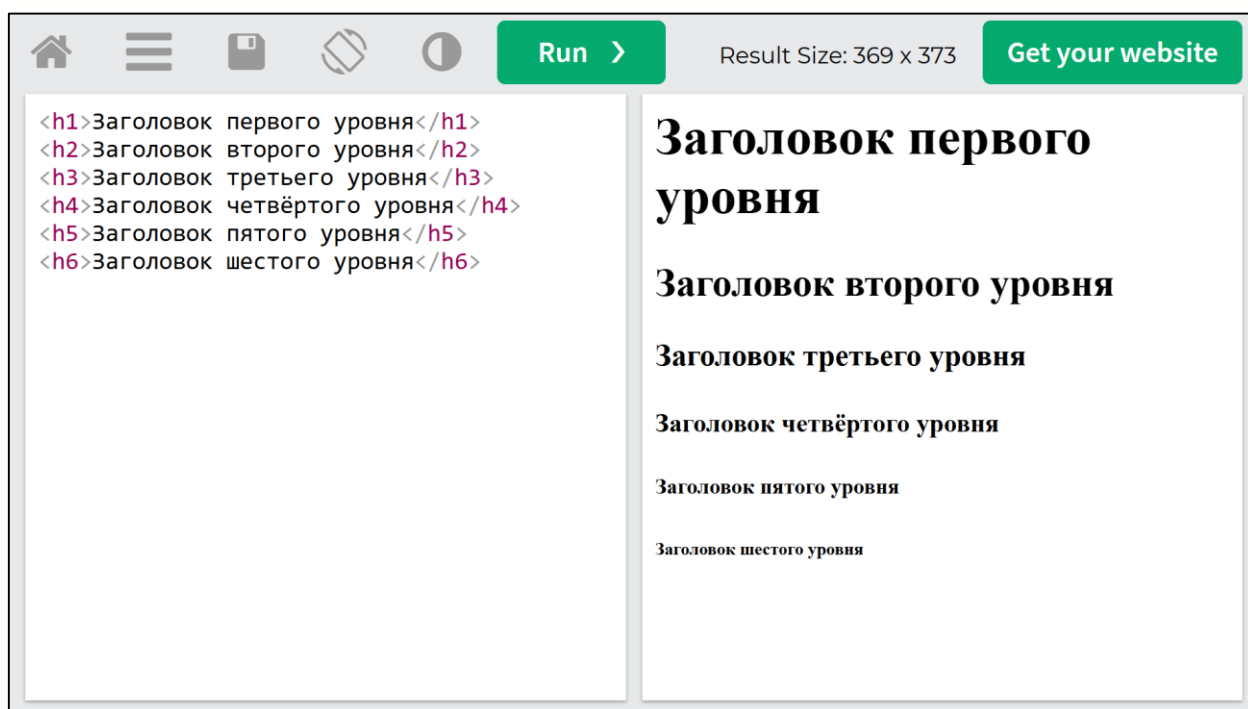


1.2 Заголовок

Для создания заголовков используется 6 парных тегов: `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, где `<h1>` — заголовок первого уровня, самый важный и описывающий главную тему текста, а `<h6>` — заголовок самого низшего уровня. Важно, чтобы на странице был только один заголовок первого уровня, так как он выполняет роль основного заголовка веб-страницы.

Пример 1:

```
<h1>Заголовок первого уровня</h1>
<h2>Заголовок второго уровня</h2>
<h3>Заголовок третьего уровня</h3>
<h4>Заголовок четвёртого уровня</h4>
<h5>Заголовок пятого уровня</h5>
<h6>Заголовок шестого уровня</h6>
```



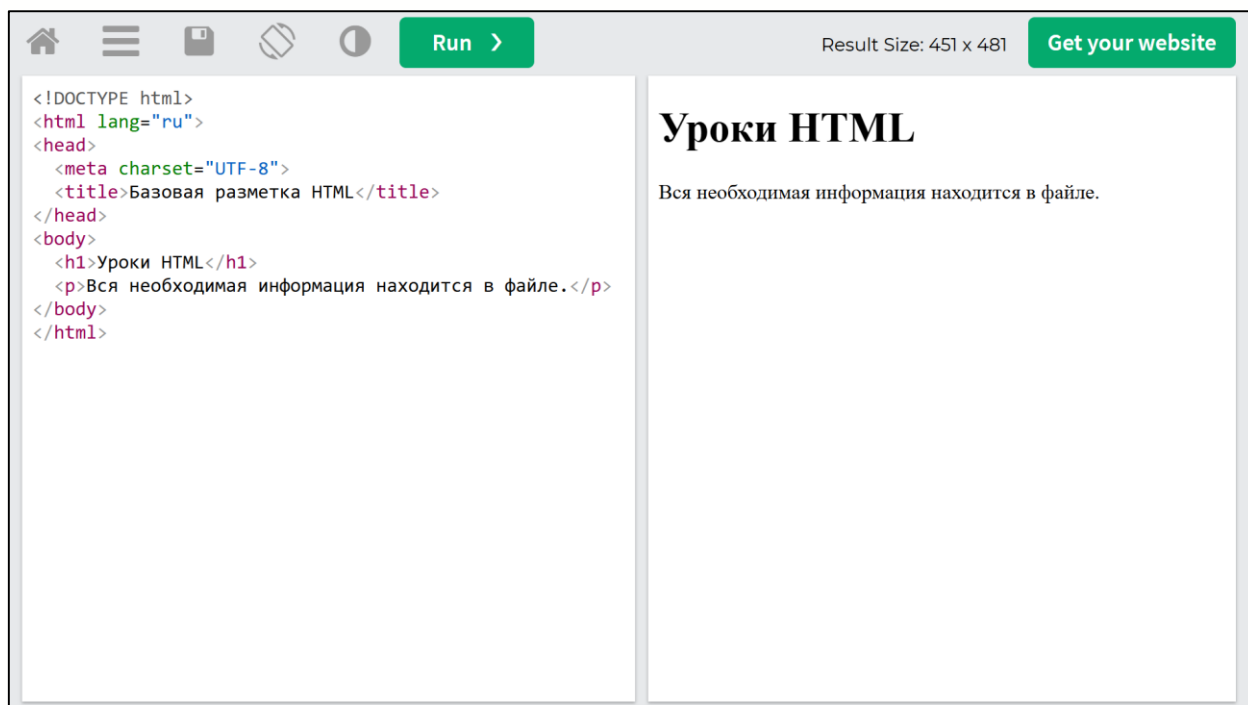
1.3 Базовая структура

Любой HTML-документ можно разбить на несколько основных составляющих:

- Блок **head**, содержащий метаинформацию о нашем сайте. Данные из этого блока не отображаются непосредственно на странице, а служат для её описания, подключения стилей и скриптов.
- Блок **body** является основным местом, где строится HTML-разметка. Данные в этом блоке будут обработаны и выведены в браузер.

Пример 1:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Базовая разметка HTML</title>
</head>
<body>
  <h1>Уроки HTML</h1>
  <p>Вся необходимая информация находится в файле.</p>
</body>
</html>
```



Обратите внимание на первую строку `<!DOCTYPE html>`. DOCTYPE (document type) — служебная информация для браузера, где описывается стандарт HTML, который нужно обработать. Текущий стандарт HTML5 поддерживается всеми браузерами, поэтому достаточно указать в первой строке документа `<!DOCTYPE html>`, который говорит, что наш документ размечен по стандарту HTML5.

Далее открывается парный тег `<html>` с атрибутом `lang="ru"`. Указание языка также необходимо для браузеров, особенно если сайт рассчитан на массовую аудиторию, в том числе и за рубежом. ([Ссылка на языковые теги](#)). Его также могут использовать и в `<meta>`, но в таком случае проще будет отойти от этого и создать новые страницы, если сайт является многоязычным.

Во избежание ошибок, обе вышеописанные конструкции, при их отсутствии, будут добавлены браузером автоматически. В некоторых ситуациях браузеры могут добавить не те строки, которые вы бы хотели, и вместо обработки страницы по стандарту HTML5 браузер будет обрабатывать всё как HTML4, что приведёт ко множеству проблем, так как старые стандарты могут не знать о существовании многих тегов, таких как `<main>`, `<nav>` и так далее.

О meta и ее атрибутах говорится в через одну главу.

Чтобы упростить себе жизнь и не создавать каждый раз с нуля новый сайт, постоянно прописывая базовые конструкции, в Visual Studio Code есть специальная комбинация клавиш – “!” + “TAB”.

1.4 Семантические теги

При создании HTML-документа разработчики используют теги `<div>` и `<span>`. Они позволяют создать блочный или строчный элемент, что помогает создавать независимые блоки с разным оформлением. Но, для браузера и поисковых систем, эти блоки не являются семантическими, т.е. они не говорят напрямую о том, что в них лежит.

С первых версий стандарта HTML теги изначально обладают семантикой, например, `<p>` — параграф, `<table>` — таблица, `<ol>`, `<ul>` — списки. Но этими тегами не получится разметить «шапку», «подвал», «боковую панель» сайта.

До выхода стандарта HTML5 для разметки таких областей использовали классы или идентификаторы.

Это решило проблемы с точки зрения разработчиков, ведь навигация по коду стала быстрее, но, с точки зрения браузера, ситуация не изменилась. Браузеры не понимают, что `id="header"` используется на сайте для обозначения шапки сайта, да и разработчики именовали области по-своему, так как не было единого стандарта.

С выпуском стандарта HTML5 появились новые теги, которые стали обозначать семантику областей страницы, а не только текста.

Основными семантическими тегами стали:

- `<article>`
- `<aside>`
- `<footer>`
- `<header>`
- `<main>`
- `<nav>`
- `<section>`

Большинство элементов интуитивно понятны и их можно сразу использовать в своих документах.

Пример 1:

```
<header>  
  // Здесь будет шапка сайта. Обычно в ней располагается логотип,  
  контактная информация, меню  
</header>  
  
<main>  
  // Уникальный контент сайта  
</main>  
  
<footer>  
  // Подвал сайта. Обычно здесь находится контактная информация,  
  дополнительное меню, юридическая информация о компании. О нем  
  расскажем чуть позже  
</footer>
```

В этом разделе мы рассмотрим основные семантические теги на едином примере. От главы к главе усложняя и дорабатывая страницу сайта.

1.5 Шапка

Шапкой на сайте принято называть верхнюю область, в которой находятся контактные данные, меню, логотип. До появления стандарта HTML5 данную область просто размечали, используя обычный блок `<div>`, что не вносило «смысла».

В HTML5 для разметки шапки сайта используется парный тег `<header>`, внутри которого располагаются элементы.

Пример 1:

```
<header>
   <!-- Логотип сайта -->
  <div id="menu"> <!-- Меню -->
    <ul>
      <li><a href="/">Главная</a></li>
      <li><a href="/about">О нас</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </div>
</header>
```



Тег <header> по своему поведению не отличается от тега <div> и не содержит отличных стандартных стилей, поэтому его можно использовать на любом сайте, не боясь нарушить внешний вид сайта.

1.6 Меню

Вернёмся к [примеру](#) из прошлого урока.

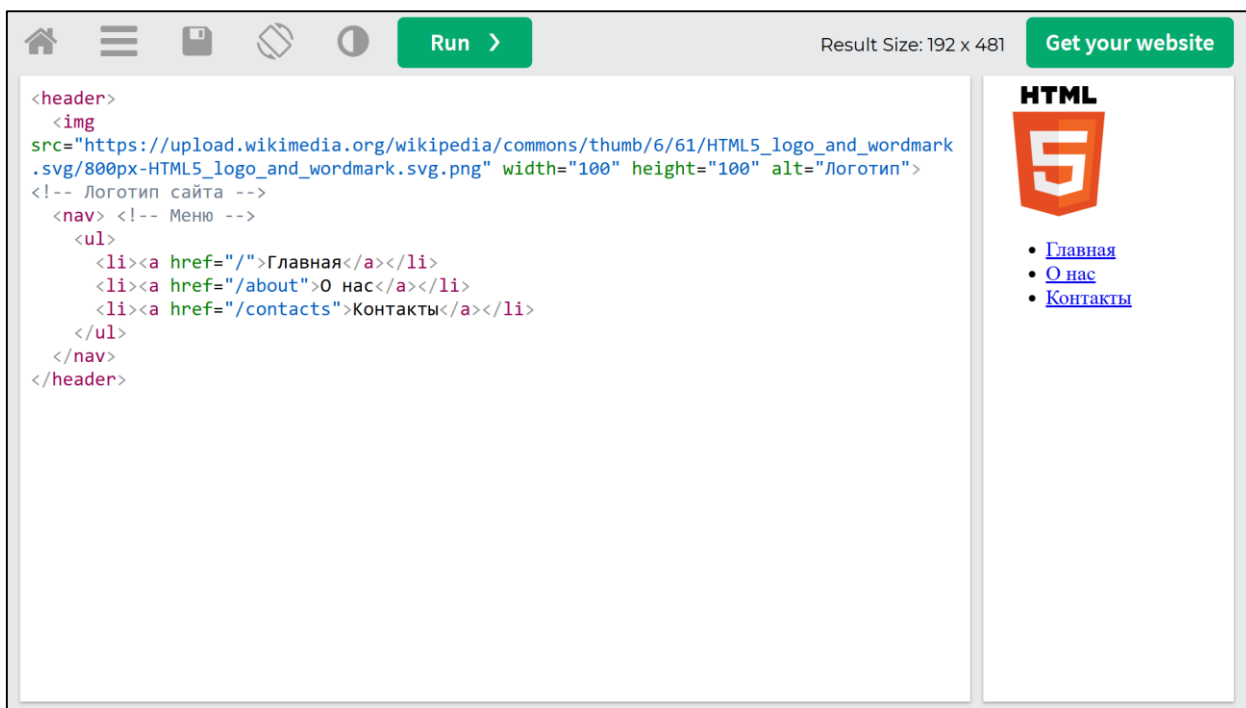
В нём меню сайта размечено с помощью обычного тега `<div>` с идентификатором `menu`. Как известно, такое обозначение работает для разработчиков, но не для браузеров. Они не понимают, что это меню, а не просто обёртка для списка.

Для создания полноценного меню используется парный тег `<nav>`, задача которого — разметить область страницы с основным меню. Также различные устройства для чтения с экрана, которыми пользуются люди со слабым зрением, используют этот тег, чтобы определить, стоит ли его отображать на странице.

Заменим `<div id="menu"></div>` на изученный тег `<nav>`.

Пример 1:

```
<header>
   <!-- Логотип сайта -->
  <nav> <!-- Меню -->
    <ul>
      <li><a href="/">Главная</a></li>
      <li><a href="/about">О нас</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </nav>
</header>
```



Элемент `nav`, как и `header` не является уникальным для страницы. Его можно использовать для любого меню на странице, но есть несколько рекомендаций:

- 1) Нет необходимости оборачивать каждое меню в элемент `nav`. В основном такой областью указываются только основные меню. Дополнительные, такие как меню в футере, не оборачивают в тег `<nav>`, хотя это не запрещено.
- 2) Хороший пример дополнительного использования `nav` — навигация по текущей странице.
- 3) `nav` может содержать не только ссылки, но также и текст, в котором есть ссылки. Ориентируйтесь на здравый смысл. Если навигация является основной для страницы или всего сайта, то оберните её в тег `<nav>`.

1.7 Элемент для основного содержания страницы

<main> — семантический тег для основного содержимого страницы. Отличает центральный контент от второстепенных блоков (шапки, подвала, навигации).

Важно:

- Данный элемент должен быть единственным на странице;
- Скринридеры используют его для быстрой навигации по странице.

Пример 1:

```
<!DOCTYPE html>
<html>

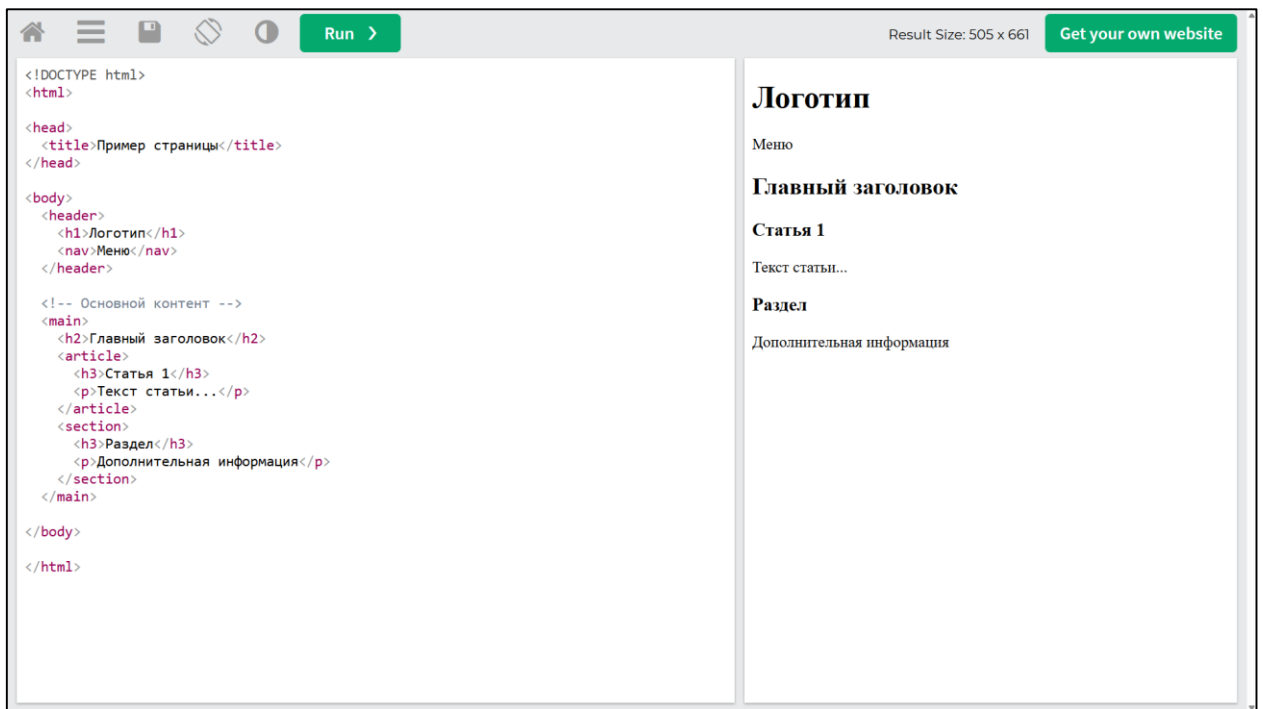
<head>
  <title>Пример страницы</title>
</head>

<body>
  <header>
    <h1>Логотип</h1>
    <nav>Меню</nav>
  </header>

  <!-- Основной контент -->
  <main>
    <h2>Главный заголовок</h2>
    <article>
      <h3>Статья 1</h3>
      <p>Текст статьи...</p>
    </article>
    <section>
      <h3>Раздел</h3>
      <p>Дополнительная информация</p>
    </section>
  </main>

</body>

</html>
```



1.8 Секция

На больших страницах содержится много контента, который помещён друг в друга. Внутри шапки может быть отдельная область с правилами использования, внутри main секции с формой, дополнительной информацией и так далее. Они могут быть выделены с помощью тега `<section>`, значение которого — разметка связанных, по смыслу, блоков внутри секции.

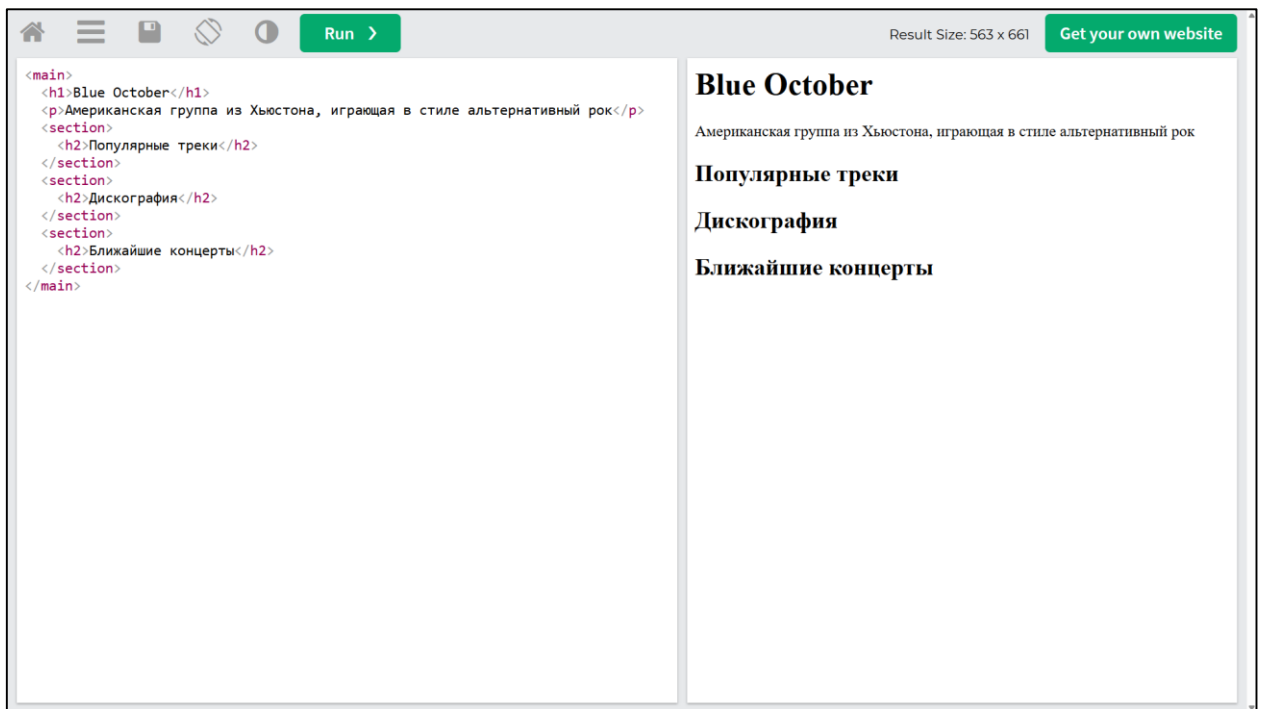
Может звучать немного запутано, но представьте себе страницу музыкального портала, у которого есть области:

- Информация об исполнителе
- Популярные треки исполнителя
- Дискография
- Список ближайших концертов

Если это страница об исполнителе, то весь этот контент будет расположен внутри секции main, при этом стоит выделить и эти 4 секции. Они связаны с общим контентом, но являются самостоятельными единицами внутри него. В примере 1 приведена такая верстка.

Пример 1:

```
<main>
  <h1>Blue October</h1>
  <p>Американская группа из Хьюстона, играющая в стиле альтернативный рок</p>
  <section>
    <h2>Популярные треки</h2>
  </section>
  <section>
    <h2>Дискография</h2>
  </section>
  <section>
    <h2>Ближайшие концерты</h2>
  </section>
</main>
```



Если «вытащить эти секции» из тега `<main>`, то они потеряют свой смысл, так как без текста об исполнителе такая информация может быть непонятна. Такая информация напрямую связана с основным содержанием и не может существовать без него.

Добавим в учебный пример форму заказа услуги. Эта форма для заказа именно той услуги, которая представлена на странице, поэтому секция находится внутри области `main` и вынесена в отдельный тег `<section>`.

Пример 2:

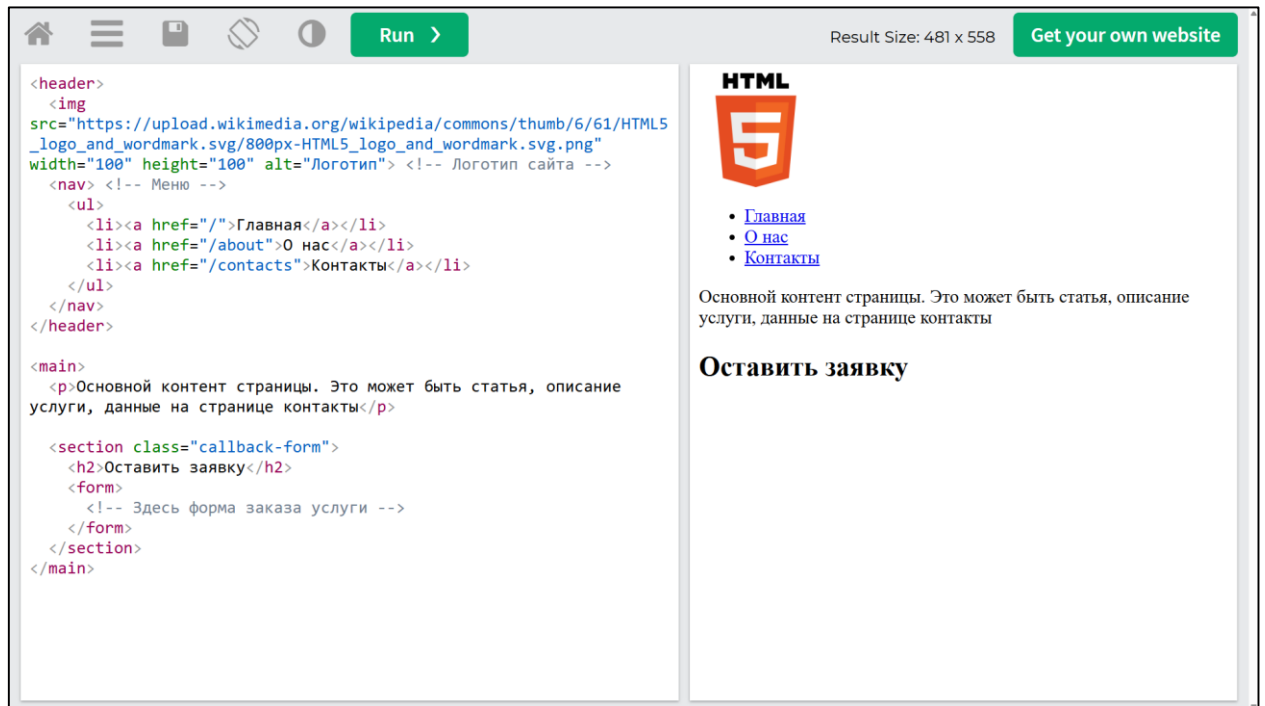
```
<header>
   <!-- Логотип сайта -->
  <nav> <!-- Меню -->
    <ul>
      <li><a href="/">Главная</a></li>
      <li><a href="/about">О нас</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </nav>
</header>

<main>
  <p>Основной контент страницы. Это может быть статья, описание
услуги, данные на странице контакты</p>
```

```

<section class="callback-form">
  <h2>Оставить заявку</h2>
  <form>
    <!-- Здесь форма заказа услуги -->
  </form>
</section>
</main>

```



В большинстве случаев все области section имеют свой заголовок. Это хороший способ понять, когда нужно добавить область section.

Полезное:

- Область section дополняет основной контент и не должна существовать сама по себе
- Следите за тем, чтобы каждая секция имела свой заголовок

1.9 Самостоятельная секция

Помимо тега `<section>`, для обозначения секции используется тег `<article>`. Он предназначен для разметки секции, которая может быть использована самостоятельно и не привязана структурно к конкретной странице. Например, блок с курсом валют, погода, превью новости или статьи. Данные блоки хоть и могут напрямую касаться текущей страницы, но они представляют собой самостоятельную единицу, которая не потеряет смысл, если взглянуть на неё отдельно от контекста.

В связи с тем, что `<article>` может быть использован самостоятельно, он обязательно внутри себя содержит заголовок.

Пример 1:

```
<header>
   <!-- Логотип сайта -->
  <nav> <!-- Меню -->
    <ul>
      <li><a href="/">Главная</a></li>
      <li><a href="/about">О нас</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </nav>
</header>

<main>
  <p>Основной контент страницы. Это может быть статья, описание
услуги, данные на странице контакты</p>

  <section class="callback-form">
    <h2>Оставить заявку</h2>
    <form>
      <!-- Здесь форма заказа услуги -->
    </form>
  </section>

  <section class="more">
    <h2>Читайте также</h2>
    <article class="article-block">
      <h3>Услуга 2</h3>
      <p>Описание новой услуги</p>
      <a href="#">Ссылка на услугу</a>
    </article>
```

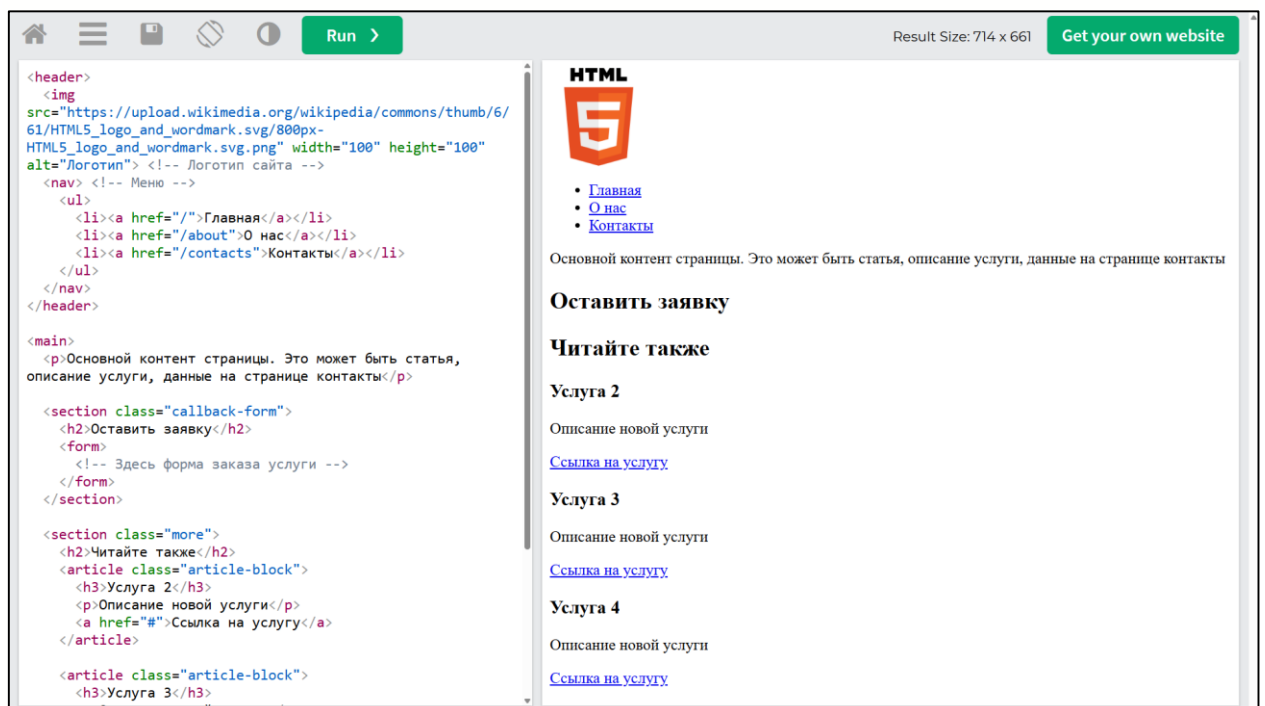


```

<article class="article-block">
  <h3>Услуга 3</h3>
  <p>Описание новой услуги</p>
  <a href="#">Ссылка на услугу</a>
</article>

<article class="article-block">
  <h3>Услуга 4</h3>
  <p>Описание новой услуги</p>
  <a href="#">Ссылка на услугу</a>
</article>
</section>
</main>

```



Обратите внимание, что сами `<article>` обернуты в `<section>`, так как эта секция напрямую связана с текущей страницей и показывает похожие услуги. А услуги размечены с помощью `<article>`. Они могут быть «выдернуты» из контекста, и всё равно будет понятно, что это описание конкретной услуги, которую мы предлагаем.

Примечание:

- Каждый элемент `<article>` должен быть идентифицирован, обычно путём добавления заголовка (элементы `<h1>`-`<h6>`) в качестве дочернего элемента.
- Когда элемент `<article>` является вложенным, внутренний элемент представляет собой контент, связанный с внешним элементом. Например, комментарии к публикации в блоге могут быть

элементами <article>, вложенными в другой <article>, являющийся публикацией в блоге.

- Информация об авторе в элементе <article> может быть представлена через элемент <address>, но это не применимо к вложенным элементам <article>.
- Дата и время публикации в элементе <article> могут быть описаны с помощью атрибута datetime элемента <time>.
- Если область не потеряет свой смысл вне страницы, то её можно обернуть в <article>.

1.10 Боковая панель

Большинство сайтов имеют боковую панель или, как её ещё называют, сайдбар. Он служит для отображения информации, которая дополняет основной текст, но не является обязательной. Чаще всего это меню раздела, список определений из статьи, виджеты и т.д.

Для того чтобы корректно разметить такую область, существует тег **<aside>**, внутри которого и располагается наша боковая панель. Добавим в пример из прошлого урока боковую панель с дополнительным меню.

Пример 1:

```
<header>
   <!-- Логотип сайта -->
  <nav> <!-- Меню -->
    <ul>
      <li><a href="/">Главная</a></li>
      <li><a href="/about">О нас</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </nav>
</header>

<aside> <!-- Боковая панель (сайдбар) -->
  <nav> <!-- Дополнительное меню страницы -->
    <ul>
      <li><a href="/service-1/">Услуга 1</a></li>
      <li><a href="/service-2/">Услуга 2</a></li>
      <li><a href="/service-3/">Услуга 3</a></li>
    </ul>
  </nav>
</aside>

<main>
  <p>Основной контент страницы. Это может быть статья, описание
услуги, данные на странице контакты</p>

  <section class="callback-form">
    <h2>Оставить заявку</h2>
    <form>
      <!-- Здесь форма заказа услуги -->
    </form>
  </section>

  <section class="more">
```

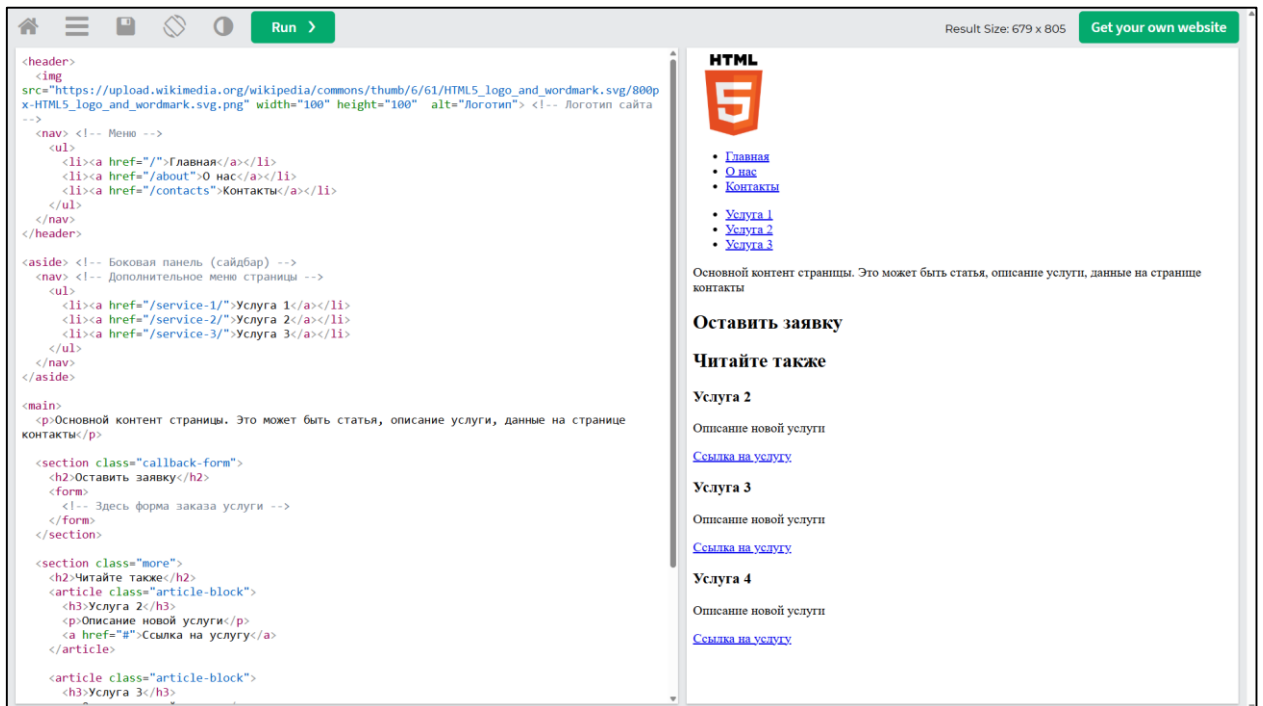
```

<h2>Читайте также</h2>
<article class="article-block">
  <h3>Услуга 2</h3>
  <p>Описание новой услуги</p>
  <a href="#">Ссылка на услугу</a>
</article>

<article class="article-block">
  <h3>Услуга 3</h3>
  <p>Описание новой услуги</p>
  <a href="#">Ссылка на услугу</a>
</article>

<article class="article-block">
  <h3>Услуга 4</h3>
  <p>Описание новой услуги</p>
  <a href="#">Ссылка на услугу</a>
</article>
</section>
</main>

```



Обратите внимание, что `<aside>` содержится вне тега `<main>`. Этот контент не обязан быть уникальным на каждой странице. Он может быть уникальным для каждого конкретного раздела (если речь идёт, например, о меню). Если боковая панель уникальна для каждой страницы, то она должна быть расположена внутри тега `<main>`.

1.11 Подвал

HTML-элемент **<footer>** представляет собой нижний колонтитул (футер, подвал) для своего ближайшего секционного контента или секционного корня. Футер обычно содержит информацию об авторе раздела, информацию об авторском праве или ссылки на связанные документы.

Пример 1:

```
<article>
  <h1>How to be a wizard</h1>
  <ol>
    <li>Grow a long, majestic beard.</li>
    <li>Wear a tall, pointed hat.</li>
    <li>Have I mentioned the beard?</li>
  </ol>
  <footer>
    <p>© 2018 Gandalf</p>
  </footer>
</article>
```

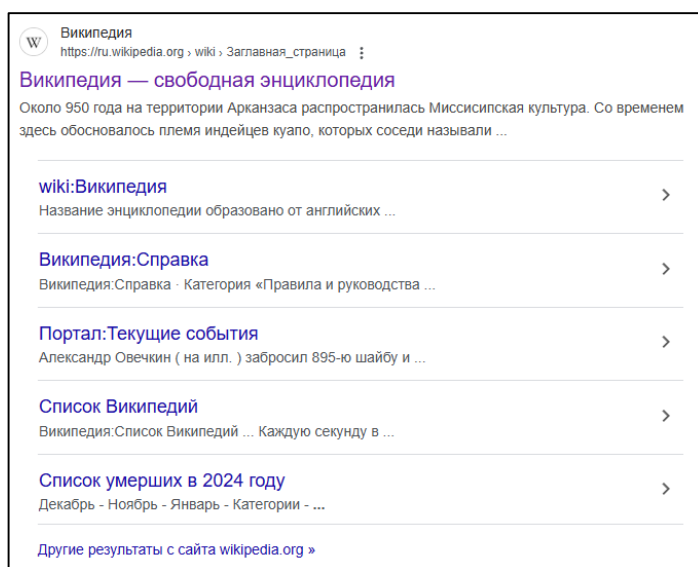


1.12 Метатеги

Метатеги — служебные теги в разметке HTML. Они предназначены для указания сведений поисковым роботам и браузерам. Взгляните на вкладку в браузере с этим уроком, и вы увидите текст, с помощью которого всегда сможете найти нужную вкладку, открытую в браузере.

При выводе результатов поискового запроса (например в Google или Yandex) также показывается этот заголовок и описание.

Пример 1:



При добавлении статьи с сайта на страницу в социальных сетях появляется картинка и описание.

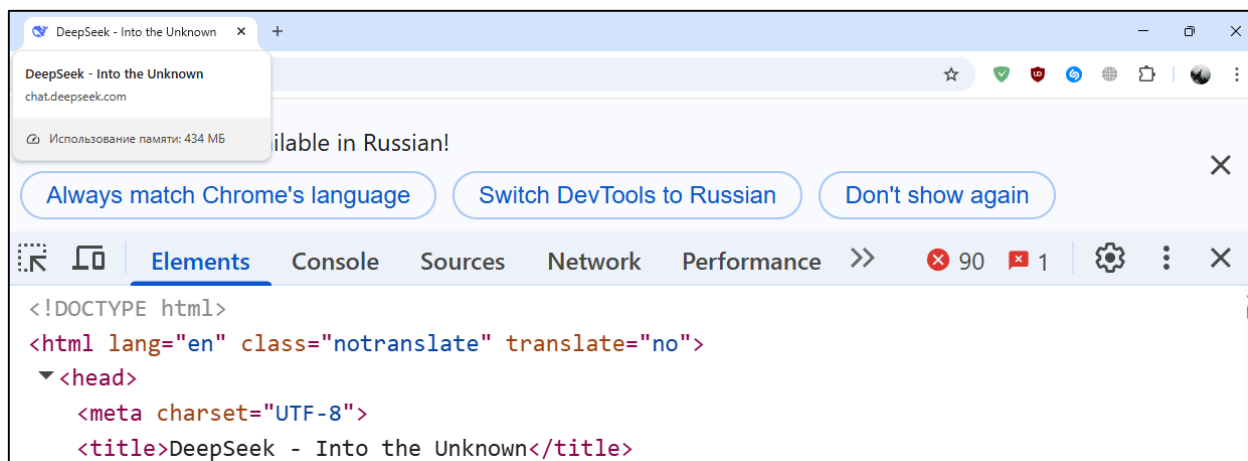
Всё это — работа метатегов. Именно в них можно определить заголовок и описание для текущей HTML-страницы, а также картинку, которая добавится при копировании ссылки в социальных сетях.

Начнем изучение метатегов с заголовка, отображаемого во вкладке любого браузера, за него отвечает парный тег **<title>** (Обязателен), внутри которого содержится название страницы.

Обычно советуют использовать заголовок от 50 до 80 символов (но никто не запрещает сделать его длиной меньшей). Такое ограничение существует из-за того, что поисковые системы не могут показать больше символов в качестве заголовка страницы в результатах поисковой выдачи. Заголовок внутри тега **<title>** отобразится во вкладке в браузере.

Пример 2:

<title>DeepSeek - Into the Unknown</title>



Большинство метатегов записываются с использованием непарного тега **<meta>**. Он принимает несколько основных атрибутов, которые помогают установить метаданные. Содержимое такой информации не видно на странице для пользователя.

Метатеги различаются набором атрибутов и их значений. Один тег содержит одно сообщение: например, описание страницы `<meta name="description">`, ключевые слова `<meta name="keywords" content="...">`, кодировку страницы `<meta charset="UTF-8">` или другие метаданные.

При создании сайта рекомендуется использовать как минимум 3 метатега: `description`, `viewport`, `charset`. Они помогут оптимизировать сайт для браузера пользователя и поисковых систем.

В `<meta>` прописывается информация, которую нельзя указать в `<base>`, `<link>`, `<script>`, `<style>` или `<title>`.

Есть отдельная категория метаразметки — OG-разметка. OG расшифровывается как open graph. Такая разметка нужна для того, чтобы при вставке ссылки на ваш сайт в соцсетях в превью отображалась красивая карточка. ([см. в интернете](#))

Примеры использования метатегов представлены в главе ниже.

1.12.1 Атрибут charset

Атрибут **charset** («набор символов») задает кодировку страницы. Рекомендуется присвоить ему значение – “UTF-8”, но может и принимать другие (“Windows-1251”). Чтобы избежать ошибок кодировки, необходимо уместить метатеги с атрибутом charset в первых 1024 байтах документа. Для этого старайтесь указывать его в самом начале тега <head>.

Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Тег META, атрибут charset</title>
</head>

<body>
  <p>Пример использования атрибута charset</p>
</body>

</html>
```


1.12.2 Атрибуты name и content

Метатег **name** указывает на то, какой тип метаинформации будет описан в теге, через атрибут content. Поэтому они указываются только в паре.

Метатег **content** является ключевым атрибутом элемента `<meta>` и служит для хранения метаданных веб-страницы. Это невидимая для пользователей, но критически важная для браузеров и поисковых систем информация.

Если **name** принимает значение **keywords**, то content будет принимать ключевые слова, по которым сайт сможет найти страницу в интернете. По сути, это самые важные слова из содержания текста страницы. Слова, вписанные в атрибут, должны разделяться запятыми. Content может принимать неограниченное количество ключевых слов, но на практике ограничиваются количеством от 10 до 20 и общей длиной до 255 символов. При использовании данного атрибута бессмысленно дублировать ключевые слова, это будет считаться “спамом”. Не смотря на полезность метатега, злоупотребление ключевыми словами привело к увеличению “спама” и снижению качества результатов поиска, поэтому некоторые браузеры стали игнорировать их или же учитывать их как второстепенный фактор.

Пример 1:

```
<meta name="keywords" content="веб-разработка, создание сайтов, HTML, CSS, JavaScript, фронтенд, программирование, адаптивный дизайн, UX, UI">
```

Если **name** принимает значение **description**, то content будет получать краткое описание страницы, которое видит пользователь, когда находит страницу в поисковике. Описание поможет поисковику найти страницу, а пользователю - узнать, о чем она. Если не добавить описание страницы, то поисковые системы при добавлении в свою базу данных возьмут первый попавшийся текст со страницы и составят из нее так называемый сниппет – текст под заголовком сайта в поисковой выдаче. Не путать с [<title>](#)!

Пример 2:

```
<meta name="description" content="Игровые ноутбуки ASUS ROG с процессорами Intel Core i9 и видеокартами RTX 4080. ✓ Рассрочка 0% ✓ Бесплатная игра в подарок. Тест-драйв в Томске!">
```

Если **name** принимает значение **author**, то content будет нести в себе имя автора страницы.

Пример 3:

```
<meta name="author" content="Вася Пупкин">
```

Если **name** принимает значение **format-detection**, то content может принять значение, которое отключит автоматическое распознавание и преобразование определенных типов данных на веб-страницах (работает только на устройствах Apple). Эти метаданные нужны, чтобы отключить своволие браузера Safari, ведь он очень любит подстраивать номера телефонов в удобный для себя вид. Значения, принимаемые content (если их несколько, то они перечисляются через запятую):

- *telephone=no* - отключает автоматическое определение телефонных номеров
- *email=no* - отключает определение email-адресов
- *address=no* - отключает определение почтовых адресов
- *date=no* - отключает определение дат (iOS 15+)
- *url=no* - отключает определение URL-адресов

Данный атрибут применяется в тех случаях, когда необходимо сохранить номера телефонов как обычный текст, для кастомного оформления кликабельных элементов или если автоматические ссылки мешают дизайну.

Важно упомянуть, что с данным атрибутом явным (те, что пишутся через `<a href="">`) ссылкам ничего не угрожает и они по-прежнему останутся кликабельными.

Пример 4:

```
<meta name="format-detection" content="telephone=no">
```

Если **name** принимает значение **viewport**, то content задаёт параметры окна просмотра в браузере (размеры, масштаб).

Viewport - это видимая пользователю область веб-страницы, то, что может увидеть пользователь, не прибегая к прокрутке.

Окно просмотра представляет собой прямоугольную (обычно прямоугольную) область в компьютерной графике, которая в настоящее время просматривается. В терминах веб-браузера это относится к той части просматриваемого документа, которая в данный момент отображается в её окне (или на экране, если документ просматривается в полноэкранном режиме). Содержимое вне окна просмотра не отображается на экране до тех пор, пока оно не будет просмотрено. Зачастую данное окно шире, чем экран текущего устройства. Это сделано для того, чтобы все элементы поместились так, как задумывалось. Данный эффект можно увидеть, если зайти на

неадаптированную HTML-страницу с мобильного устройства. В таком случае появляется горизонтальная полоса прокрутки.

Впервые данный метатег появился в браузерах от компании Apple «Safari». До появления мобильных устройств браузер Safari по умолчанию считал, что все страницы в интернете имеют ширину в 980 пикселей, из-за чего на первых версиях iPod и iPhone сайты отображались не полностью, а только первые 320 пикселей по ширине.

Метатег viewport может иметь следующие атрибуты, указанные через запятую (,).

width – ширина области просмотра.

Значением атрибута является целое неотрицательное число от 200 до 10000 пикселей или константа *device-width*, которая задаёт ширину страницы в соответствии с размером экрана.

Если значение не задано, по умолчанию устанавливается – в мобильном Safari = 980px, Opera = 850px, Android WebKit = 800px, IE = 974px.

Полезное:

- Для сайтов с адаптивным дизайном рекомендуется использовать: *width=device-width*. Такое значение сообщает браузеру, что ширина страницы равна ширине устройства.

height – высота области просмотра.

Значением атрибута является целое неотрицательное число от 233 до 10000 пикселей или константа *device-height*, которая задаёт высоту страницы в соответствии с размером экрана.

Полезное:

- Если указан атрибут *width*, указывать атрибут *height* не обязательно.

initial-scale – начальный масштаб страницы.

Значением атрибута является вещественное число от 0.1 до 1.0. Значение 1.0 определяет масштаб 1:1, т.е. «не масштабировать».

Полезное:

- В некоторых операционных системах (iOS, Windows Phone и т.д.) ширина страницы, при повороте, остаётся неизменной. Вместо перераспределения контента выполняется его масштабирование. Поэтому рекомендуется использовать: *initial-scale=1.0*.

user-scalable – доступность масштабирования страницы пользователем.

Значение атрибута является логическое «yes» (1) – можно масштабировать или «no» (0) – нельзя масштабировать.

Полезное:

- Рекомендуется использовать значение «yes», а т.к. оно установлено по умолчанию, то user-scalable можно и не указывать.

minimum-scale – минимальный масштаб области просмотра.

Значением атрибута является вещественное число от 0.1 до 1.0. В мобильном браузере Safari по умолчанию 0.25. Значение 1.0 определяет масштаб 1:1, т.е. «не масштабировать».

maximum-scale – максимальный масштаб области просмотра.

Значением атрибута является вещественное число от 0.1 до 1.0. В мобильном браузере Safari по умолчанию 1.6. Значение 1.0 определяет масштаб 1:1, т.е. «не масштабировать».

Полезное:

- Избегайте атрибутов user-scalable, minimum-scale и maximum-scale, т.к. они отрицательно сказываются на доступности содержания.

Пример 5:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=yes">
```

Если **name** принимает значение **HandheldFriendly**, то content принимает значения либо true (отключает масштабирование, отображает страницу в одну колонку и упрощает навигацию), либо false (устройство показывает desktop-версию с возможностью зумирования).

Метатег HandheldFriendly определяет оптимизирована ли страница сайта под мобильные устройства на Palm и Blackberry, в таком браузере как AvantGo. Сейчас распознаётся и многими другими мобильными браузерами.

В настоящее время данный тег устаревает, а его функциональность заменяет адаптивный дизайн на CSS3. Поэтому использование его в новых проектах будет являться архаикой.

Пример 6:

```
<meta name="HandheldFriendly" content="true">
```

Если **name** принимает значение **MobileOptimized**, то content может получить значение, указывающее фиксированную ширину в пикселях, или width, указывающее ширину в соответствии с размером окна.

Метатег MobileOptimized ([подробнее](#)) задаёт ширину области просмотра в мобильных браузерах IE Mobile или Pocket IE. Является аналогом

атрибута width в meta-теге viewport. Но сейчас является устаревшим, ведь на смену ему пришел viewport.

Пример 7:

```
<!-- фиксированная ширина в 320 пикселей -->
<meta name="MobileOptimized" content="320">
<!-- ширина страницы в соответствии с размером экрана, аналог device-
width -->
<meta name="MobileOptimized" content="width">
```

Если **name** принимает значение **apple-mobile-web-app-capable**, то content может получить значение, указывающее фиксированную ширину в пикселях, или width, указывающее ширину в соответствии с размером окна.

Метатег apple-mobile-web-app-capable ([подробнее](#)) позволяет странице работать в полноэкранном режиме, актуален для мобильных устройств Apple.

Если пользователь добавит сайт на свой домашний экран через меню, то при запуске с иконки приложение откроется без интерфейса браузера и возможности масштабировать страницу.

Пример 8:

```
<meta name="apple-mobile-web-app-capable" content="yes">
```

Так какие теги **рекомендуется использовать** при написании страницы сайта?

Также не забывайте использовать набор meta-тегов для сайтов с адаптивным дизайном, заточенных под мобильные устройства.

Пример 9:

```
<meta name="description" content="Все про HTML и его метатеги">
<meta name="keywords" content="HTML, веб-разработка,
программирование">
<meta name="author" content="Вася Пупкин">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="format-detection" content="telephone=no, email=no,
address=no, date=no, url=no">
<!-- Устаревшие метатеги, их можно не использовать -->
<meta name="HandheldFriendly" content="true">
<meta name="MobileOptimized" content="width">
```

1.12.3 Атрибуты `http-equiv` и `content`

Атрибут **`http-equiv`** может изменять поведение страницы или серверов. Используется в паре с [content](#). У него есть несколько значений.

Если **`http-equiv`** принимает значение **`refresh`**, то `content` будет нести в себе то количество секунд, через которое страница будет автоматически перезагружена.

Пример 1:

```
<meta http-equiv="refresh" content="3">
```

Если **`http-equiv`** принимает значение **`default-style`** (предпочтительный стиль таблиц, который используется на странице), то в `content` будет прописываться заголовок из элемента `<link>`, который связан с таблицей CSS-стилей, или заголовок элемента `<style>`, который содержит таблицу CSS-стилей. Такой метод редко используется в современной веб-разработке. Чаще применяются CSS переменные для тем оформления и JavaScript для переключения стилей. (Глава в разработке)

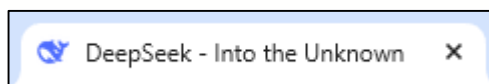
Пример 2:

```
<head>
  <meta http-equiv="default-style" content="light-theme">

  <link rel="stylesheet" href="light.css" title="light-theme">
  <link rel="stylesheet" href="dark.css" title="dark-theme"
media="(prefers-color-scheme: dark)">
</head>
```

1.13 Фавиконки

Почти у всех вкладок в браузере существуют маленькие иконки, которые называются фавиконки. Они, как и заголовок, позволяют быстро ориентироваться во вкладках и переходить на нужную. Их используют поисковые системы, показывая рядом с названием страницы в поиске.



Для того, чтобы установить такую иконку на страницу, используется тег `<link>`, у которого указываются два атрибута:

- **rel** для указания контента
- **href** для указания ссылки на иконку

Пример 1:

```
<link rel="icon" href="/favicon.png">
```

Разные устройства могут отображать иконки разных размеров и форматов, поэтому сейчас добавляется не одна, а сразу несколько иконок. У тега `<link>` возможно указать тип изображения и его размер. Делается это с помощью атрибутов **type** и **sizes**.

Пример 2:

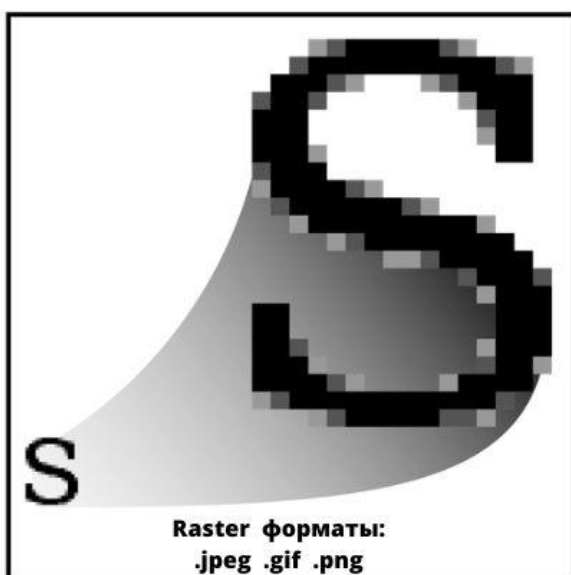
```
<link rel="icon" href="/favicon32.png" type="image/png" sizes="32x32">
```

После неё можно указать ещё несколько иконок с разными размерами, чтобы браузер выбрал ту, которая ему подходит. При указании размеров первым числом указывается ширина в пикселях, а вторым значением высота.

Пример 3:

```
<link rel="icon" href="/favicon64.png" type="image/png" sizes="64x64">  
<link rel="icon" href="/favicon32.png" type="image/png" sizes="32x32">  
<link rel="icon" href="/favicon16.png" type="image/png" sizes="16x16">
```

На этом моменте возникает вопрос. А есть ли конкретные размеры, которые нужно прописать для иконок сайта. Ответ – нет. До сих пор нет конкретного ответа на этот вопрос, каждый делает оп своему, что приводит к засорению кода. Точно такая же проблема обстоит и с форматом самой иконки. Раньше их использовали в формате .ico, но сейчас его уже не используют, так как появились другие форматы, которые имеют наименьший вес и наилучшее качество. Приоритетом же сейчас выступает использование векторного изображения, так как он является наиболее функциональным и в случае чего не теряет качество, в отличие от растрового изображения.



Но на этом трудности не заканчиваются, ведь Apple решает пойти по своему сценарию развития и предлагает собственные размеры фавиконок для своих устройств – Iphone и Ipad. Важно знать, что устройства Apple работают с большими размерами иконок. Тут необходимо отметить, что устройства, которые умеют понимать иконки большого размера, можно подключить более современным способом с помощью манифеста, который позволит скрыть подключение всех размеров из <head>, что сделает разметку более читаемой ([Подробнее о манифесте](#)).

Рекомендуемые форматы и размеры:

- PNG - 16x16, 32x32, 192x192, 512x512
- SVG - векторный вариант (лучшее качество)
- Apple Touch Icon - 180x180 (обязательно), другие размеры опционально

Пример 4:

```
<head>
  <!-- Базовые иконки -->
  <link rel="icon" href="/icon.svg" type="image/svg+xml"
sizes="180x180">
  <link rel="apple-touch-icon" href="/apple-touch-icon.png">

  <!-- Иконки разных размеров -->
  <link rel="icon" href="/favicon-16x16.png" type="image/png"
sizes="16x16">
  <link rel="icon" href="/favicon-32x32.png" type="image/png"
sizes="32x32">

</head>
```

Ниже приведен полный пример с использованием фавиконок.

Пример 5:

```
<head>
  <link rel="apple-touch-icon" sizes="180x180" href="/path/to/apple-
touch-icon.png">
  <link rel="icon" type="image/png" sizes="32x32"
href="/path/to/favicon-32x32.png">
  <link rel="icon" type="image/png" sizes="16x16"
href="/path/to/favicon-16x16.png">
  <link rel="manifest" href="/path/to/site.webmanifest">
  <link rel="mask-icon" href="/path/to/safari-pinned-tab.svg"
color="#5bbad5">
  <meta name="theme-color" content="#ffffff">
</head>
```

1.14 Семантический WEB

С ростом количества страниц в интернете появилась проблема прочтения информации не только человеком, но и роботами. Если человек отделяет смысловые блоки друг от друга, видит расположение и может быстро найти нужный ему участок страницы, то робот видит всю страницу в виде текста и не может самостоятельно разбить её для анализа смысла информации.

Эта проблема остро заявила о себе с начала нового тысячелетия, когда количество страниц в интернете неуклонно стало расти, и поисковые роботы (Google, Bing, Yandex, Yahoo) имели малые возможности для корректного анализа каждой страницы в интернете, из-за чего веб-мастера специально указывали популярные, на то время, ключевые слова на сайте, чтобы подняться в результатах поиска.

Сейчас появились возможности показать роботам смысл (семантику) HTML-страницы. Благодаря этому при запросе в поисковую систему можно увидеть не только список ссылок на сайты, но и конкретную информацию, например, рецепт пирога, маршрут до нужной улицы, контакты организации.

Правильная семантика также позволяет устройствам правильно воспринимать части веб-сайта. Например, режим чтения в телефонах старается отбросить не связанные с контентом блоки, чтобы оставить только смысловую часть.

Ниже приведен пример, взятый с сайта с курсами по программированию.

The screenshot displays a web page titled "HTML разметка" (HTML markup) from a site about programming courses. The page is divided into two main sections to illustrate how HTML markup is interpreted differently in standard display versus reading mode.

Left Section: Стандартное отображение урока на Code Basics

This section shows the page as it appears in a standard browser. It includes a dark header with navigation links (home, html, Основы разметки, HTML 1/27) and a "Войти" (Login) button. Below the header, there are tabs for "Урок" (Lesson) and "Обсуждение" (Discussion). The main content area has the title "HTML разметка" and a paragraph: "Каждый день мы смотрим на десятки интернет-страниц, читаем на них тексты, которые выводятся, как будто из журналов. За такое отображение отвечает HTML." Below this is another paragraph: "HTML — стандарт разметки текста (гипертекста, если говорить точнее), и является набором правил по которым браузер отличает заголовки от списков, таблицы от картинок и так далее. HTML появился в 1993 году и был призван стандартизировать правила для вывода текста внутри веб-страниц."

Right Section: Отображение урока в режиме чтения

This section shows the same content in a reading mode. The header and navigation elements are removed, leaving only the main text. The title "HTML разметка" is at the top. The first paragraph is identical to the standard view. The second paragraph is also identical, but the text is formatted to be more readable, with line breaks and indentation. At the bottom, there is a link: "Взгляните на типичный пример HTML".

Существуют две реализации семантического веба:

- На основе микроразметки. Это специальные атрибуты, которые добавляются в HTML-разметку и помогают роботам найти нужную информацию. Микроразметка используется чаще всего, так как имеет множество возможностей и полей для разметки
- Используя стандарт HTML5. В стандарте появилось множество тегов, которые помогают роботам анализировать информацию, находить логические связи между блоками (или понимать, что их нет), искать необходимые части для корректного отображения на устройствах для чтения

Хотя стандарт HTML5 и не вытеснил микроразметку, ввиду меньших возможностей, его использование вместе с микроразметкой позволяет роботам корректнее обрабатывать сайт и находить в нём логические блоки.

Пример 1:

```
<main> <!-- Определение основной части страницы -->
  <article> <!-- Определение статьи -->
    <header> <!-- Заголовочная информация статьи -->
      <h1>Название статьи</h1>
      
    </header>

    <p>Текст нашей статьи</p>

    <footer> <!-- «Подвал статьи» с указанием автора и даты -->
      <address> <!-- Информация об авторе статьи -->
        <p>Автор: Люнден</p>
        <p>Email: lundennn@mail.ru</p>
      </address>
      <time datetime="2025-05-03">3 мая</time> <!-- Дата написания
статьи -->
    </footer>
  </article>
</main>
```

Run >

Result Size: 255 x 481

Get your website

```

<main> <!-- Определение основной части страницы -->
<article> <!-- Определение статьи -->
  <header> <!-- Заголовочная информация статьи -->
    <h1>Название статьи</h1>
    
  </header>

  <p>Текст нашей статьи</p>

<footer> <!-- «Подвал статьи» с указанием автора и даты -->
  <address> <!-- Информация об авторе статьи -->
    <p>Автор: Люнден</p>
    <p>Email: lundenmn@mail.ru</p>
  </address>
  <time datetime="2025-05-03">3 мая</time> <!-- Дата написания статьи -->
</footer>
</article>
</main>

```

Название статьи

HTML



Текст нашей статьи

Автор: Люнден

Email: lundenmn@mail.ru

3 мая

1.15 Микроразметка

Говоря о семантике, был затронут такой формат разметки, как микроразметка, используемая поисковыми роботами для анализа информации на странице. Правильная разметка страницы с помощью микроразметки помогает выводить в результатах поискового запроса не только ссылку на сайт и описание, но и дополнительную информацию:

- Название организации
- Время работы
- Адрес
- Номер телефона

Микроразметка — инструмент, который реализуется с помощью дополнительных атрибутов HTML-тегов.

Разработчики создали несколько вариантов микроразметки, наиболее распространённым из которых является *Schema.org* с широким выбором «словарей» для описания информации на странице.

Словарь — это набор правил, по которым описываются блоки на странице. Сеансы фильмов в кинотеатре, информация отдела продаж, цены на товары, отзывы о статье — всё это представлено в небольших словарях.

Действия для создания микроразметки:

- Выбор словаря. Узнать о поддержке тех или иных словарей поисковой системой можно с помощью документации. Для поисковой системы Яндекс доступные словари указаны здесь
- Определение блока, который будет размечаться. Это может быть не только блок, а вся страница сайта
- Выделение информации. На примере разметки контактной информации это: название организации, телефон, email, адрес
- Указание атрибутов в зависимости от типа информации и словаря

Полную инструкцию по микроразметке дать в одном уроке невозможно. Это большая тема, со множеством различных нюансов. По мере получения опыта вы научитесь быстро определять необходимую информацию и размечать её. При простой схеме работы, микроразметка даёт огромные возможности — как в плане визуальной выдачи информации в поисковой системе, так и для поискового продвижения.

1.15.1 Микроразметка контактов

Для примера возьмём контактные данные вымышленной компании «Прауд», и разметим контакты, используя *Schema.org*. Изначально блок с контактами выглядит следующим образом:

Пример 1:

```
<section>
  <h1>Компания «Прауд»</h1>
  <p>Адрес: г.Мотино, улица Строителей, дом 6</p>
  <p>Телефон: 8 (8765) 333-00-00</p>
  <p>Email: info@proud-company.test</p>
</section>
```

Первое, нужно выбрать словарь. Поисковая система Яндекс для этого предлагает несколько вариантов. Возьмём словарь *Organization*. Полное описание этого словаря можно найти на сайте <https://schema.org/>.

Для того чтобы подсказать поисковой системе, что данные размечены словарём, указывается два атрибута:

- *itemscope* — указатель на то, что блок — единая сущность, и данные в нем связаны. Указывается только у главного блока и не имеет своего значения
- *itemtype* — указывает на то, какой словарь будет использоваться. Например, для описания контактов выбран словарь **Organization**. Полный вид атрибута: *itemtype*=<http://schema.org/Organization>

Пример 2:

```
<section itemscope itemtype="http://schema.org/Organization">
  <h1>Компания «Прауд»</h1>
  <p>Адрес: г.Мотино, улица Строителей, дом 6</p>
  <p>Телефон: 8 (8765) 333-00-00</p>
  <p>Email: info@proud-company.test</p>
</section>
```

Теперь поисковая система будет знать, что внутри этого блока информация об организации. Приступим к разметке адреса. *Schema.org* позволяет отдельно указывать улицу, дом, строение и так далее, но мы обобщим это и используем только одно значение *itemprop*="address", чтобы обозначить адрес предприятия. Обратите внимание, что элементы разметки указываются с помощью атрибута **itemprop**.

Пример 3:

```
<section itemscope itemtype="http://schema.org/Organization">
  <h1>Компания «Прауд»</h1>
  <p>Адрес: <span itemprop="address">г.Мотино, улица Строителей, дом
6</span></p>
  <p>Телефон: 8 (8765) 333-00-00</p>
  <p>Email: info@proud-company.test</p>
</section>
```

По аналогии с адресом разметим остальные данные: название, телефон, email. Для каждого из них есть своё значение атрибута **itemprop**. Обратите внимание, что размечаются только данные, оставляя за «бортом» название поля. Для этого нужные данные оборачивают в `<span>` и для него указывают атрибут.

Пример 4:

```
<section itemscope itemtype="http://schema.org/Organization">
  <h1 itemprop="name">Компания «Прауд»</h1>
  <p>Адрес: <span itemprop="address">г.Мотино, улица Строителей, дом
6</span></p>
  <p>Телефон: <span itemprop="telephone">8 (8765) 333-00-00</span></p>
  <p>Email: <span itemprop="email">info@proud-company.test</span></p>
</section>
```

Для проверки корректности микроразметки можно пользоваться инструментами популярных поисковых систем:

- Google
- Yandex

Попробуйте вставить тестовый пример в эти инструменты и взгляните на результат. Можете попробовать «сломать» микроразметку и посмотреть на вывод ошибок.

2. Текст

2.1 Форматирование текста

Ряд элементов html предназначены для форматирования текстового содержимого, например, для выделения жирным или курсивом и т.д. Элементы могут иметь физическое значение или логическое значение соответственно. Ниже приведена таблица с такими элементами.

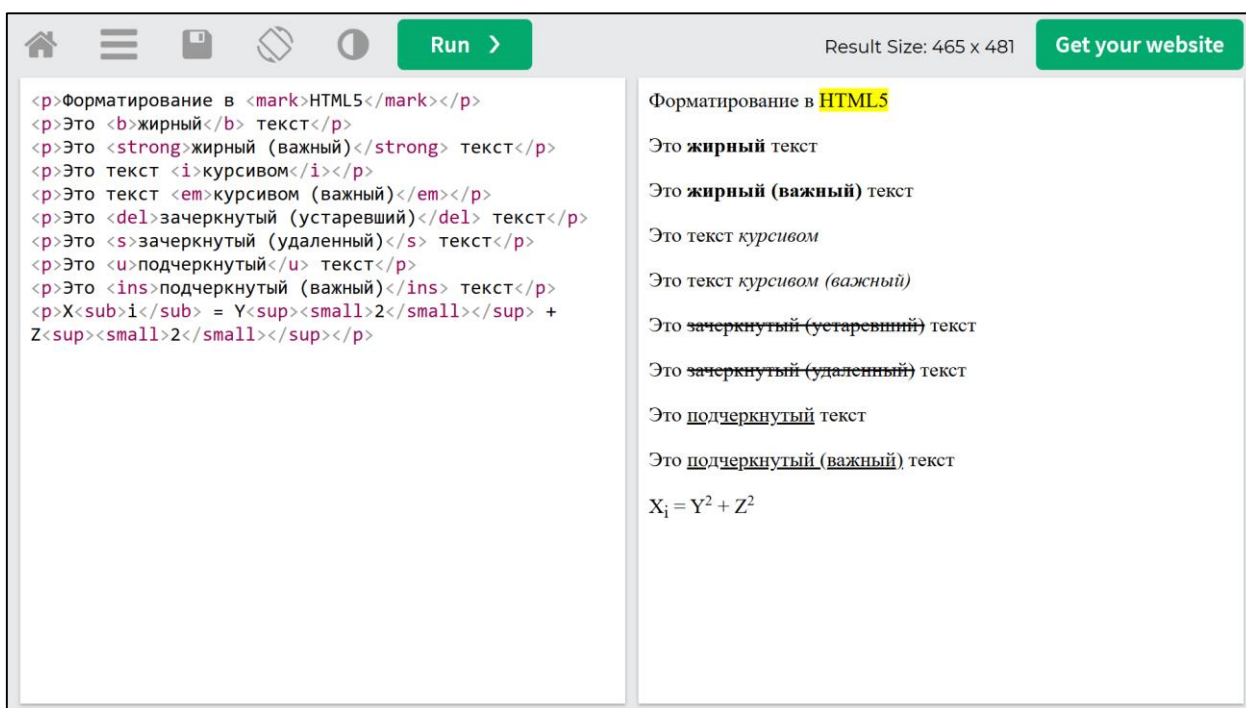
Тэг	Что делает
	Жирный
	Жирный (логическое значение придает оттенок важности)
<i>	Курсив
	Курсив (логическое значение придает оттенок важности)
<s>	Зачеркнутый (устаревший текст)
	Зачеркнутый (удаленный текст)
<u>	Подчеркнутый
<ins>	Подчеркнутый (логическое значение придает оттенок важности)
<sub>	помещает текст под строкой
<sup>	помещает текст над строкой
<small>	Уменьшает текст
<mark>	Выделяет текст цветом (логическое значение придает оттенок важности)

Полезное:

- Теги для визуального выделения короче, чем для смыслового. Это поможет запомнить, когда использовать <i> и , а когда и ;
- В практике вместо <i> и используют стилизацию с помощью CSS.

Пример 1:

```
<p>Форматирование в <mark>HTML5</mark></p>
<p>Это <b>жирный</b> текст</p>
<p>Это <strong>жирный (важный)</strong> текст</p>
<p>Это текст <i>курсивом</i></p>
<p>Это текст <em>курсивом (важный)</em></p>
<p>Это <del>зачеркнутый (устаревший)</del> текст</p>
<p>Это <s>зачеркнутый (удаленный)</s> текст</p>
<p>Это <u>подчеркнутый</u> текст</p>
<p>Это <ins>подчеркнутый (важный)</ins> текст</p>
<p>X<sub>i</sub> = Y<sup><small>2</small></sup> +
Z<sup><small>2</small></sup></p>
```

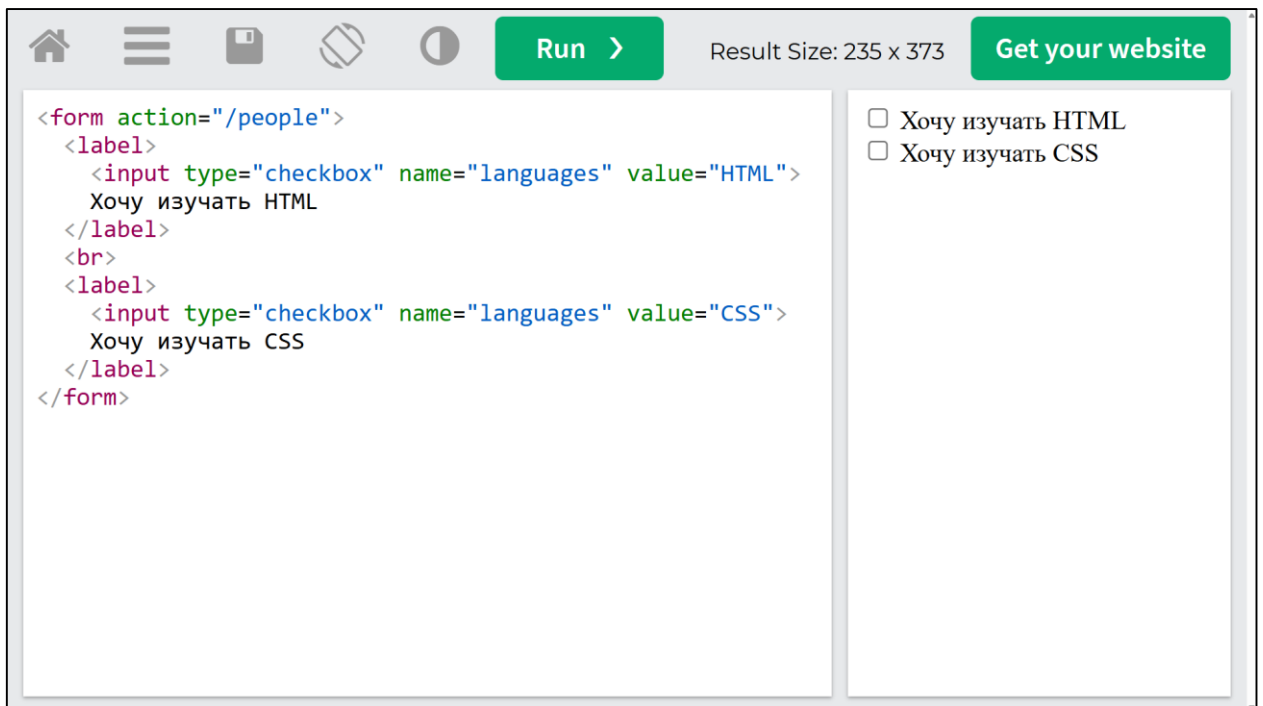


2.2 Перенос строки

Для переноса строки необходимо использовать тег `<br>` (от английского *break*). Он добавит разрыв строки в тексте, послужив аналогом клавиши Enter. Данному тегу не нужен парный элемент, так как он является самозакрывающимся. В примере ниже показано его применение (конструкция чекбокса будет рассмотрена в последующем разделе).

Пример 1:

```
<form action="/people">
  <label>
    <input type="checkbox" name="languages" value="HTML">
    Хочу изучать HTML
  </label>
  <br>
  <label>
    <input type="checkbox" name="languages" value="CSS">
    Хочу изучать CSS
  </label>
</form>
```

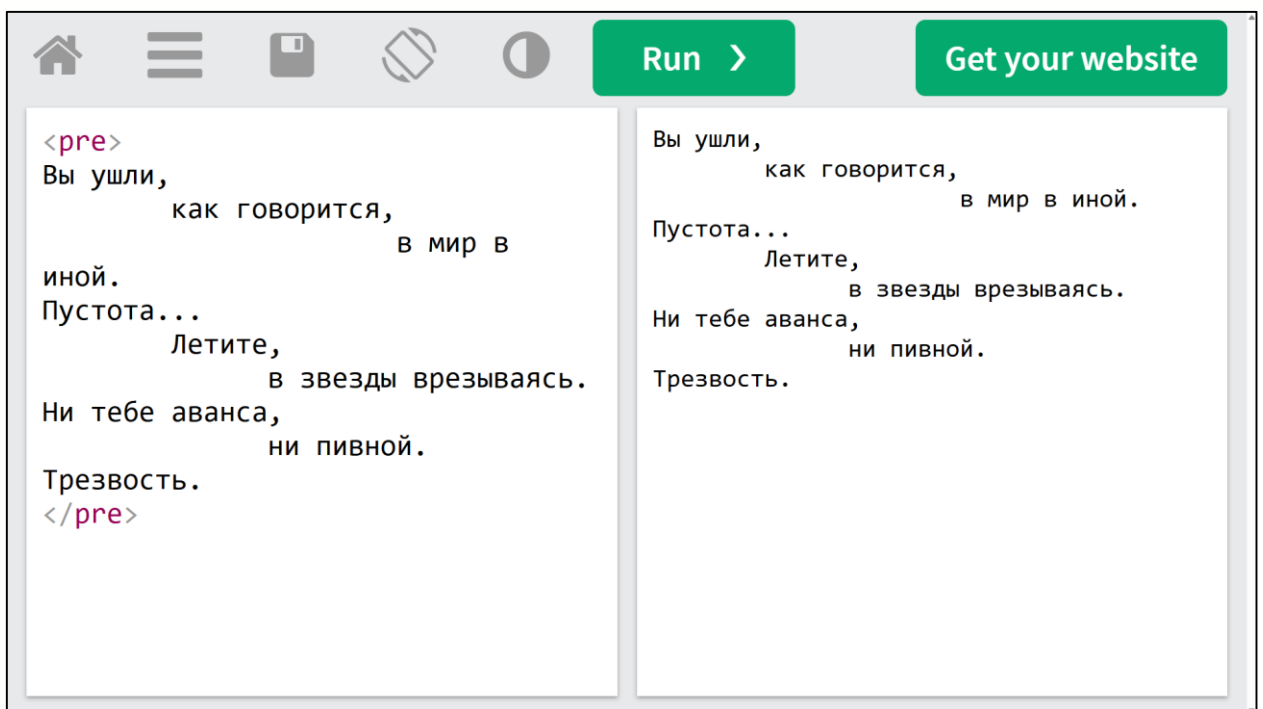


2.3 Сохранения оригинального форматирования текста

Иногда возникает потребность вставить текст, сохраняя его оригинальное форматирование. В таком случае проставлять пробелы, используя спец. символы будет очень сложно. Поэтому существует специальный тег `<pre>`, задача которого — вывести текст внутри себя с сохранением всех пробельных символов. По умолчанию тег `<pre>` выводит текст моноширинным шрифтом, в котором все символы имеют одинаковую ширину, в отличие от обычных шрифтов. Внутри тега `pre` можно использовать теги для оформления текста: `<i>`, `<em>`, `<b>`, `<strong>`.

Пример 1:

```
<pre>
Вы ушли,
    как говорится,
                                в мир в иной.
Пустота...
    Летите,
        в звезды врезываясь.
Ни тебе аванса,
        ни пивной.
Трезвость.
</pre>
```



2.4 Спец. Символы

В тексте возможно использование специальных символов — мнемоник. Они позволяют использовать знаки, которые присутствуют в таблице символов, но отсутствуют на клавиатуре. Они начинаются с амперсанда (&) и заканчиваются точкой с запятой (;).

Имя	Вид	Описание
 		неразрывный пробел
« и »	« »	«кавычки-ёлочки»
"	"	двойные кавычки
'	'	апостроф
—	—	длинное тире
< и >	< >	знак больше и знак меньше
€	€	знак евро
₽	₽	знак рубля
§	§	параграф
©	©	знак copyright
®	®	знак зарегистрированной торговой марки
™	™	знак торговой марки
°	°	градус

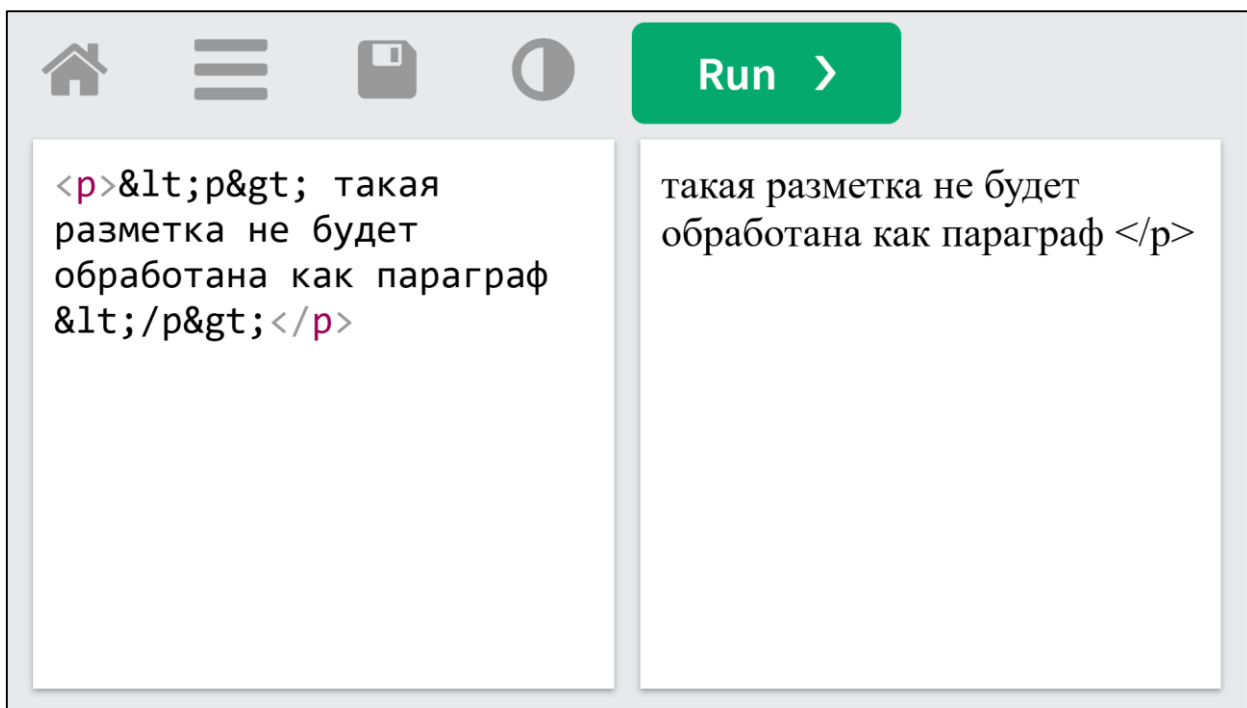
Со многими из этих символов можно ознакомиться на [«Википедии»](#).

Внутри значений атрибутов не разрешается применять те же кавычки, в которых взято само значение. Но допустимо сочетать разные типы кавычек между собой (также возможно применение спец. символов). Если внутри текста необходимы одинарные кавычки или апостроф, то сам текст следует взять в двойные кавычки.

Мнемоники распространены, когда необходимо вставить теги без их обработки браузером. В таком случае можно воспользоваться мнемониками < и >.

Пример 1:

`<p><p> такая разметка не будет обработана как параграф
</p></p>`

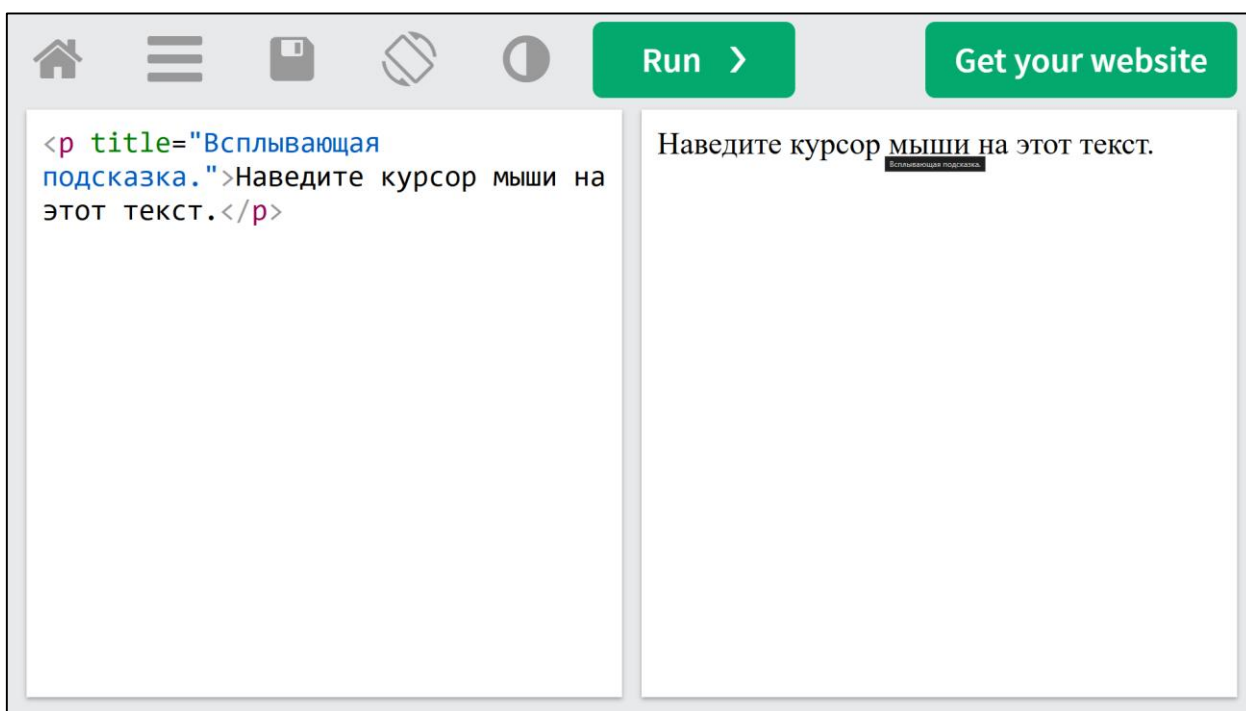


2.5 Всплывающая подсказка

При наведении курсором мыши на элемент страницы в браузере можно вывести дополнительную информацию, это и будет называться всплывающей подсказкой (также его называют тултип). Такое поведение задаётся с помощью атрибутов — специальных конструкций, которые могут влиять на вывод информации на странице. Каждый тег в HTML имеет несколько стандартных атрибутов, которые записываются по шаблону атрибут="значение". Для появления всплывающей подсказки использовался атрибут title. Значение атрибута бывает разным, оно зависит от конкретного примера. Именно значение выводится при наведении на элемент.

Пример 1:

```
<p title="Всплывающая подсказка.">Наведите курсор мыши на этот текст.</p>
```

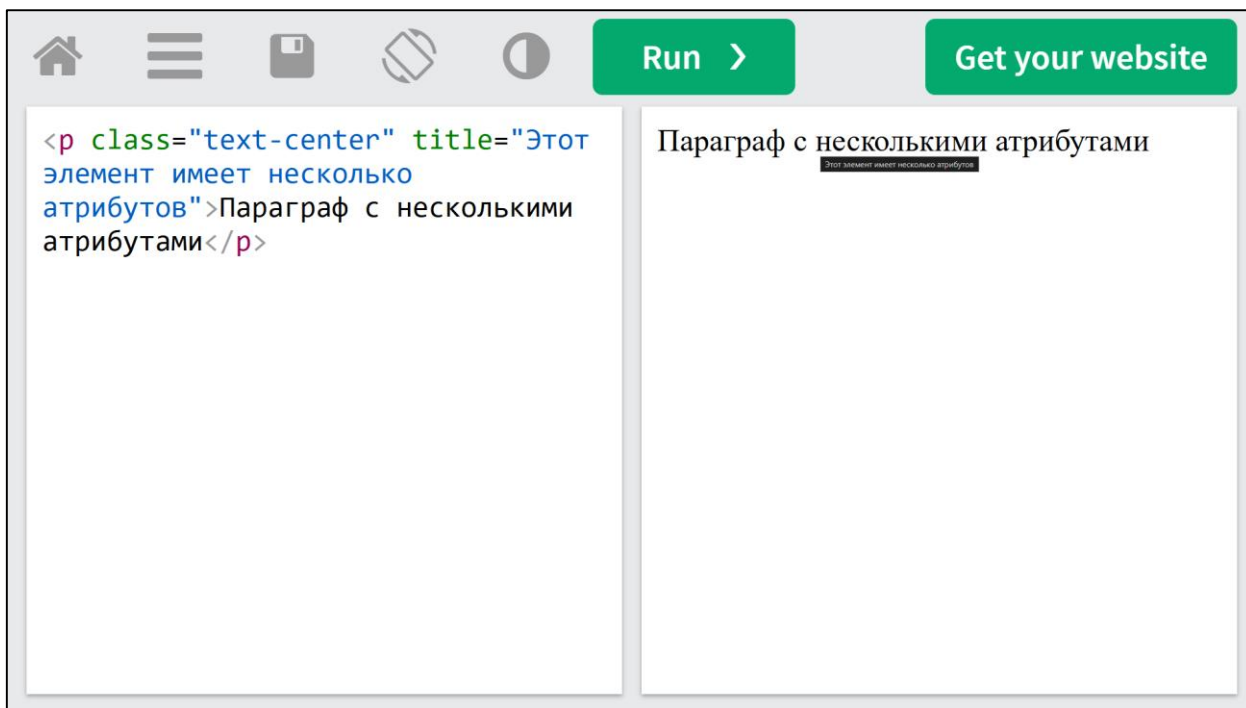


Атрибут title можно использовать не только на параграфах, а на любом элементе страницы. Такие атрибуты называются глобальными, так как не зависят от элемента. В противовес им есть атрибуты, которые уникальны для одного или нескольких тегов. Например, атрибут colspan, который указывает на объединение ячеек в таблицах. Его использование вне таблиц не имеет никакого смысла.

У одного тега может быть указано сразу несколько атрибутов. Тогда они записываются последовательно, при этом порядок указания не имеет значения.

Пример 2:

```
<p class="text-center" title="Этот элемент имеет несколько атрибутов">Параграф с несколькими атрибутами</p>
```



2.6 Вставка кода в HTML

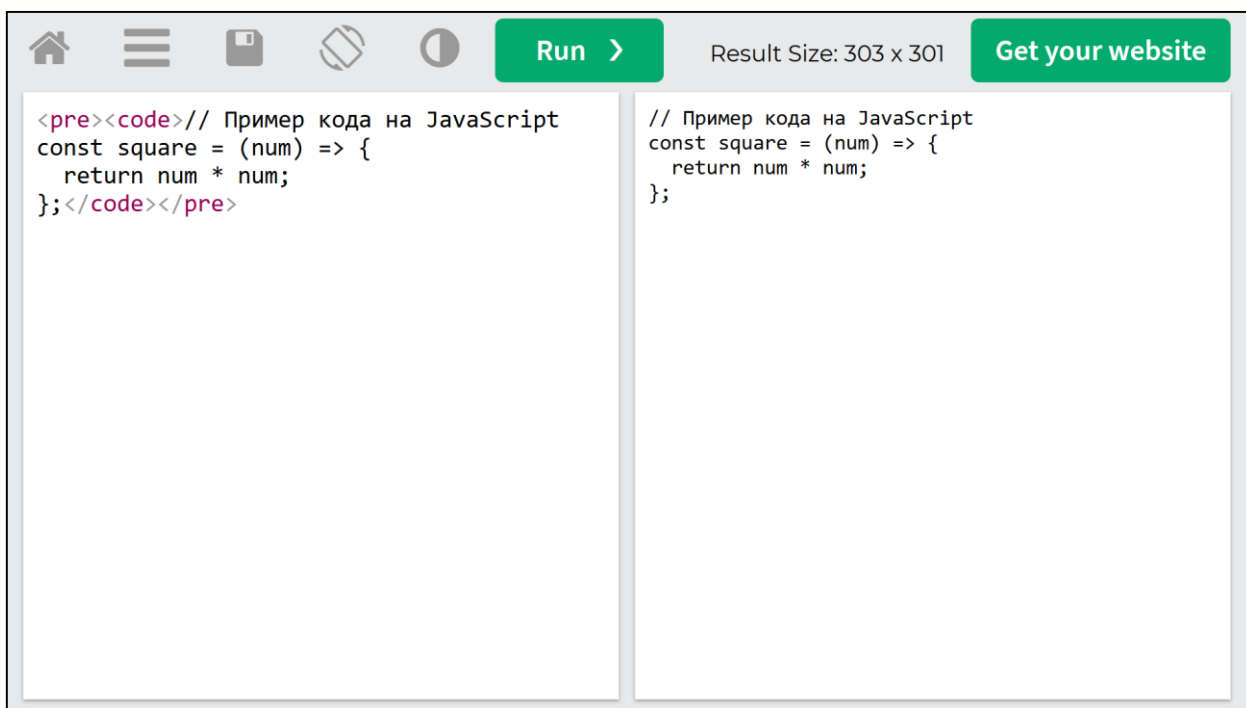
При ведении учебных блогов по программированию, либо при написании статей, возникает необходимость продемонстрировать код.

Для создания блока с кодом используется тег `<code>`, внутри которого записывается код на любом языке программирования.

Тег `<code>` часто используют вместе с тегом `<pre>`, что позволяет сохранить форматирование. Если тег `<pre>` не используется, то переносы строк нужно контролировать самостоятельно, например, оборачивая каждую строку в тег `<p>`.

Пример 1:

```
<pre><code>// Пример кода на JavaScript
const square = (num) => {
  return num * num;
};</code></pre>
```



Важно, что все теги внутри `<code>` будут обработаны браузером, по этой причине внутри используются специальные символы. Также частое решение — оборачивание названия тега в отдельный тег, например, `<span>`.

3. Ссылки

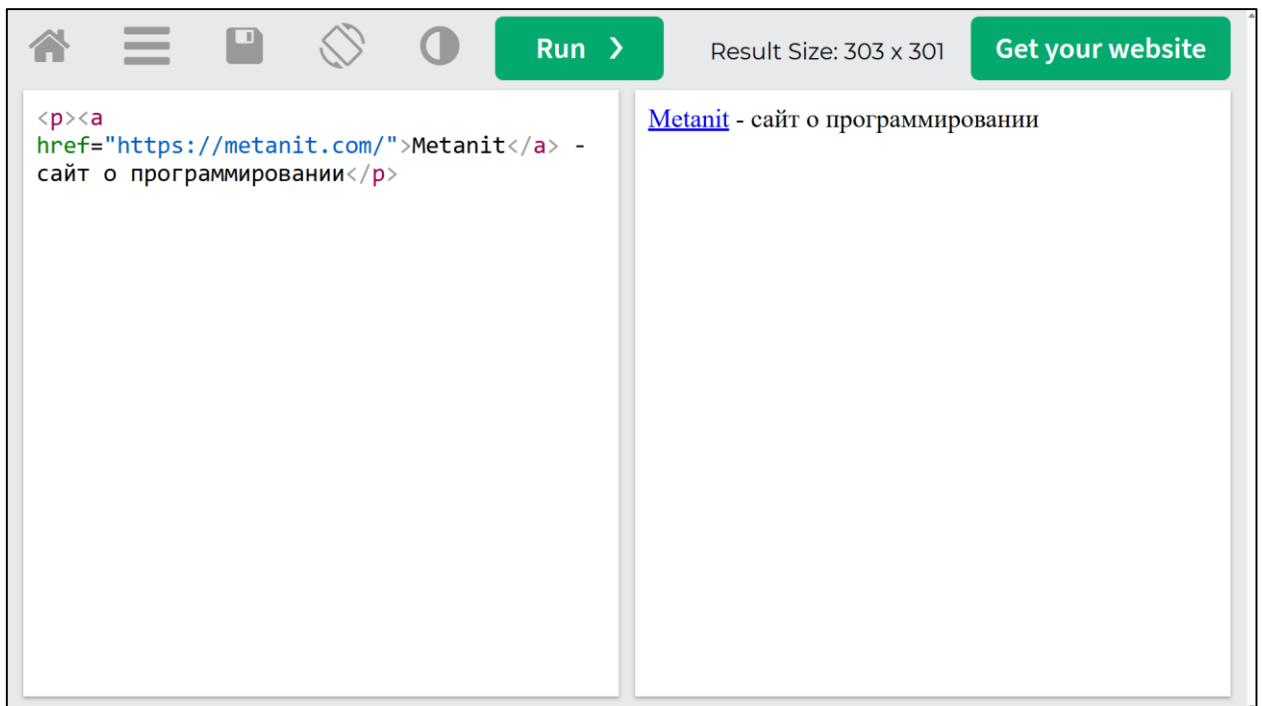
Для перемещения между страницами был придуман главный элемент в интернете — гиперссылки.

Гиперссылки могут ссылаться не только на другой ресурс в интернете, но и позволяют перемещаться внутри текущего документа.

Для создания ссылки используется парный тег `<a>` с обязательным атрибутом `href` который указывает адрес страницы, на которую произойдёт переход после клика. Чтобы ссылка отправляла нас на новую вкладку, нужно в первом теге `<a>` после `href` дописать `target="_blank"`.

Пример 1:

```
<p><a href="https://metanit.com/">Metanit</a> - сайт о  
программировании</p>
```



Ссылки можно поделить на несколько типов:

- **Относительные**. Такие ссылки ссылаются на тот же сайт, с которого и происходит переход
- **Абсолютные**. Позволяют переходить на сторонние ресурсы в интернете
- **Ссылки-якоря**. Используются для перемещения внутри текущего документа
- **Ссылки на файл**. По ссылкам можно не только переходить на другие сайты, но и скачивать файлы.

- **Ссылки на электронную почту.** При нажатии на такую ссылку откроется почтовая программа, заданная по умолчанию для обработки ссылок-mailto.
- **Ссылки на телефон.** При нажатии на такую ссылку будет предложено перейти в специальное приложение для звонка, а с телефона звонок начнется сразу.
- **Пустые ссылки.** Пустые ссылки делают, когда страницы ещё нет, а ссылка на неё уже есть.

3.1 Относительные ссылки

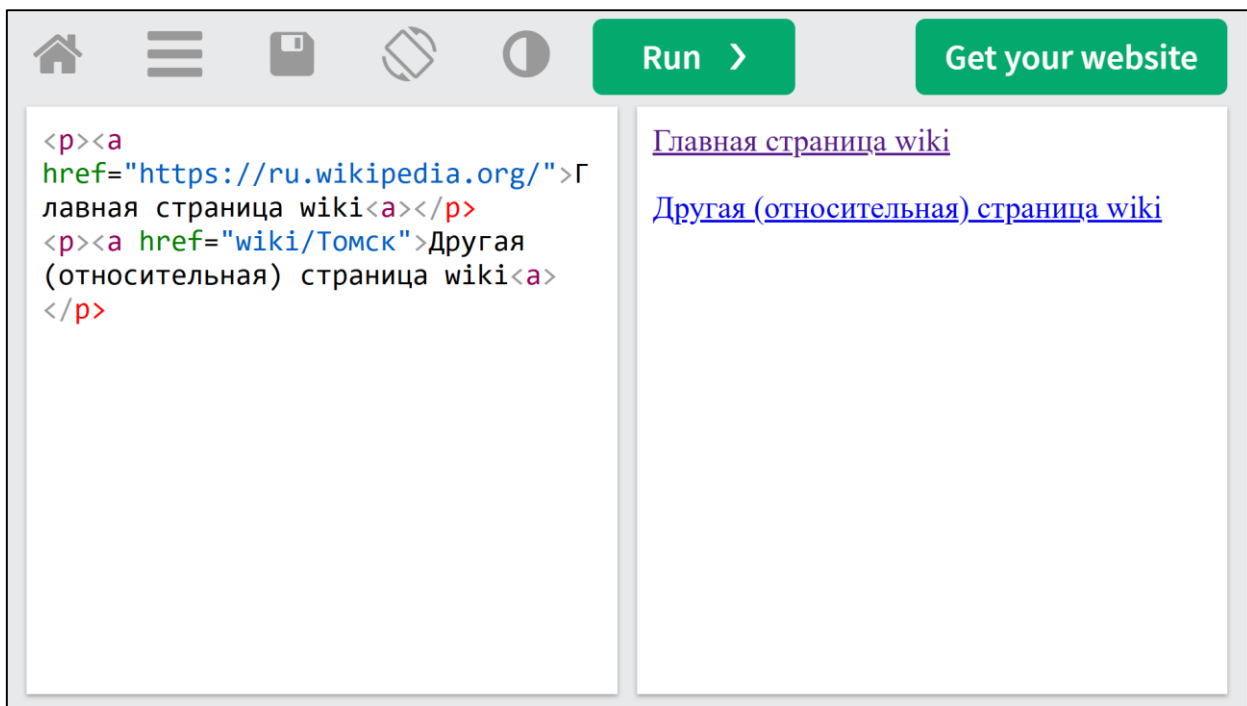
Когда файл по ссылке лежит на вашем компьютере, используются относительные адреса. В отличие от «обычных» адресов, в них нет адреса сайта.

Относительные адреса иногда используют и для страниц в интернете. Если выложить два каких-то файла в интернет (не меняя их взаимное расположение), то их ссылка друг на друга всё равно будет работать.

Если в адресе нет имени сервера или протокола, то это относительный адрес.

Пример 1:

```
<p><a href="https://ru.wikipedia.org/">Главная страница wiki<a></p>  
<p><a href="wiki/Томск">Другая (относительная) страница wiki<a></p>
```



3.2 Абсолютные ссылки

Если нужно сделать ссылку на другой сайт в интернете, используйте полный адрес сайта — он называется абсолютным.

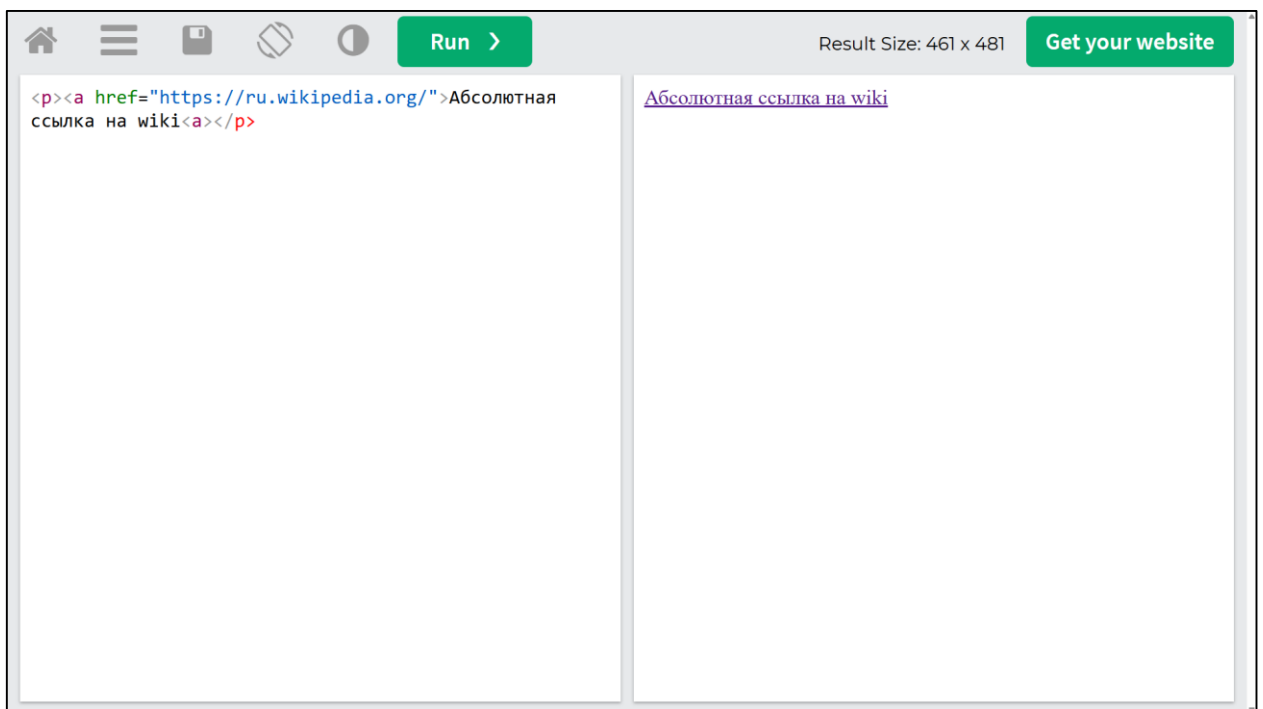
Абсолютные адреса содержат минимум три части: протокол, имя сервера и путь. В нашем случае это:

- `https://` — протокол, по которому мы обращаемся к сайту.
- `site.ru/` — имя сервера.
- `/blog/index.html` — путь к файлу.

Если в адресе есть все эти части — это абсолютный адрес.

Пример 1:

```
<p><a href="https://ru.wikipedia.org/">Абсолютная ссылка на  
wiki<a></p>
```

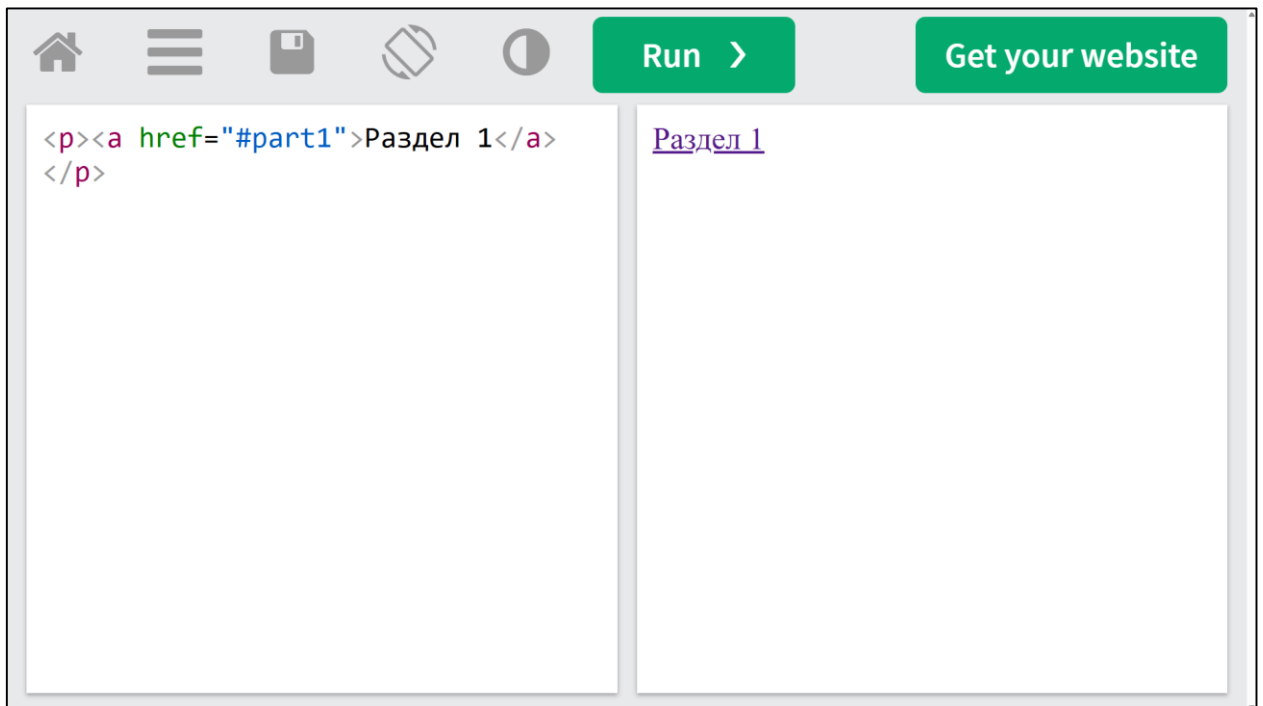


3.3 Ссылки-якоря

Ссылка-якорь — это обычная ссылка, в адресе которой используется символ #, после которого следует идентификатор элемента. Идентификатор создаётся с помощью атрибута id у того тега, к которому надо перейти при щелчке по ссылке.

Пример 1:

```
<p><a href="#part1">Раздел 1</a></p>
```



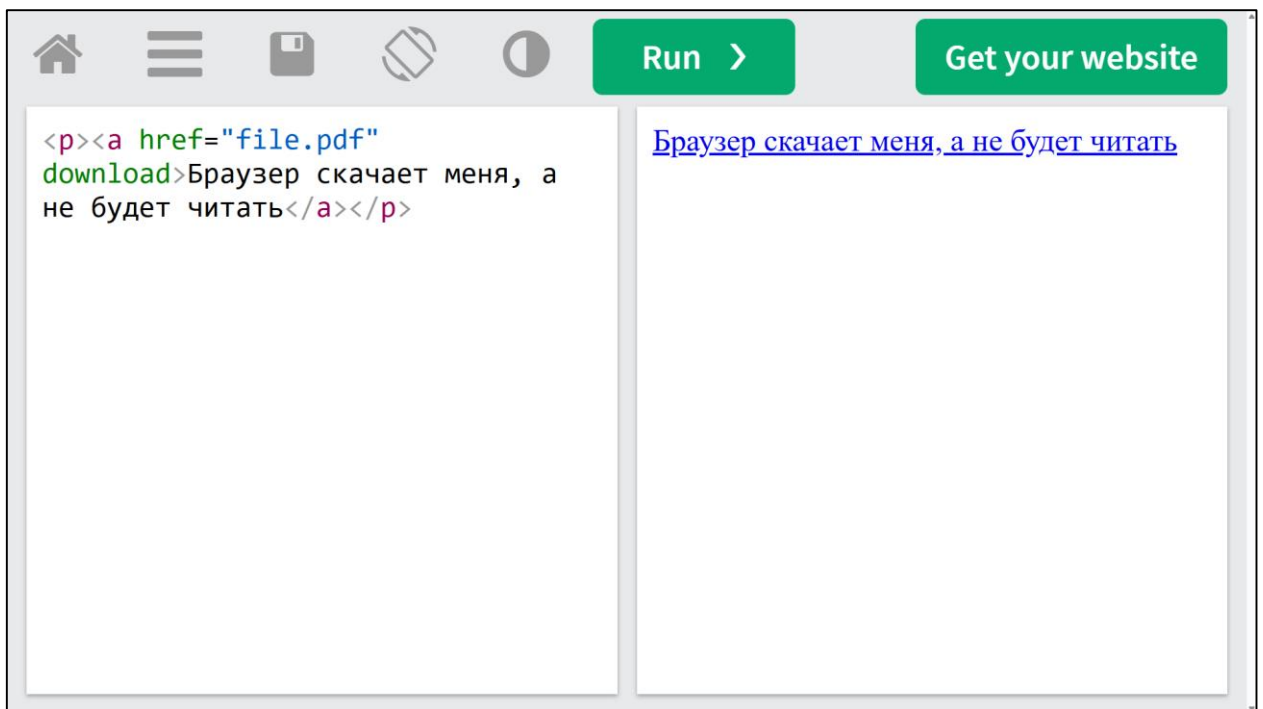
Ссылка-якорь используется для прокрутки к заданной части страницы, в том числе используется и в абсолютных адресах.

3.4 Ссылки на файл

По ссылкам можно не только переходить на другие сайты, но и скачивать файлы. Для этого пропишите в атрибуте href ссылку на этот файл. А чтобы предотвратить открытие файлов прямо в браузере, добавьте атрибут download у тега <a>.

Пример 1:

```
<p><a href="file.pdf" download>Браузер скачает меня, а не будет читать</a></p>
```

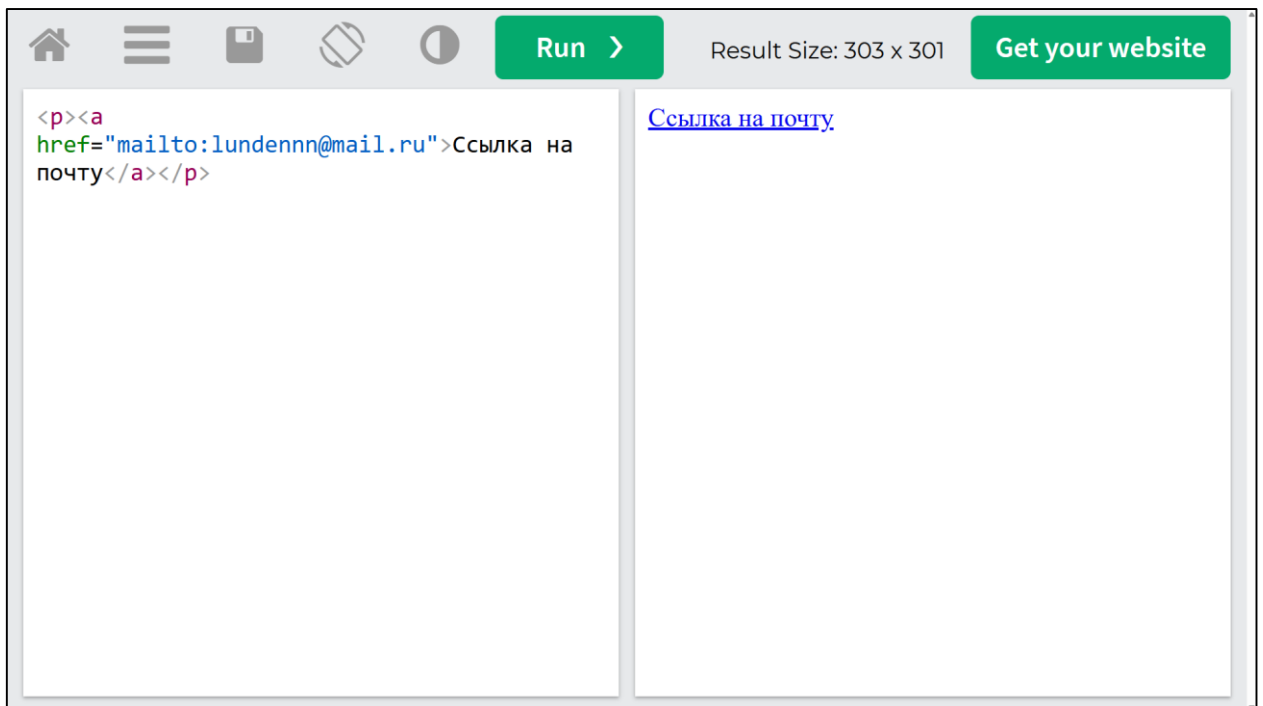


3.5 Ссылки на электронную почту

Адрес электронной почты пишется в формате lundennn@mail.ru, где lundennn — это имя пользователя, а mail.ru — имя домена. Для создания ссылки также используется элемент `<a>`, но в атрибуте href сперва пишется ключевое слово `mailto:`, затем без пробела указывается адрес электронной почты пользователя. Если получателей несколько, то они указываются через запятую.

Пример 1:

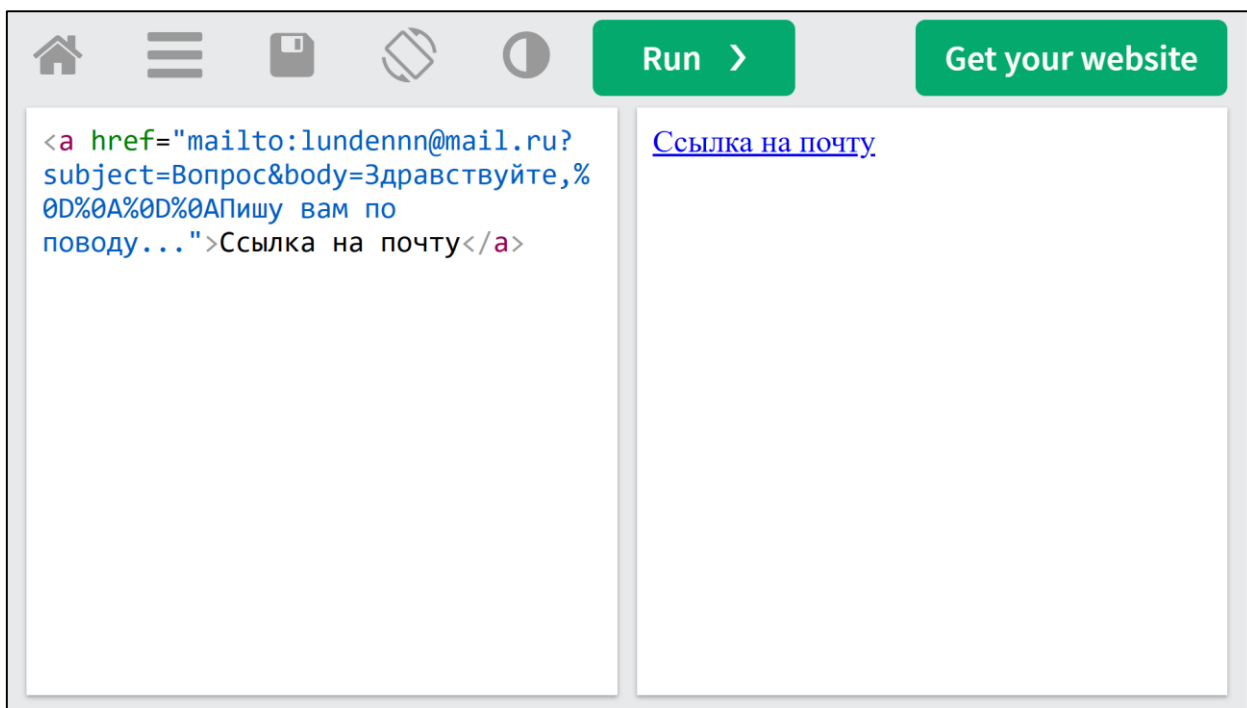
```
<p><a href="mailto:lundennn@mail.ru">Ссылка на почту</a></p>
```



При щелчке по такой ссылке открывается почтовая программа по умолчанию. Если подобные ссылки ещё не открывались, то операционная система выведет окно со списком программ.

Пример 2:

```
<a href="mailto:lundennn@mail.ru?subject=Вопрос&body=Здравствуйете,%0D%0A%0D%0AПишу вам по поводу...">Ссылка на почту</a>
```



В данном примере subject отвечает за заполнение темы электронного сообщения, body заполняет текст письма, а %0D%0A отвечает за перенос строки. Важно обратить внимание, что пишется это в поле href и разделяется с почтой получателя знаком «?», а между собой с помощью «&». Таким образом можно легко создать шаблон для письма или же ссылку с перезаполненными полями.

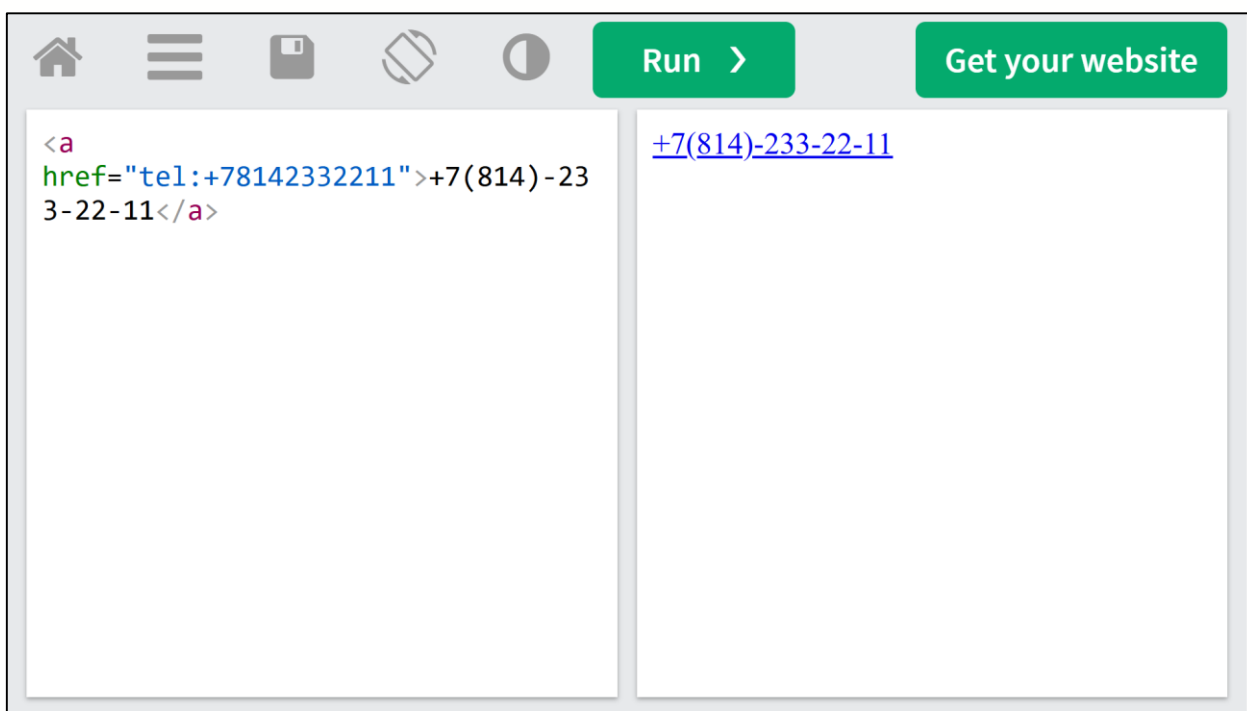
Также ссылка может иметь расширенные параметры такие как cc – копия письма или bcc – скрытая копия, в таком случае пробелы заменяются на %20. Не исключено добавление [всплывающей подсказки](#), при наведении на текст ссылки.

3.6 Ссылки на телефон

С каждым годом стремительно растет количество интернет-пользователей, использующих мобильные платформы. И чтобы не потерять своего клиента, необходимо предоставить посетителю возможность позвонить прямо сейчас. Для этого и используют ссылки на телефон в подвале сайта (футер).

Пример 1:

```
<a href="tel:+78142332211">+7(814)-233-22-11</a>
```



Важно указать номер в ссылке без пробелов.

3.7 Пустые ссылки

Пустые ссылки делают, когда страницы ещё нет, а ссылка на неё уже есть. Например, в меню может быть ссылка на каталог, который ещё не начали верстать. Второй случай использования — когда нужно что-то показать на странице, но без перезагрузки страницы. Например, при создании модальных окон.

Есть несколько способов указывать пустые ссылки. Рассмотрим, чем грозит использование того или иного способа.

Пример 1:

```
<a href="">Каталог</a>
```

Один из самых худших вариантов, так как полностью перезагружает страницу. Скорее всего, с целью борьбы с ним и были созданы остальные способы создания пустых ссылок.

Пример 2:

```
<a href="#">Каталог</a>
```

Изменяет адрес страницы, добавляет в конце #. Если было metanit.com/, то после клика станет metanit.com/#. Прокручивает страницу к самому верху сайта. Если ссылка была в футере (подвал сайта), то сайт прокрутится до самого начала. Это не всегда желательное поведение, поэтому этот вариант тоже отбросим.

Пример 3:

```
<a href="##">Каталог</a>
```

Или

```
<a href="#!">Каталог</a>
```

Эта версия лучше, чем предыдущий способ, так как страница не прокручивается вверх. При этом меняется адрес страницы, а также добавляется # в конце адреса. Этот способ работает не во всех браузерах, поэтому есть другой.

Пример 4:

```
<a href="#no_scroll">Каталог</a>
```

Можно сделать полноценную якорную ссылку, но при этом без элементов с `id="noscroll"`. Этот способ работает во всех браузерах, не скроллит страницу к началу, но меняет ссылку, добавляя `#noscroll` в конец адреса.

Пример 5:

```
<a href="javascript:void(0);">Каталог</a>
```

Достаточно старый способ, который благодаря JavaScript буквально просит ссылку ничего не делать. Правильнее было бы написать `javascript:undefined;`, но этот вариант длиннее, поэтому придумали более короткую замену `void (0)`, который также возвращает `undefined`.

Пример 6:

```
<a href="bla-bla-bla" onclick="return false;">Ссылка</a>
```

В этом методе ничто никуда не прокручивается и не прыгает. В `href` можно добавить что угодно, а в JavaScript не добавлять новый `onclick`.

4. Изображения

Для лучшего восприятия текста на странице используют изображения. Они помогают ориентироваться в тексте, разбивать его на составные части, а также создают настроение у читателя.

Картинки на странице не существуют сами по себе. Предварительно они должны быть загружены в директорию на сервере или на специальный хостинг картинок. Для вставки картинки на страницу используется непарный тег `<img>`, у которого два обязательных атрибута: `src` и `alt`.

4.1 Атрибуты изображений

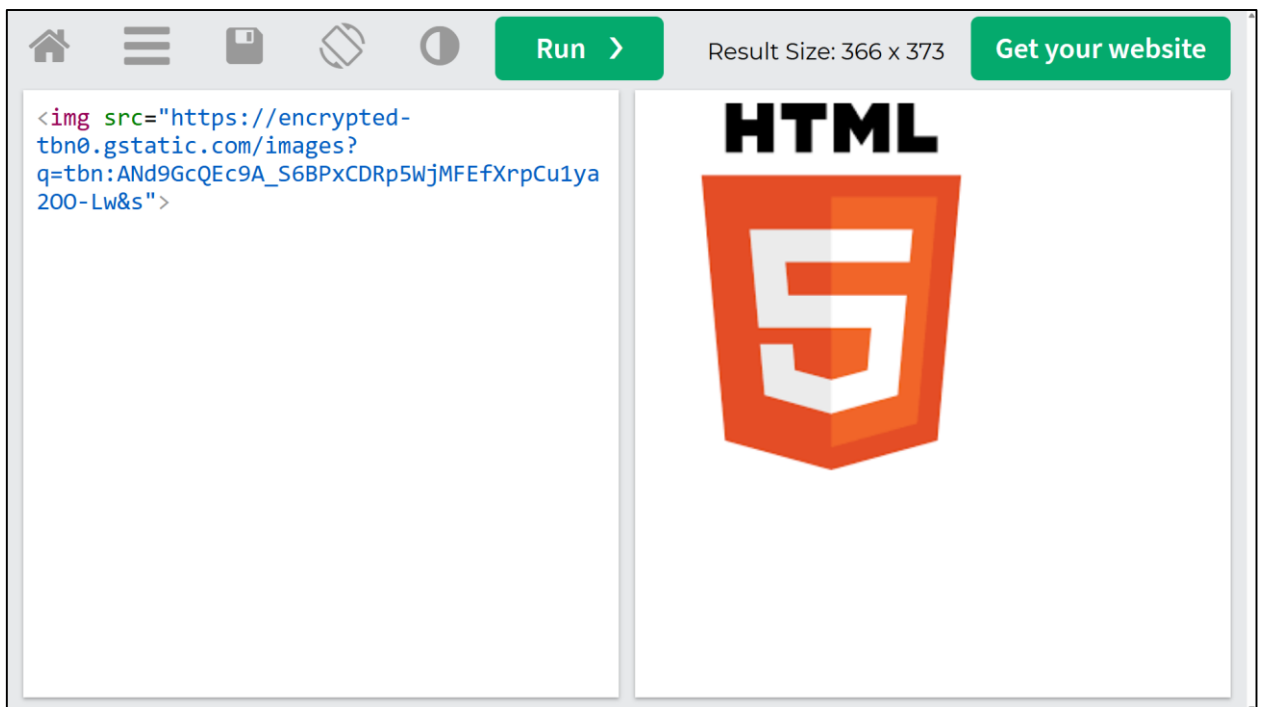
4.1.1 Атрибут `src`

В атрибуте `src` указывается путь к изображению. Это тоже ссылка, как и в теге `<a>`, только теперь не происходит перенаправление пользователя, а браузер загружает изображение по этому пути, если оно существует. Путь к картинке может быть как относительный, так и абсолютный.

Пример 1:

```

```



4.1.2 Атрибут alt

Что делать, если картинка в данный момент недоступна? В этом случае браузеры показывают маленькую картинку, обозначающую, что изображение недоступно.

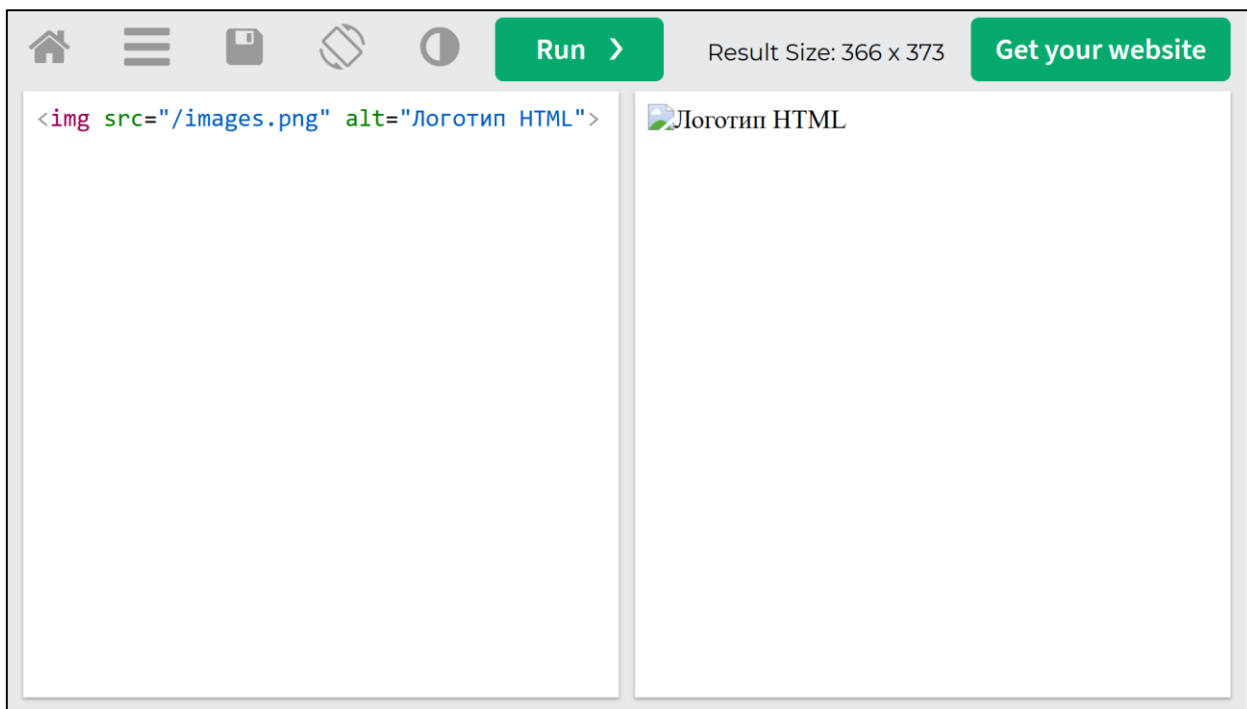
Но как пользователь поймёт, что здесь было? Может изображение критически важно для статьи, и без него теряется смысл написанного? Можно посоветовать пользователю зайти попозже, когда изображение будет доступно, а можно описать ему, что было изображено.

Для этого используется атрибут alt, значением которого является текст. Он будет показан при невозможности загрузить изображение.

Пример 1:

```

```



Помимо функции «рассказать то, что не удалось показать» атрибут alt выполняет важнейшую роль для людей, которые пользуются скринридерами — программами для чтения информации с экрана. Так как изображение невозможно прочитать, то программы читают то, что было указано в атрибуте alt. Именно поэтому в данном атрибуте не следует указывать специальные символы, ведь это усложнит восприятие.

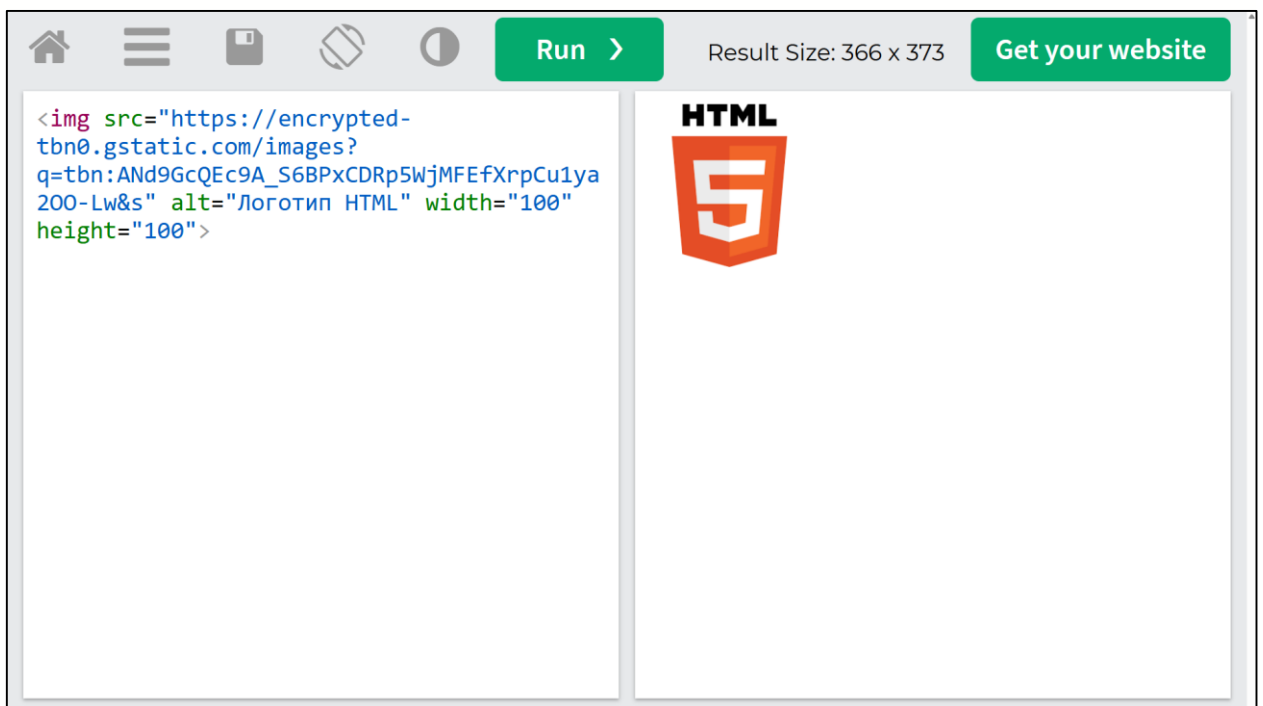
4.2 Размеры изображений

Важной составляющей при работе с изображениями являются его размеры. При скачивании картинки со стороннего ресурса и её вставки на страницу, часто возникает ситуация, что изображение очень большое, и его нужно уменьшить. Для этого существуют атрибуты `width` и `height`, которые позволяют установить ширину и высоту для картинки.

Пример 1:

```

```



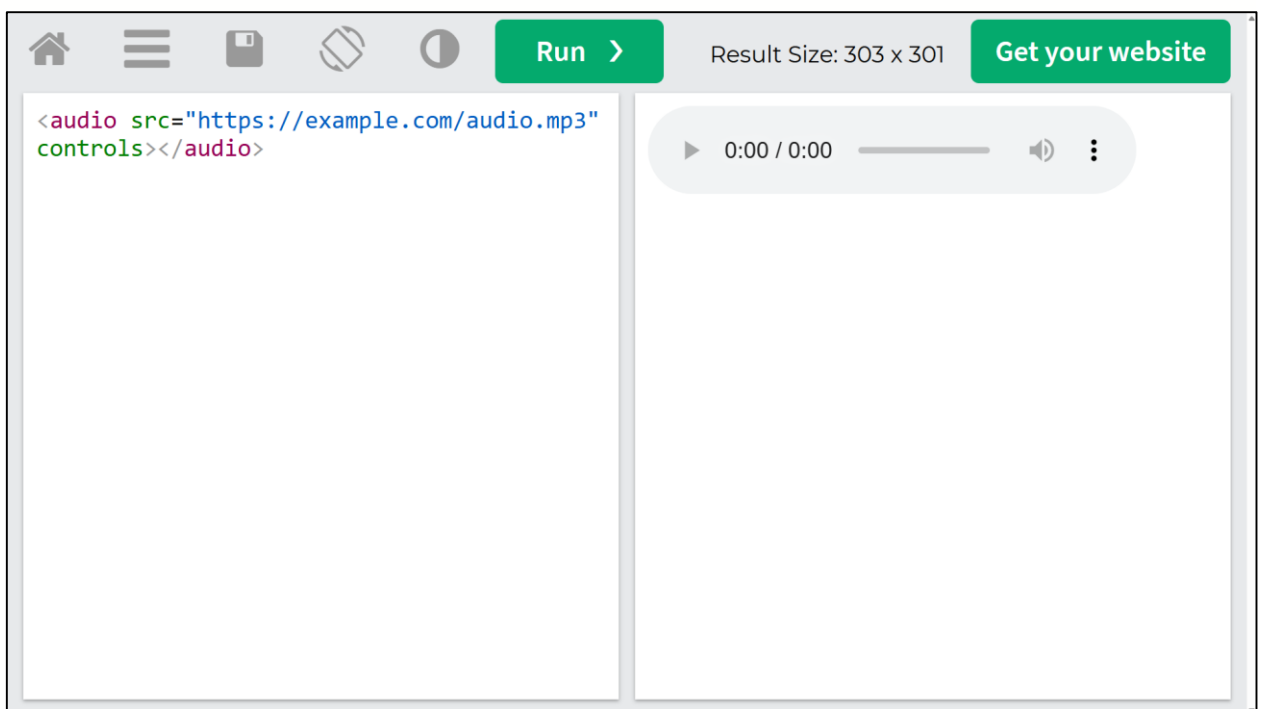
Важно помнить, что использование атрибутов `width` и `height` не меняет физический размер картинки — она будет весить столько же, изменится только визуальный размер. Для оптимизации страницы подбирайте картинки нужного размера или уменьшайте их с помощью графического редактора, либо специальных сервисов. Сейчас многие пользуются мобильными телефонами и мобильным интернетом, поэтому вес страницы имеет важное значение.

5. Аудио

Ранее, для добавления аудио на HTML-страницу, необходимо было пользоваться специальными «расширениями», которые давали возможность встраивать на страницу аудиоплеер. Начиная со стандарта HTML5, данная возможность есть по умолчанию. Она реализуется с помощью тега `<audio>`. Данный тег является парным. У данного тега также есть свой атрибут – `controls`, который добавляет элементы интерфейса для плеера. Каждый браузер реализует их по-своему, и единого стандарта на этот счёт нет.

Пример 1:

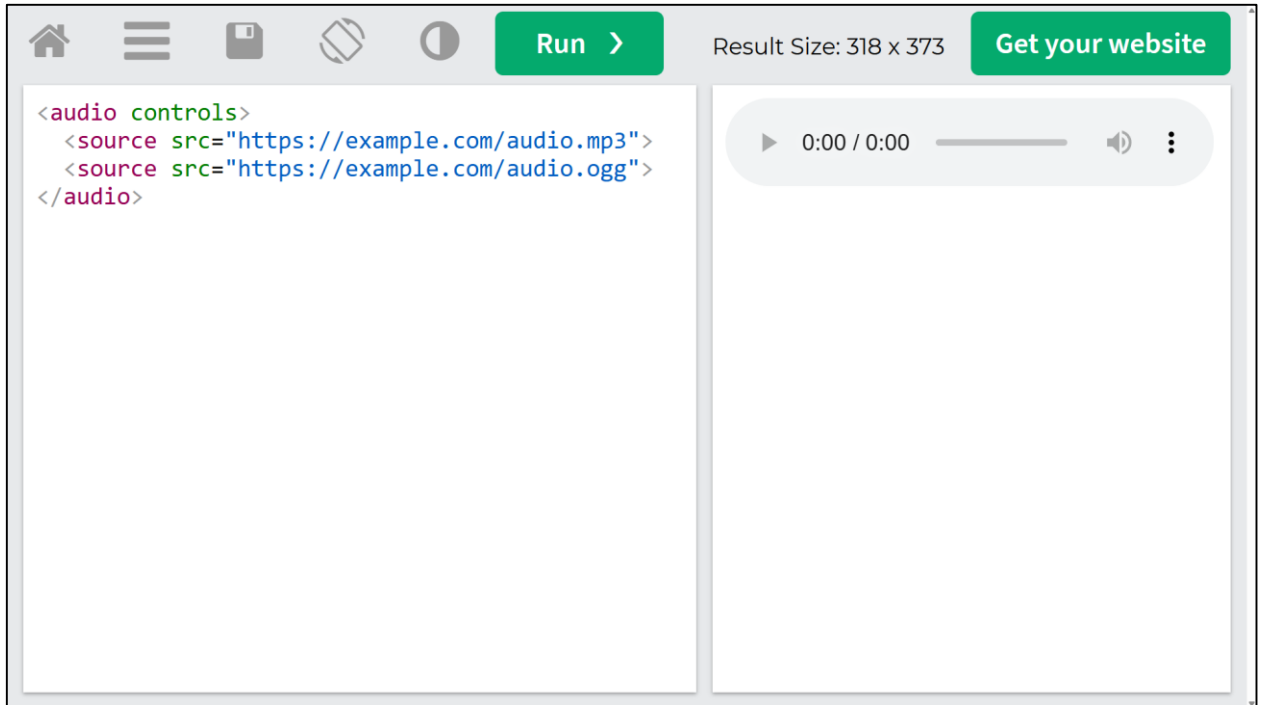
```
<audio src="https://example.com/audio.mp3" controls></audio>
```



При добавлении аудио на HTML-страницу важно помнить, что не все браузеры поддерживают одни и те же форматы аудио. Специально для этого существует вложенный тег `source` («источник»), в котором указываются пути на другие форматы аудио файлов. В таком случае браузер выберет тот, который поддерживается в настоящий момент. При такой разметке указывать `src` у тега `<audio>` необязательно.

Пример 2:

```
<audio controls>  
  <source src="https://example.com/audio.mp3">  
  <source src="https://example.com/audio.ogg">  
</audio>
```



5.1 Атрибуты аудио

Атрибут **autoplay** (логического типа). Если он указан, аудио начнёт автоматически воспроизводиться, как только сможет это сделать, не дожидаясь завершения загрузки всего файла. Но сайты, которые автоматически проигрывают аудио (или видео с аудиодорожкой) могут быть неприятными для пользователей, поэтому этого следует по возможности избегать. Такой атрибут можно задействовать, когда пользователь сам включил его. Прописывается без подробных указаний.

Атрибут **loop** (логического типа). Если он указан, проигрыватель будет автоматически возвращаться в начало при достижении конца аудио.

Атрибут **muted** (логического типа), который указывает, будет ли звук изначально отключён. Значением по умолчанию является false.

Атрибут **preload** предназначен для того, чтобы указать браузеру, что, по мнению автора, приведёт к лучшему взаимодействию с пользователем. Он может иметь одно из следующих значений:

- **none**: указывает, что аудио не должно предварительно загружаться;
- **metadata**: указывает, что предварительно загружаются только метаданные аудио (например, продолжительность аудио);
- **auto**: указывает, что весь файл аудио может быть предварительно загружен, даже если пользователь не предполагает его использовать;
- **пустая строка**: синоним значения auto. Значение по умолчанию отличается для каждого браузера. Спецификация рекомендует установить значение metadata.

Пример 1:

```
<audio controls preload="auto" muted autoplay loop>  
  <source src="https://www.w3schools.com/html/horse.ogg">  
  <source src="https://www.w3schools.com/html/horse.mp3">  
</audio>
```

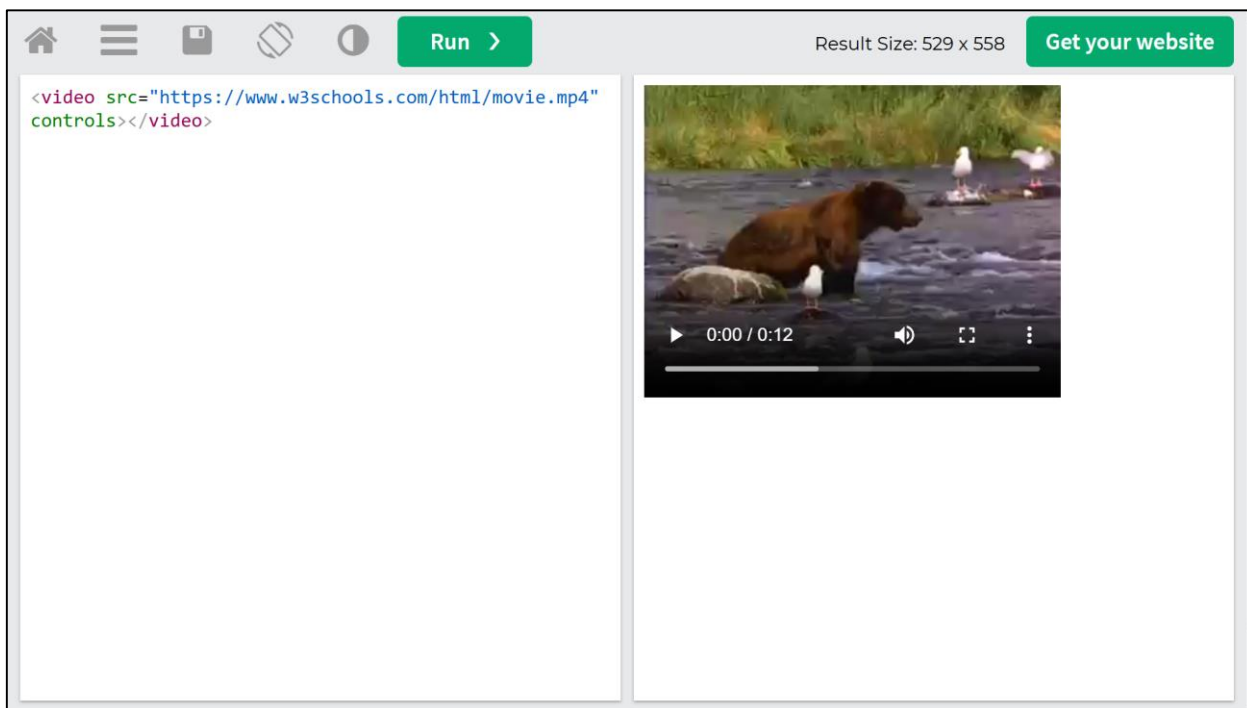
6. Видео

Как и с аудио, ещё недавно в HTML-разметку нельзя было «комфортно» вставить видеофайл. Начиная с HTML5 появился элемент `<video>`, который, как и `<audio>`, позволяет быстро добавить необходимое видео на страницу.

Тег `<video>` парный, в котором, при отсутствии вложенных тегов `<source>`, указывается путь к видеофайлу в атрибуте `src`. С помощью атрибута `controls` можно вывести элементы управления для проигрывателя. Визуальное оформление управляющих элементов зависит от конкретного браузера.

Пример 1:

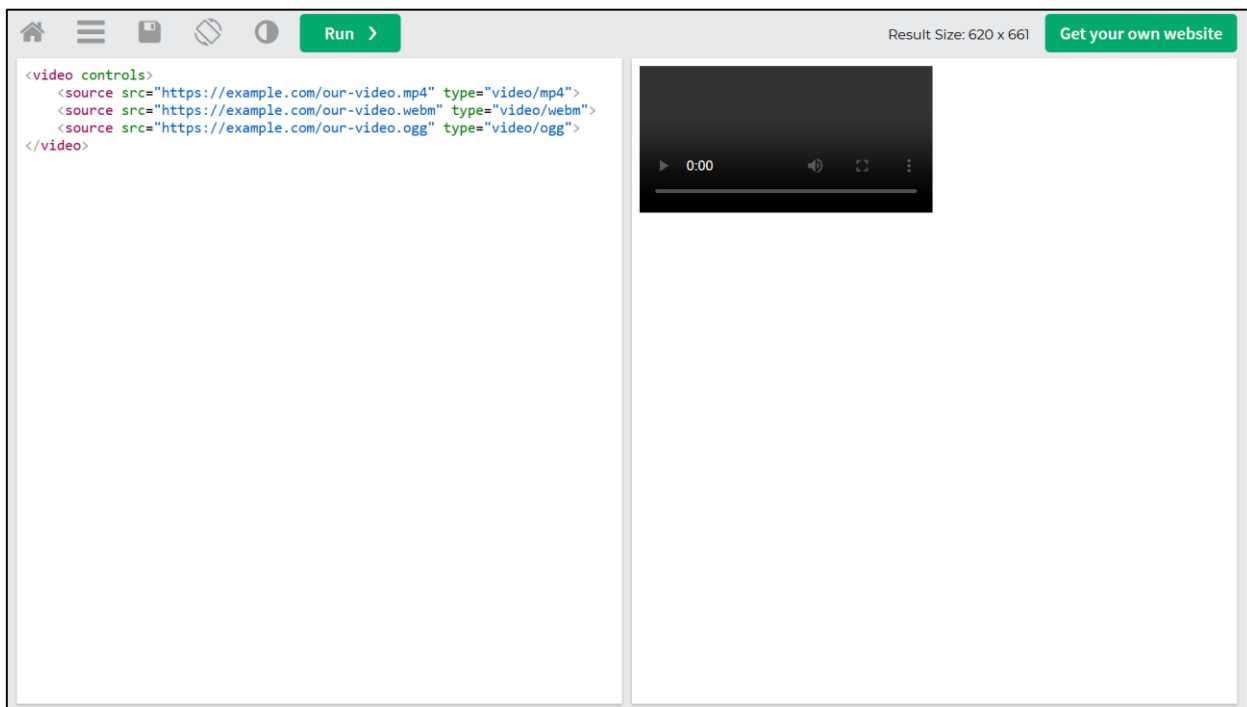
```
<video src="https://www.w3schools.com/html/movie.mp4"
controls></video>
```



С помощью специальных тегов `<source>` («источник») возможно добавлять несколько форматов видео. Это необходимо по причине того, что каждый браузер умеет воспроизводить только определённые форматы видео. Единственный формат, который корректно обрабатывают все браузеры — `mp4`.

Пример 2:

```
<video controls>
  <source src="https://example.com/our-video.mp4" type="video/mp4">
  <source src="https://example.com/our-video.webm"
type="video/webm">
  <source src="https://example.com/our-video.ogg" type="video/ogg">
</video>
```



Наличие трёх форматов файлов гарантирует работоспособность видео на любых устройствах. Также обязательным является наличие атрибута `type`, который сообщит браузеру формат видео. Именно по этому атрибуту браузер примет решение, какой файл необходимо загрузить.

6.1 Атрибуты видео

Видео файлы имеют точно такие же атрибуты, как и аудио ([src](#), [controls](#), [autoplay](#), [loop](#), [muted](#), [preload](#)), но помимо этого у них есть и другие, не характерные для аудио формата.

Атрибут	Описание
width/height	Задают размеры (ширину/высот) видео в (пикселях или процентах).
poster	Задает URL изображения-заставки, показываемого до загрузки видео.
playsinline	Используется для IOS (мобильных устройств), чтобы предотвратить автоматический полноэкранный режим при воспроизведении.

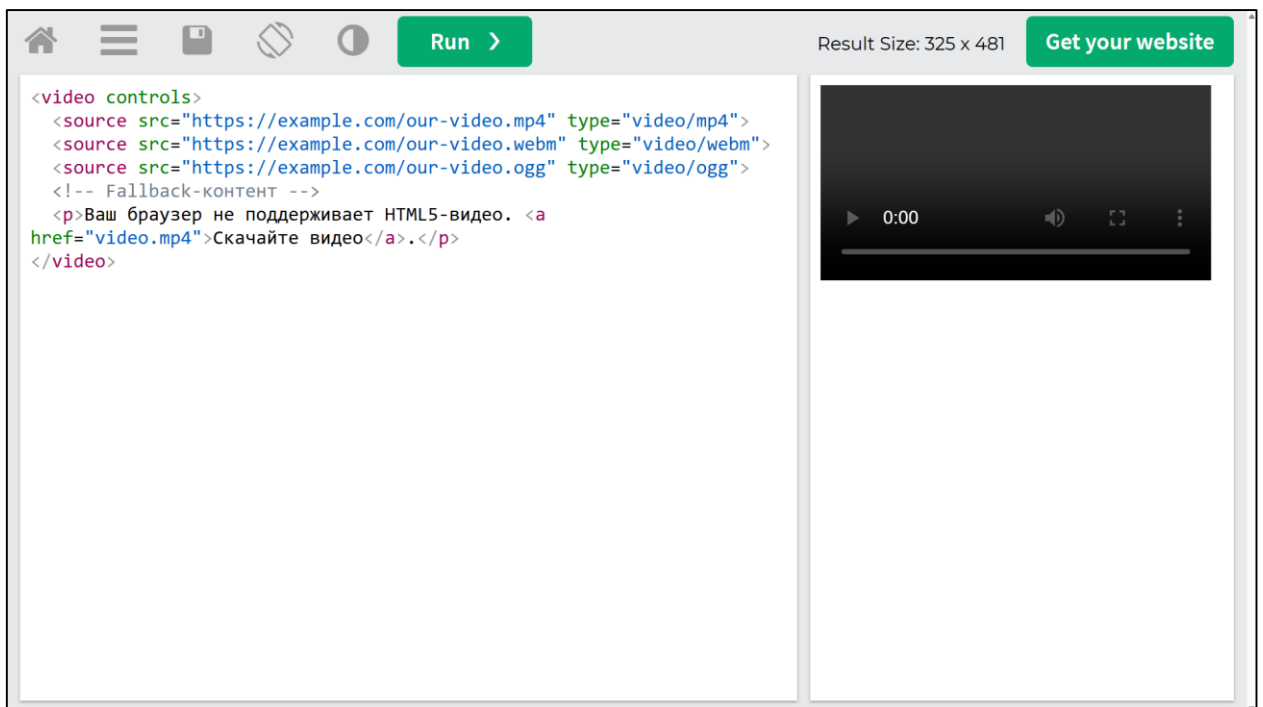
Если нужно кастомное управление (например, своя кнопка play/pause), используйте JavaScript и методы `play()`, `pause()`, `currentTime` и др.

6.2 Fallback-контент

Если браузер не поддерживает видео файл, показывается содержимое между тегами `<video>` и `</video>`. Таким образом можно поместить между этими тегами текст со ссылкой на нужный нам контент. Но если видео работает, то fallback-контент отображаться не будет. Он необходим для старых браузеров.

Пример 1:

```
<video controls>
  <source src="https://example.com/our-video.mp4" type="video/mp4">
  <source src="https://example.com/our-video.webm" type="video/webm">
  <source src="https://example.com/our-video.ogv" type="video/ogg">
  <!-- Fallback-контент -->
  <p>Ваш браузер не поддерживает HTML5-видео. <a
href="video.mp4">Скачайте видео</a>.</p>
</video>
```

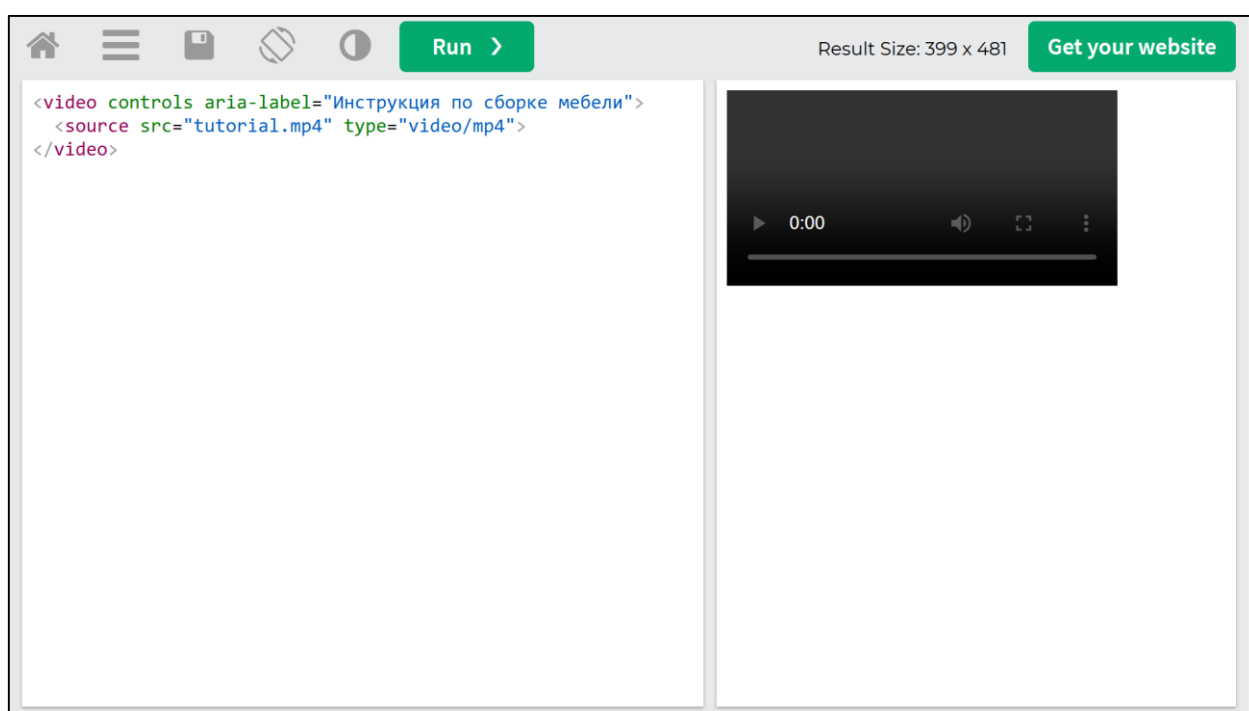


6.3 Атрибут aria-label

Данный атрибут добавляет текстовое описание видео для людей с ограниченными возможностями (для скринридеров). При его использовании скринридер озвучит текст внутри кавычек, принадлежащих атрибуту. Без его использования пользователь услышит только “Видео, кнопка воспроизведения”.

Пример 1:

```
<video controls aria-label="Инструкция по сборке мебели">  
  <source src="tutorial.mp4" type="video/mp4">  
</video>
```



6.4 Субтитры

Тэг **<track>** является непарным и добавляет скрытые титры, перевод и описание звуков. Данный тэг поддерживает расширение файлов “.vvt” (формат WebVVT). Так же **<track>** имеет свои атрибуты, одни из которых являются обязательными при его использовании. Ниже представлен пример оформления данного файла.

A screenshot of a code editor window with a dark theme. The title bar at the top shows 'vtt' on the left and 'Copy' on the right. The main content area displays the following text:

```
WEBVTT

00:00:01.000 --> 00:00:04.000
Привет, это пример субтитров.

00:00:05.000 --> 00:00:08.000
Теперь текст меняется.
```

Атрибут **default** в HTML-теге **<track>** используется для указания, что данная текстовая дорожка должна быть включена по умолчанию при загрузке медиафайла (видео или аудио). Если пользователь ранее выбирал субтитры, то его выбор сохранится, иначе будет использована любая другая дорожка (при ее наличии). Данный атрибут значения не требует.

Атрибут **kind** решает, как текстовый трек должен быть использован. Если значение опущено, тип по умолчанию — **subtitles** (субтитры). Если атрибут отсутствует, будет использоваться **subtitles**. Если атрибут содержит недопустимое значение, оно принимает значение **metadata**. (Версии Chrome ранее 52 рассматривали недопустимое значение как **subtitles**.) Допускаются следующие ключевые слова:

subtitles

- Субтитры обеспечивают перевод контента, который не может быть понят зрителем. Например, диалог или текст, который не является английским, в фильме на английском языке.
- Субтитры могут содержать дополнительный контент, обычно дополнительную справочную информацию. Например, текст в начале фильмов «Звёздных войн» или дата, время и место действия сцены.

captions

- Подписи обеспечивают транскрипцию и, возможно, перевод аудио.

- Подписи могут включать важную невербальную информацию, такую как музыкальные сигналы или звуковые эффекты. Возможно указывать источник сигналов (например, музыка, текст, персонаж).
- Подходит для пользователей со слабым слухом или, когда звук отключён.

descriptions

- Текстовое описание видеоконтента.
- Подходит для слепых пользователей или там, где видео не видно.

chapters

- Названия глав предназначены для использования при навигации по медиа-ресурсу.

metadata

- Данные, используемые скриптами. Не видны пользователю.

Атрибут **label** это видимый пользователю заголовок текстовой дорожки, который используется браузером при выводе списка доступных текстовых дорожек. (“English”, “Русский” и т.д.)

Атрибут **srclang** отвечает за язык текстовых данных трека. Это должен быть валидный [BCP 47](#) языковой тег (см. также языковые тэги в HTML и XML). Если для атрибута **kind** установлено значение **subtitles**, должен быть определён атрибут **srclang**.

Атрибут **draggable="false"** добавляется для запрета на перетаскивание видео в новую вкладку. Таким образом можно усложнить процесс скачивания видео файла. Но при этом важно понимать, что 100% защиты от скачивания видео нет.

Атрибут **oncontextmenu="return false;"** блокирует меню при нажатии правой кнопкой мыши, но пользователь его может обойти, отключив JavaScript или используя DevTools. Данный атрибут обычно используют в связке с **draggable**.

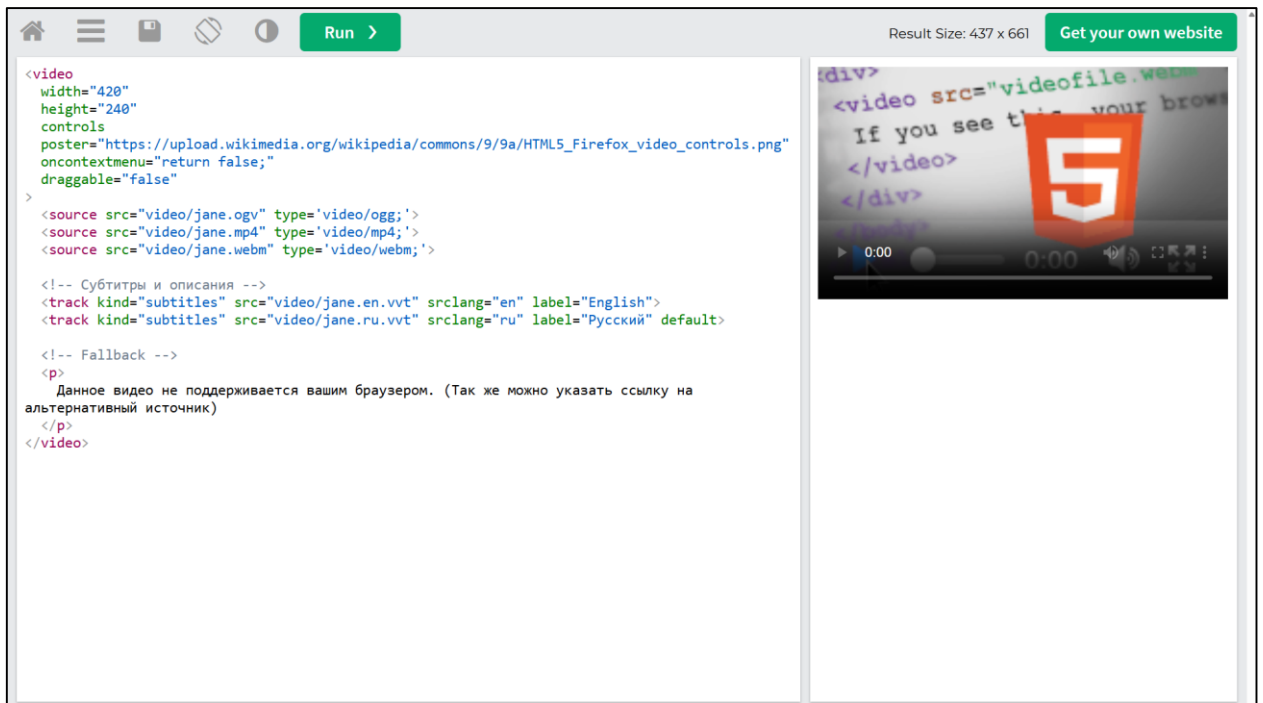
Ниже приведен пример, обобщающий, сказанную ранее информацию.

Пример 1:

```
<video
width="420"
height="240"
controls
poster="https://upload.wikimedia.org/wikipedia/commons/9/9a/HTML5_Firefox_video_controls.png"
oncontextmenu="return false;"
draggable="false">
  <source src="video/jane.ogv" type='video/ogg;'>
  <source src="video/jane.mp4" type='video/mp4;'>
  <source src="video/jane.webm" type='video/webm;'>

  <!-- Субтитры и описания -->
  <track kind="subtitles" src="video/jane.en.vvt" srclang="en"
label="English">
  <track kind="subtitles" src="video/jane.ru.vvt" srclang="ru"
label="Русский" default>

  <!-- Fallback -->
  <p>
    Данное видео не поддерживается вашим браузером. (Так же можно
указать ссылку на альтернативный источник)
  </p>
</video>
```



7. Списки

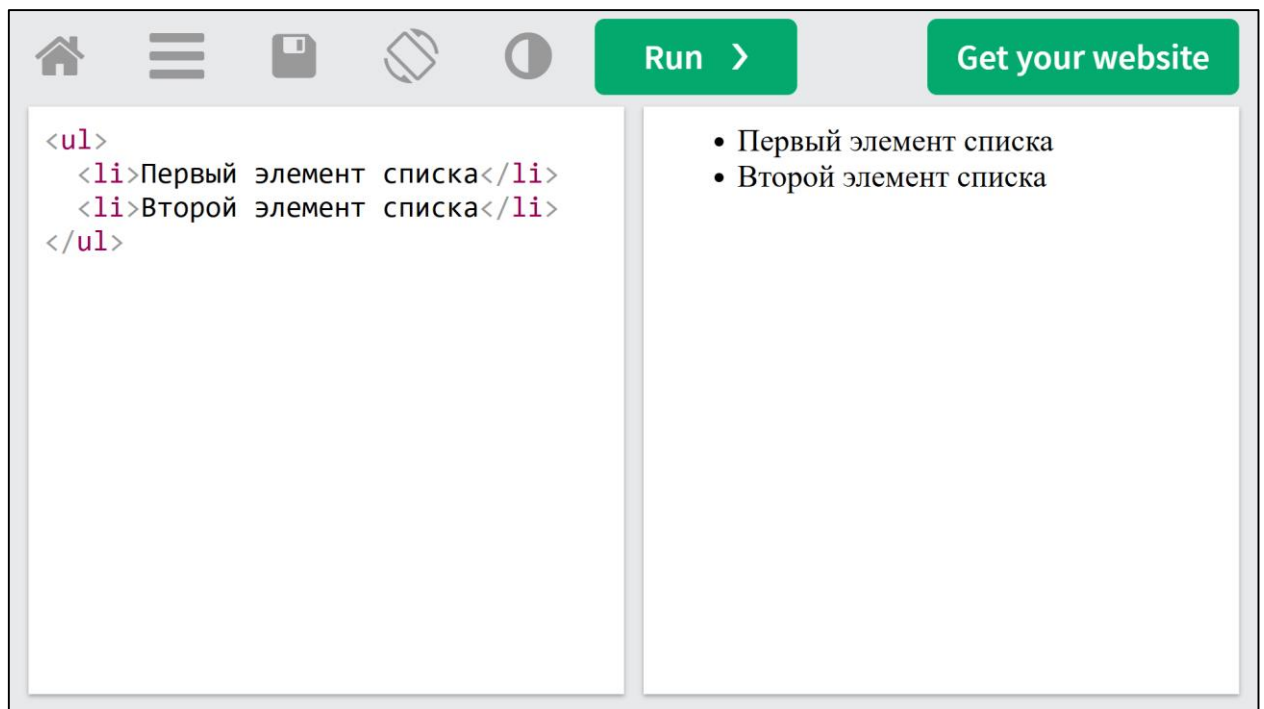
7.1 Ненумерованный список

Можно поменять стиль с помощью list-style-type, пример:
<ul style="list-style-type:disc;"> (список оформлен с помощью черных кружков).

Стили	
disc	Закрашенный кружок
circle	Пустой кружок
square	Черный квадратик

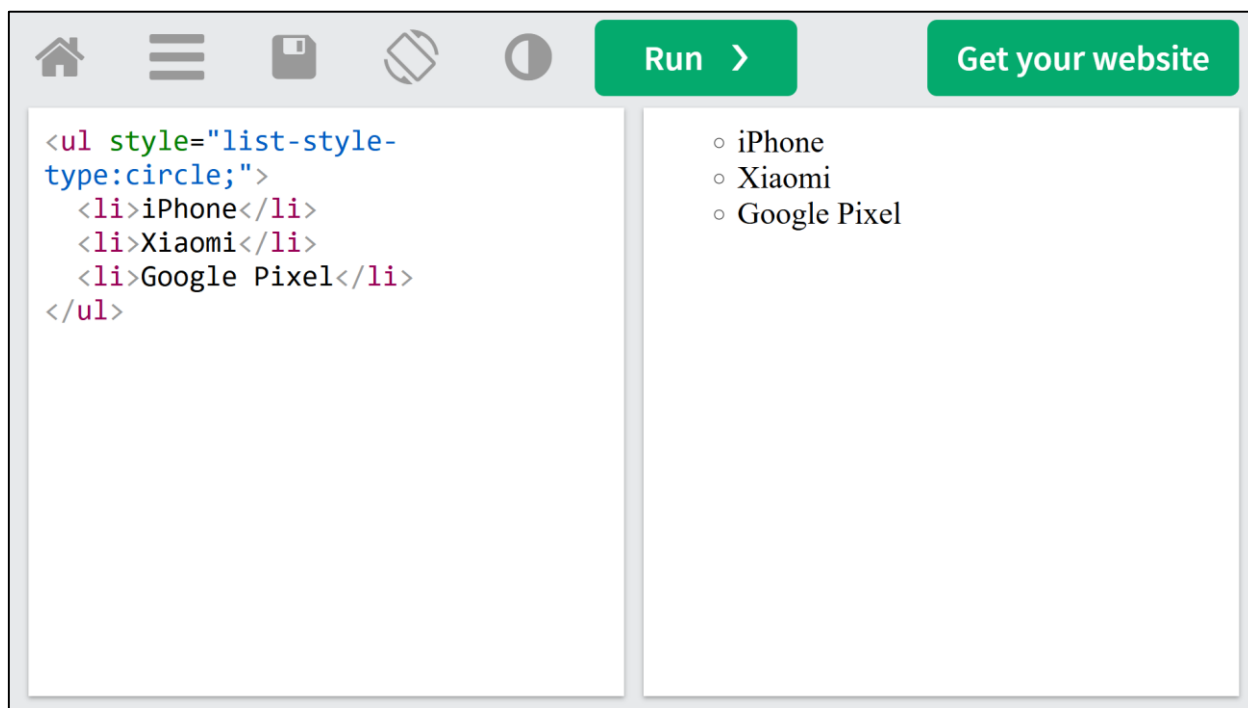
Пример 1:

```
<ul>
  <li>Первый элемент списка</li>
  <li>Второй элемент списка</li>
</ul>
```



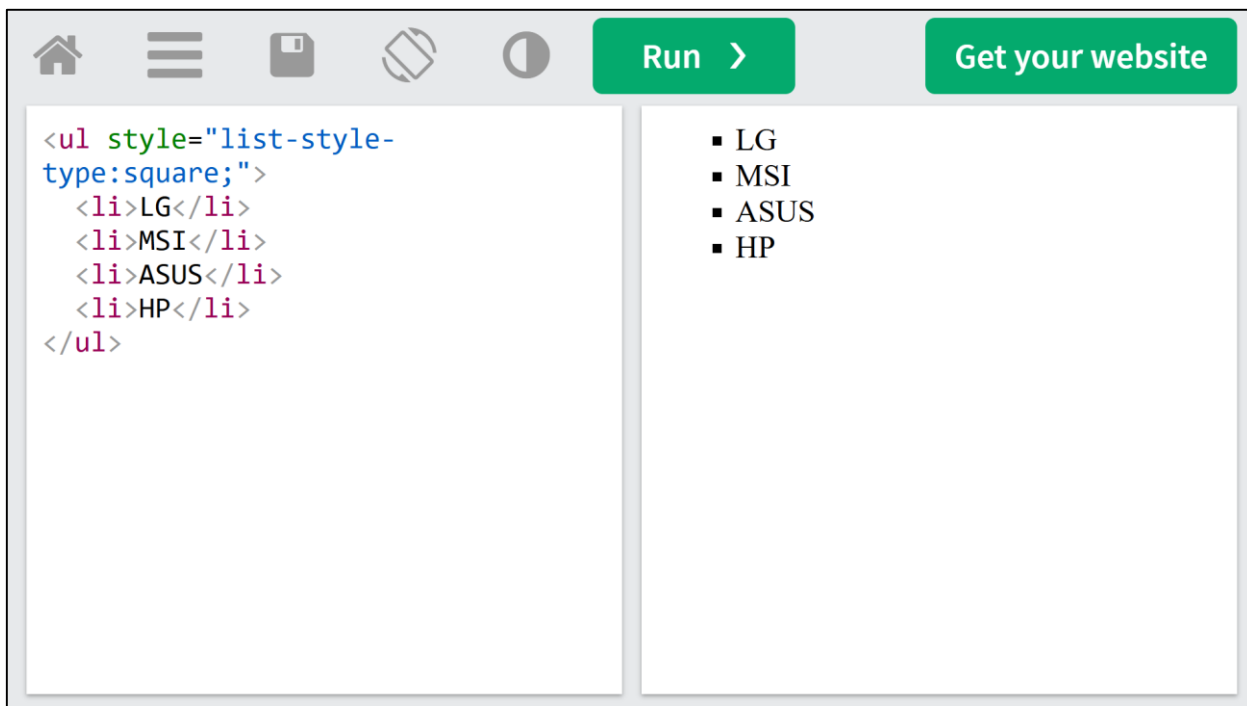
Пример 2:

```
<ul style="list-style-type:circle;">
  <li>iPhone</li>
  <li>Xiaomi</li>
  <li>Google Pixel</li>
</ul>
```



Пример 3:

```
<ul style="list-style-type:square;">
  <li>LG</li>
  <li>MSI</li>
  <li>ASUS</li>
  <li>HP</li>
</ul>
```



Используя в настройке списков стиль `list-style-image` можно задать собственное изображение, которое будет отображаться рядом с элементом списка.

Пример 3:

```
<ul style="list-style-image:url(phone_touch.png);">
  <li>iPhone 6S</li>
  <li>Galaxy S7</li>
  <li>Nexus 5X</li>
  <li>Lumia 950</li>
</ul>
```

Стиль `list-style-image` принимает в качестве значения `url(phone_touch.png)`, где `"phone_touch.png"` - это название файла изображения. То есть в данном случае предполагается, что в одной папке с веб-страницей `index.html` у меня находится файл изображения `phone_touch.png`.

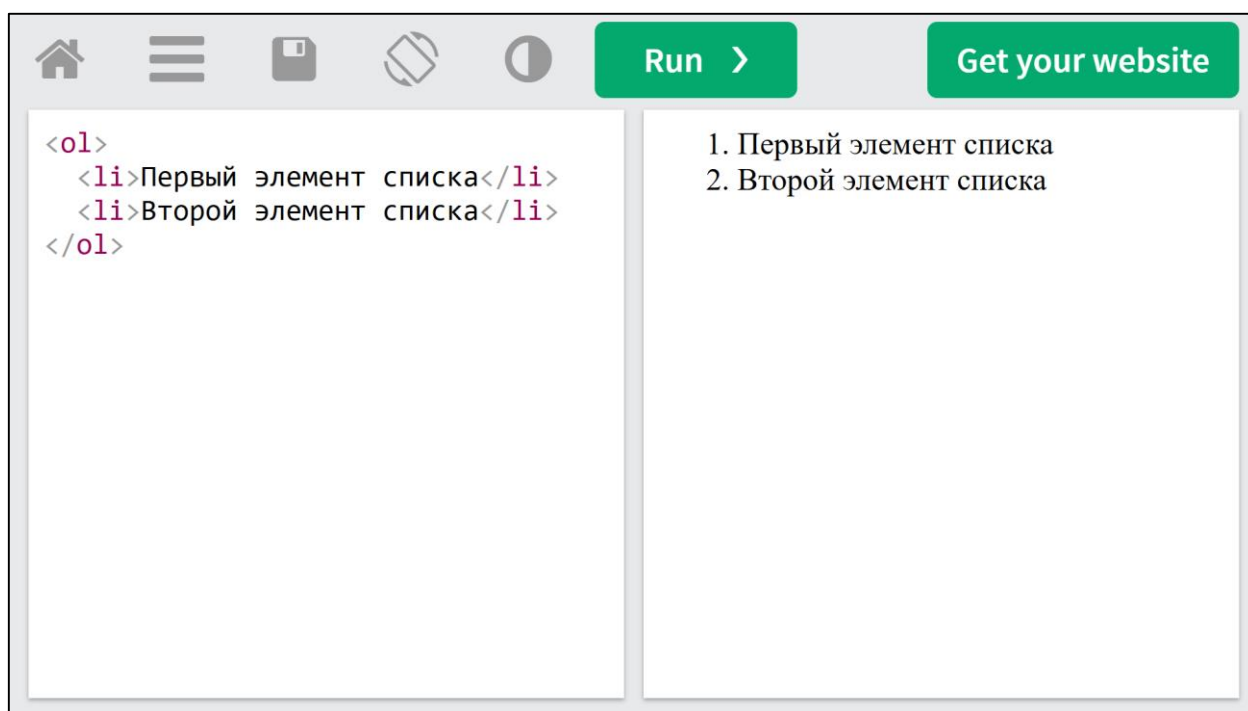
7.2 Нумерованный список

Можно поменять стиль с помощью list-style-type, пример:
<ol style="list-style-type:decimal;" start="3"> (нумерация десятичными цифрами, которая начинается с цифры 3).

Стили	
decimal	1, 2, 3, ... 98, 99
decimal-leading-zero	01, 02, 03, ... 98, 99
lower-roman	i, ii, iii, iv, v...
upper-roman	I, II, III, IV, V...
lower-alpha	a, b, c, ..., z
upper-alpha	A, B, C, ..., Z

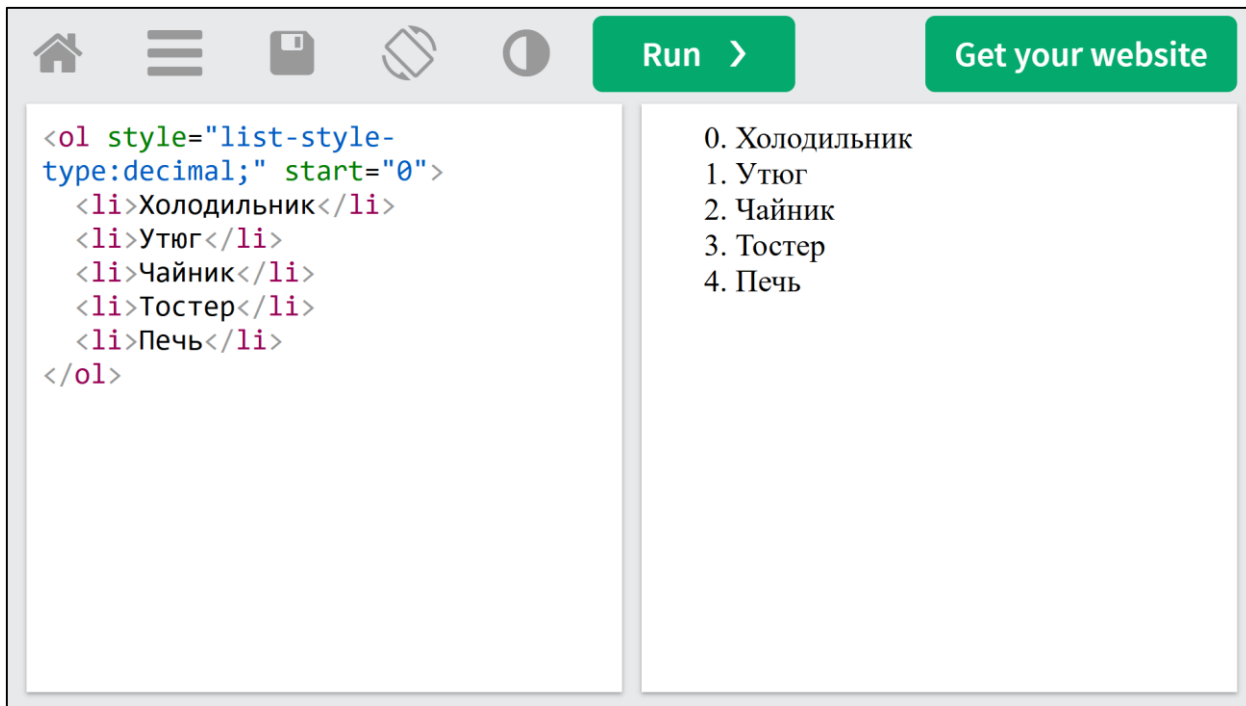
Пример 1:

```
<ol>
  <li>Первый элемент списка</li>
  <li>Второй элемент списка</li>
</ol>
```



Пример 2:

```
<ol style="list-style-type:decimal;" start="0">
  <li>Холодильник</li>
  <li>Утюг</li>
  <li>Чайник</li>
  <li>Тостер</li>
  <li>Печь</li>
</ol>
```



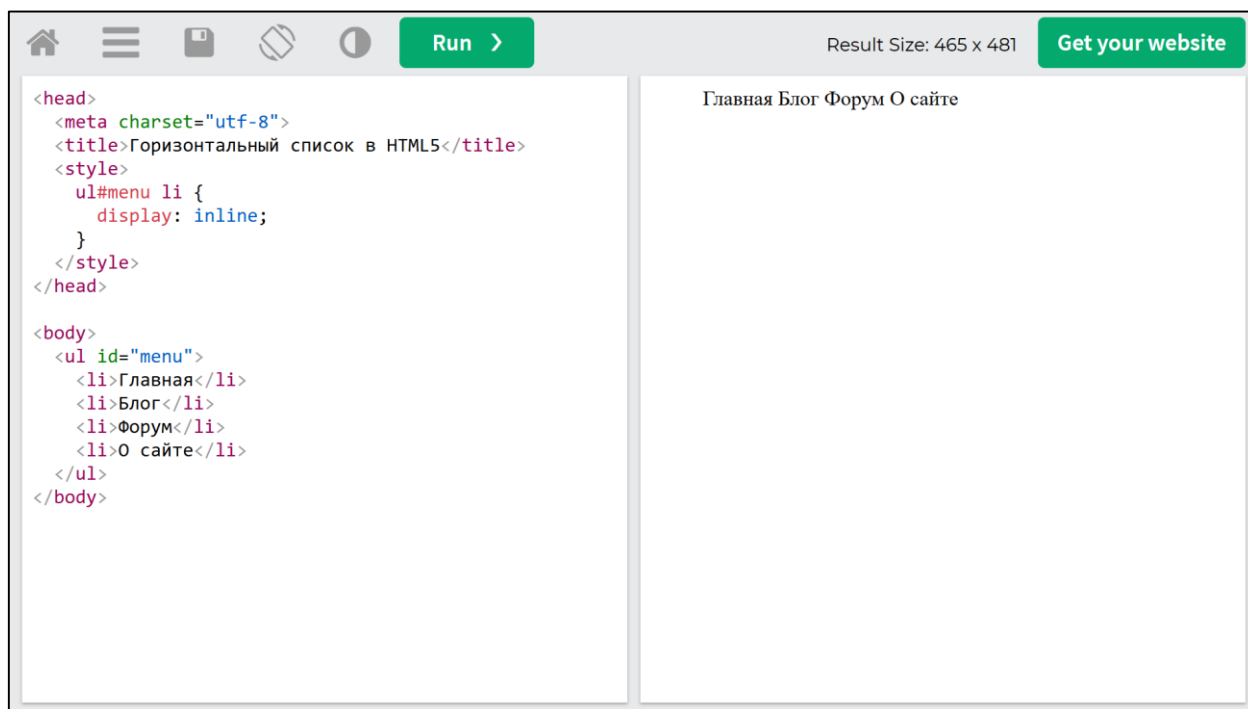
7.3 Горизонтальный список

Одним из распространенных способов стилизации списков представляет создание горизонтального списка. Для этого для всех элементов списка надо установить стиль `display:inline`. Применяется для создания навигационного меню сайта.

Пример 1:

```
<head>
  <meta charset="utf-8">
  <title>Горизонтальный список в HTML5</title>
  <style>
    ul#menu li {
      display: inline;
    }
  </style>
</head>

<body>
  <ul id="menu">
    <li>Главная</li>
    <li>Блог</li>
    <li>Форум</li>
    <li>О сайте</li>
  </ul>
</body>
```



7.4 Вложенные списки

В некоторых задачах могут понадобиться вложенные списки. Что позволит вложить в качестве элемента списка не просто текст, а новый список, создавая сложную структуру.

Важно отметить, что вложенный список является частью пункта списка и находится внутри тега `<li>`.

Такая вложенность позволяет сохранить семантику и указать, что вложенный список относится именно к нужному ему пункту, а не к какому-либо ещё.

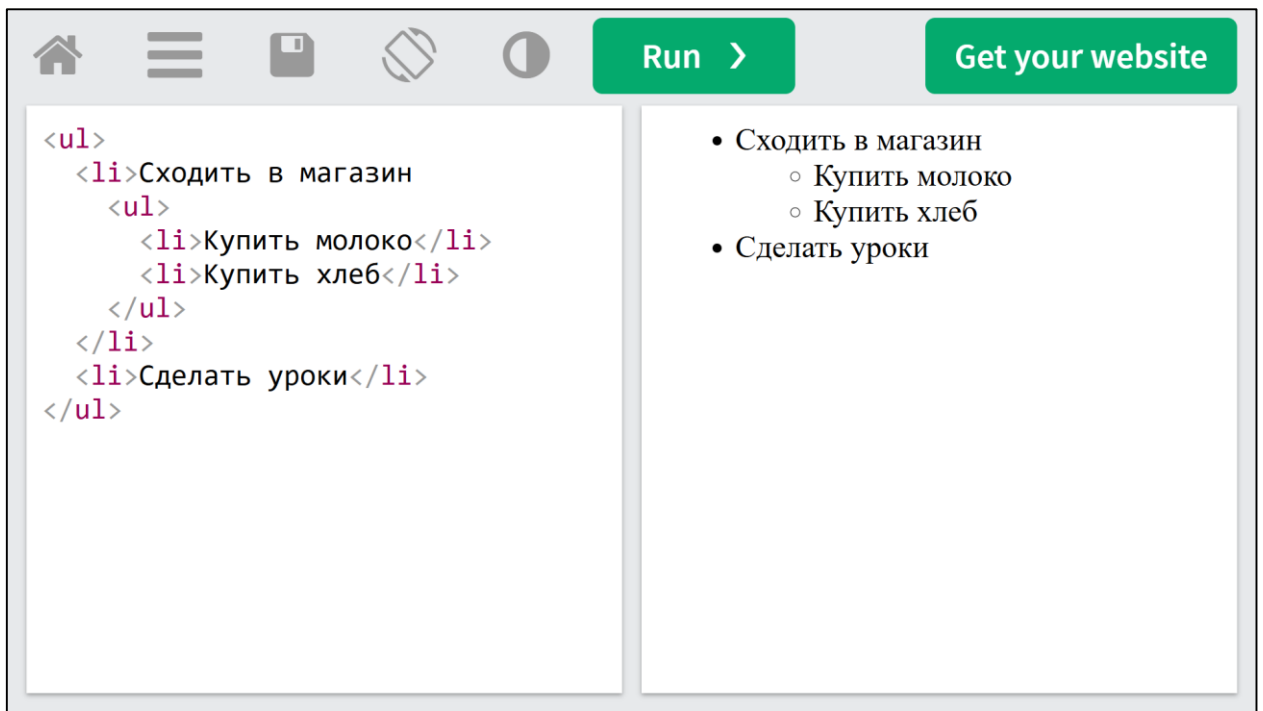
Вы можете вкладывать списки разных видов друг в друга: маркированные в нумерованные и наоборот. Главное — следить за открытием и закрытием тегов. В случае ошибки браузер попытается самостоятельно исправить ошибку, но это может быть некорректно.

Полезное:

- На каждом уровне вложенности маркированный список меняет свой маркер. Это сделано для визуального отличия вложенных списков.

Пример 1:

```
<ul>
  <li>Сходить в магазин
    <ul>
      <li>Купить молоко</li>
      <li>Купить хлеб</li>
    </ul>
  </li>
  <li>Сделать уроки</li>
</ul>
```

8. Таблицы

Одна из самых привычных форм подачи информации — таблицы. Таблица — составной элемент, который формируется сразу из нескольких вложенных друг в друга тегов. Это похоже на то, как верстаются списки — имеется контейнер и внутри него специальные теги-элементы.

Разметка содержит в себе несколько основных элементов:

- `<table>` — область таблицы
 - `<tr>` — строка
 - `<thead>` — шапка таблицы
 - `<th>` — ячейка шапки
 - `<tbody>` — тело таблицы
 - `<td>` — ячейка

Любая таблица начинается с парного тега `<table>`. Именно внутри этого тега создаются строки и столбцы таблицы. Теперь можно создать строки и столбцы. Для этого используются теги `<tr>` и `<td>`. Стоит отметить, что тэг `<th>` не только делает текст шапки жирным, но и располагает его по середине ячейки.

Также таблицы имеют свои собственные атрибуты. [Подробнее про них по ссылке.](#)

Пример 1:

```
<table>
  <thead>
    <tr>
      <th>Жирный заголовок 1</th>
      <th>Жирный заголовок 2</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Обычный текст 1</td>
      <td>Обычный текст 2</td>
    </tr>
  </tbody>
</table>
```



Run >

Result Size: 369 x 373

Get your website

```
<table>
  <thead>
    <tr>
      <th>Жирный заголовок 1</th>
      <th>Жирный заголовок 2</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Обычный текст 1</td>
      <td>Обычный текст 2</td>
    </tr>
  </tbody>
</table>
```

Жирный заголовок 1	Жирный заголовок 2
Обычный текст 1	Обычный текст 2

8.1 Атрибуты таблиц

8.1.1 Атрибуты тега <table>

Атрибут **align** («выровнять») задает выравнивание таблицы по краю окна браузера. Допустимые значения: `left` — выравнивание таблицы по левому краю, `center` — по центру и `right` — по правому краю. Когда используются значения `left` и `right`, текст начинает обтекать таблицу сбоку и снизу.

Пример 1:

```
<table align="right">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **bgcolor** («цвет фона») устанавливает цвет фона таблицы.

Пример 2:

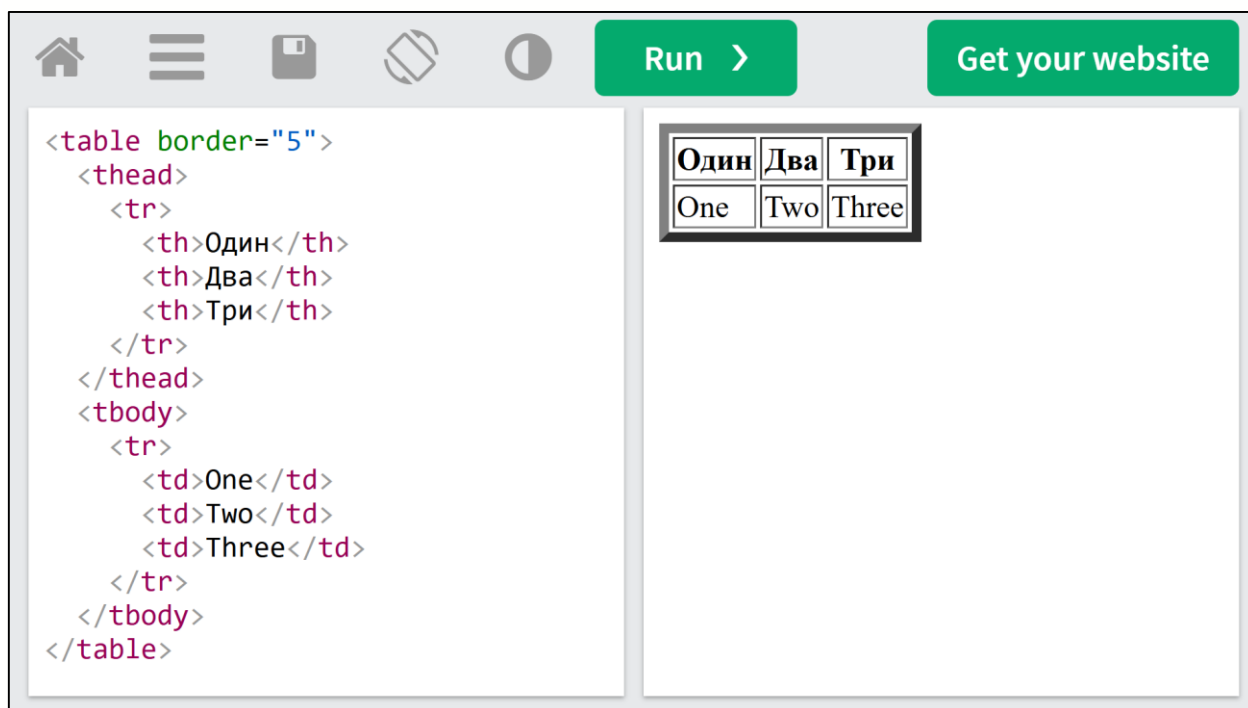
```
<table bgcolor="pink">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **border** («граница») устанавливает толщину границы в пикселах вокруг таблицы. При наличии этого атрибута также отображаются границы между ячеек.

Пример 3:

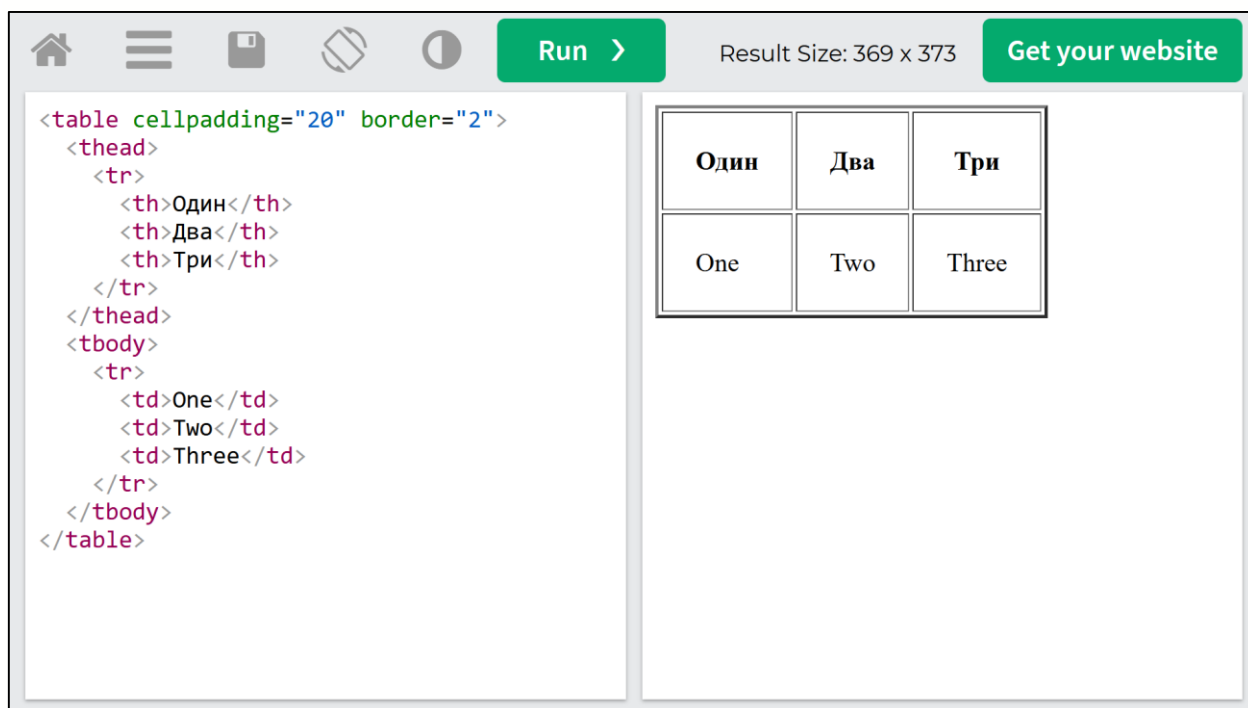
```
<table border="5">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **cellpadding** («заполнение ячеек») определяет расстояние между границей ячейки и ее содержимым. Этот атрибут добавляет пустое пространство к ячейке, увеличивая тем самым ее размеры. Без cellpadding текст в таблице «налипает» на рамку, ухудшая тем самым его восприятие. Добавление же cellpadding позволяет улучшить читабельность текста. При отсутствии границ особого значения этот атрибут не имеет, но может помочь, когда требуется установить пустой промежуток между ячейками.

Пример 4:

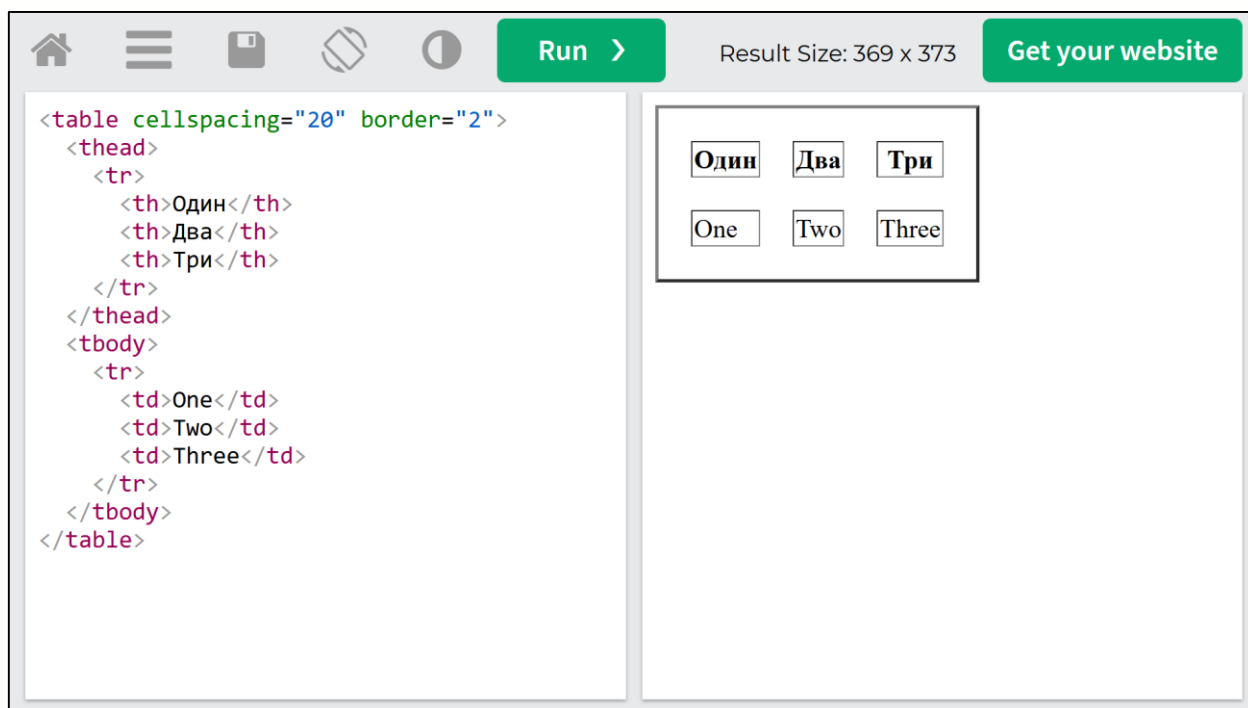
```
<table cellpadding="20" border="2">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **cellspacing** («расстояние между ячейками») Задает расстояние между внешними границами ячеек. Если установлен атрибут border, толщина границы принимается в расчет и входит в общее значение.

Пример 5:

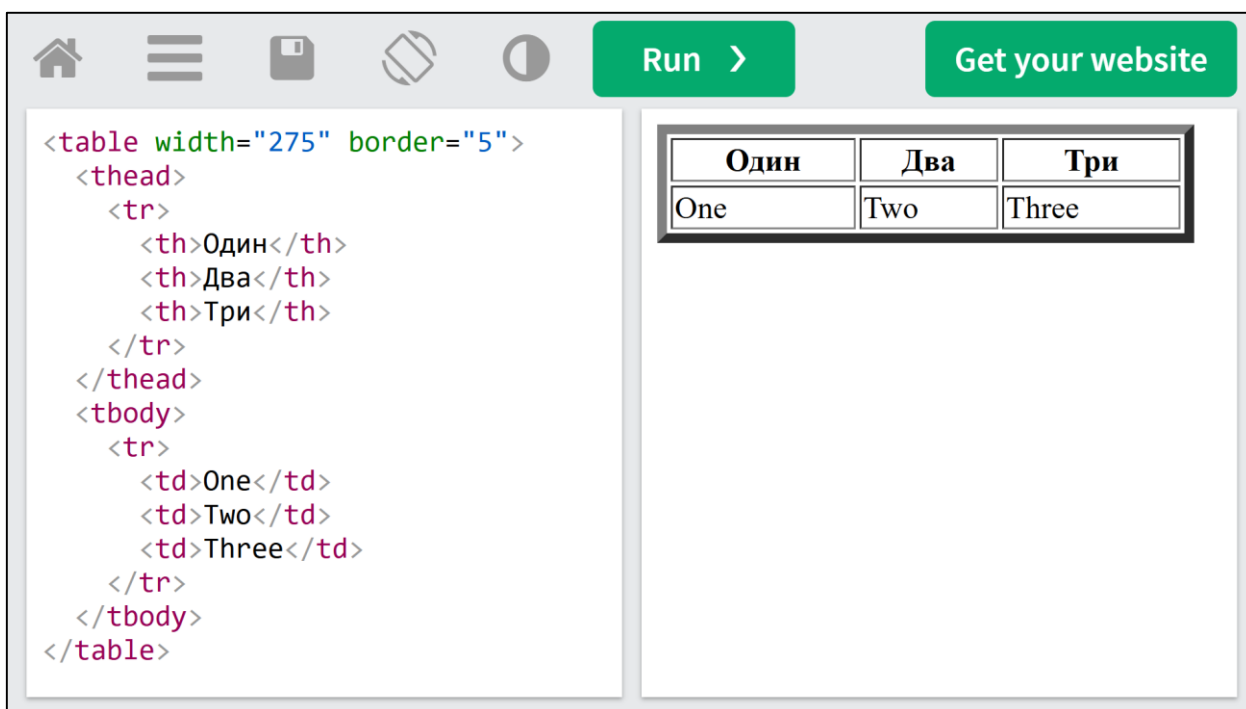
```
<table cellpadding="20" border="2">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **width** («ширина») задает ширину таблицы. Если общая ширина содержимого превышает указанную ширину таблицы, то браузер будет пытаться «втиснуться» в заданные размеры за счет форматирования текста. В случае, когда это невозможно, например, в таблице находятся изображения, атрибут **width** будет проигнорирован, и новая ширина таблицы будет вычислена на основе ее содержимого.

Пример 6:

```
<table width="275" border="5">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```



The screenshot shows a web development IDE interface. On the left, a code editor displays the HTML code for a table. On the right, a preview window shows the rendered table. The IDE has a toolbar at the top with icons for home, menu, save, undo, and a 'Run' button. A green button labeled 'Get your website' is also present.

Code editor content:

```
<table width="275" border="5">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```

Rendered table content:

Один	Два	Три
One	Two	Three

8.1.2 Атрибуты тега <td>

Каждая ячейка таблицы, задаваемая через тег <td>, в свою очередь тоже имеет свои атрибуты, часть из которых совпадает с атрибутами тега <table>.

Атрибут **align** («выровнять») задает выравнивание содержимого ячейки по горизонтали. Возможные значения: left — выравнивание по левому краю, center — по центру и right — по правому краю ячейки.

Пример 1:

```
<table border="5" width="275">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td align="left">One</td>
      <td align="center">Two</td>
      <td align="right">Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **bgcolor** («цвет фона») устанавливает цвет фона ячейки. Используя этот атрибут совместно с атрибутом bgcolor тега <table> можно получить разнообразные цветовые эффекты в таблице.

Пример 2:

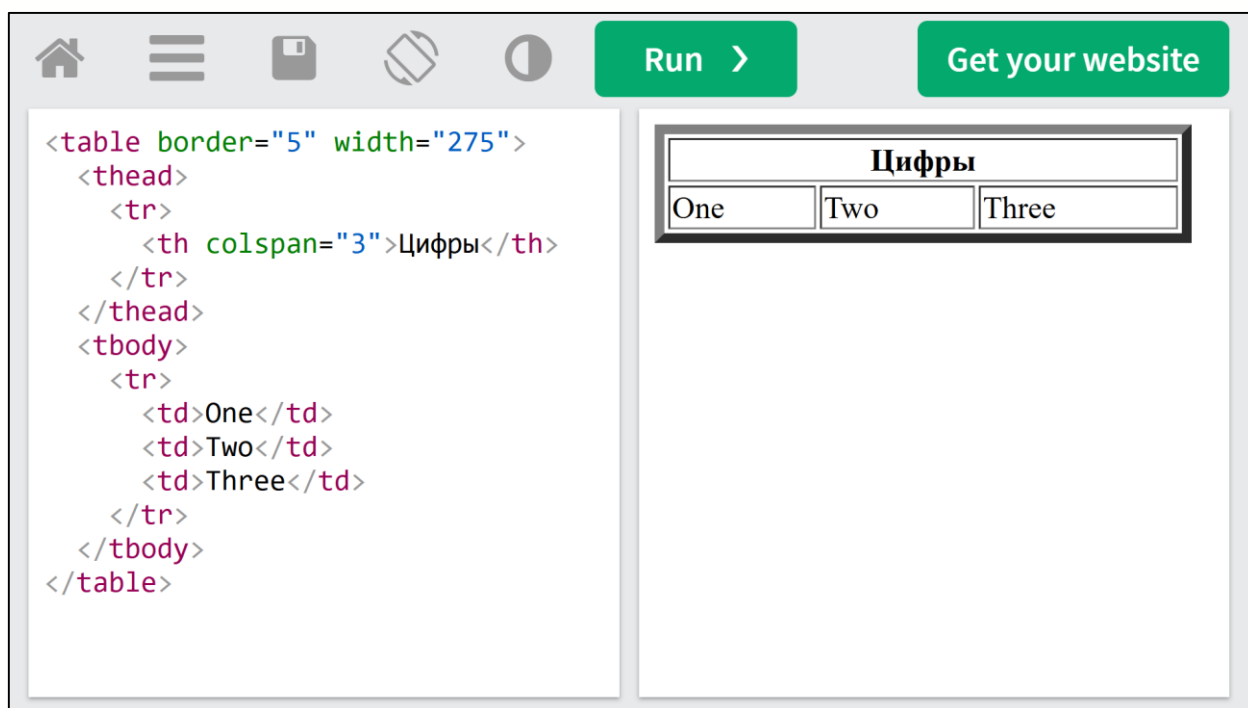
```
<table border="5" width="275">
  <thead>
    <tr>
      <th>Один</th>
      <th>Два</th>
      <th>Три</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td bgcolor="red">One</td>
      <td bgcolor="yellow">Two</td>
      <td bgcolor="green">Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **colspan** (от англ. "columns span" – «охват столбцов») устанавливает число ячеек, которые должны быть объединены по горизонтали. Этот атрибут имеет смысл для таблиц, состоящих из нескольких столбцов.

Пример 3:

```
<table border="5" width="275">
  <thead>
    <tr>
      <th colspan="3">Цифры</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>One</td>
      <td>Two</td>
      <td>Three</td>
    </tr>
  </tbody>
</table>
```



Атрибут **rowspan** (от англ. "row span" – «охват строк») устанавливает число ячеек, которые должны быть объединены по горизонтали. Этот атрибут имеет смысл для таблиц, состоящих из нескольких столбцов.

Пример 4:

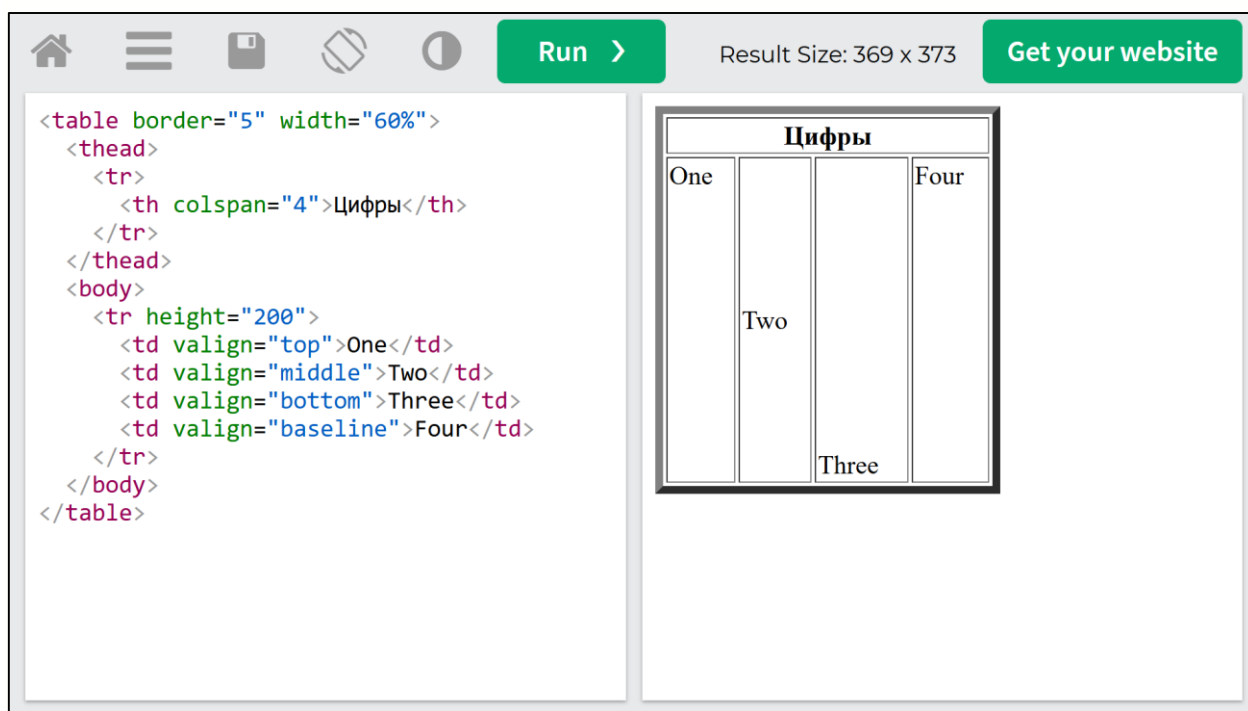
```
<table border="5" width="275">
  <tbody>
    <tr>
      <th rowspan="2">Цифры</th>
      <td>One</td>
    </tr>
    <tr>
      <td>Two</td>
    </tr>
  </tbody>
</table>
```



Атрибут **valign** («от англ. "vertical alignment" – «вертикальное выравнивание»») устанавливает вертикальное выравнивание содержимого ячейки. По умолчанию содержимое ячейки располагается по ее вертикали в центре. Возможные значения: **top** — выравнивание по верхнему краю строки, **middle** — выравнивание по середине, **bottom** — выравнивание по нижнему краю, **baseline** — выравнивание по базовой линии, при этом происходит привязка содержимого ячейки к одной линии.

Пример 5:

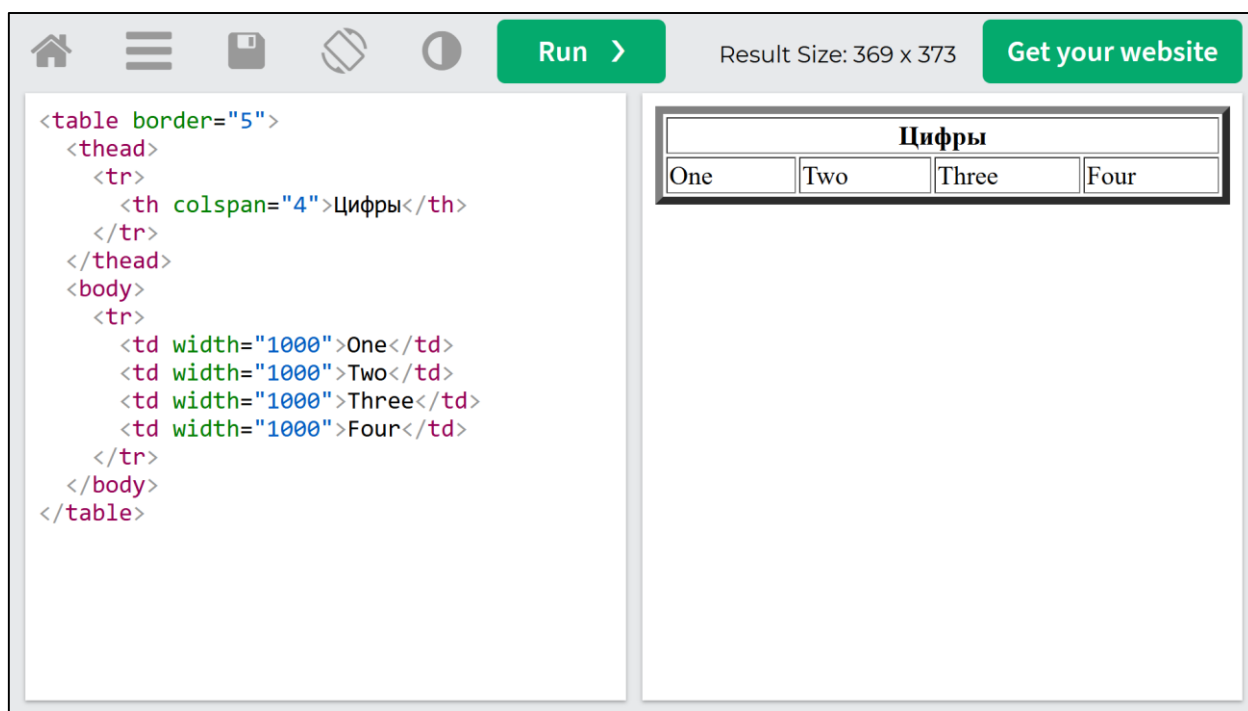
```
<table border="5" width="60%">
  <thead>
    <tr>
      <th colspan="4">Цифры</th>
    </tr>
  </thead>
  <body>
    <tr height="200">
      <td valign="top">One</td>
      <td valign="middle">Two</td>
      <td valign="bottom">Three</td>
      <td valign="baseline">Four</td>
    </tr>
  </body>
</table>
```



Атрибут **width** («ширина») задает ширину ячейки. Суммарное значение ширины всех ячеек может превышать общую ширину таблицы только в том случае, если содержимое ячейки превышает указанную ширину.

Пример 6:

```
<table border="5">
  <thead>
    <tr>
      <th colspan="4">Цифры</th>
    </tr>
  </thead>
  <body>
    <tr>
      <td width="1000">One</td>
      <td width="1000">Two</td>
      <td width="1000">Three</td>
      <td width="1000">Four</td>
    </tr>
  </body>
</table>
```



Полезное:

- Количество ячеек в каждой строке таблицы должно быть одинаковым после объединения. Важно различать количество тегов `<td>` и общее количество ячеек. Например, используя атрибут `colspan="2"` вы получите две логические ячейки, только они будут объединены. По этой причине необходимо убрать одну физическую ячейку из разметки для компенсации;
- Если используется атрибут `colspan`, то из HTML нужно удалить ячейки в той же строке. Если используется атрибут `rowspan`, то удаляются ячейки в строках ниже.

9. Формы

Формы являются одним из важных элементов любого сайта и предназначены для обмена данными между пользователем и сервером. Область применения форм не ограничена отправкой данных на сервер, с помощью клиентских скриптов можно получить доступ к любому элементу формы, изменять его и применять по своему усмотрению.

Любая форма начинается с главного тега `<form>`, внутри которого и будут располагаться поля для заполнения и кнопка действия.

Пример 1:

```
<form action="/search">  
  // Данные, после заполнения, будут отправлены на страницу /search  
</form>
```

Если данный атрибут не указан, данные уйдут на ту же страницу, откуда была отправлена форма.

В данной главе ниже в соответствии с формой будут указаны атрибуты, с которыми она работает, все те атрибуты, которые повторялись ранее будут выделены серым цветом!

9.1 Поле для ввода текста

Однострочное текстовое поле предназначено для ввода строки символов с помощью клавиатуры.

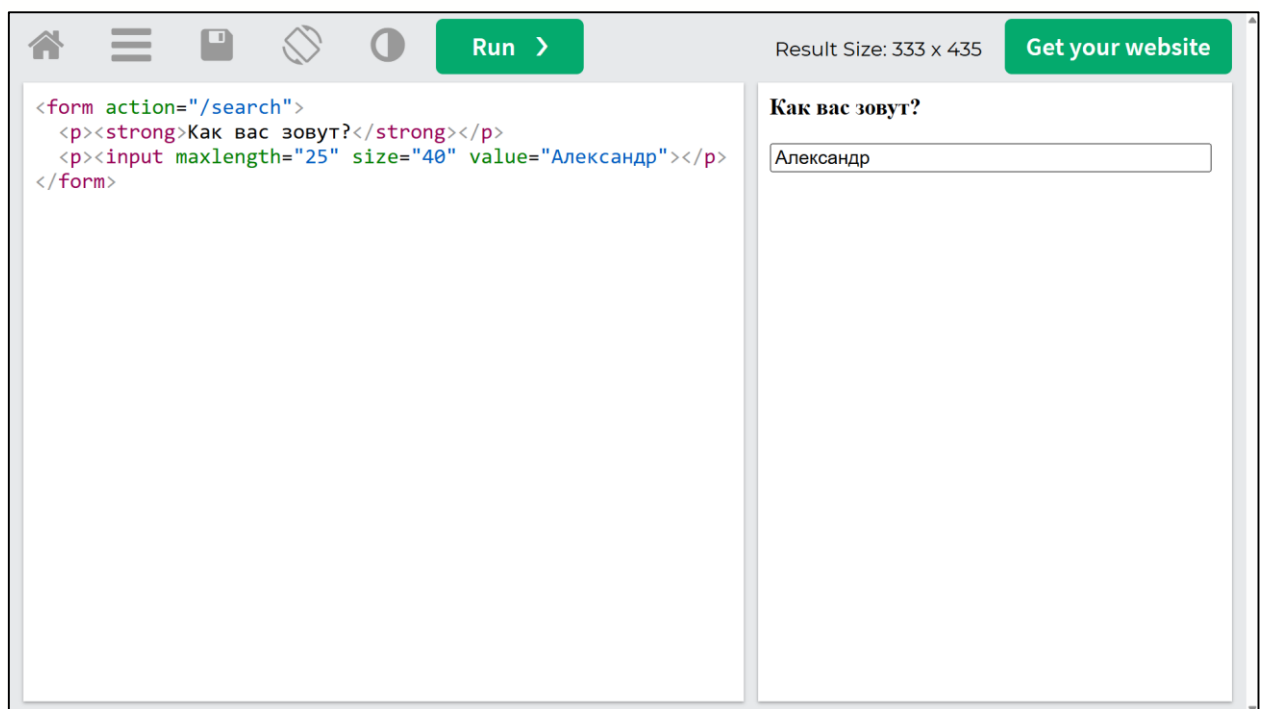
Значение атрибута **type** для непарного тега `<input>` по умолчанию определено как `text`, поэтому его можно не указывать явно. Помимо атрибута `type` тег `<input>` имеет и другие атрибуты.

Атрибут	Описание
name	Принимает имя поля, предназначенное для того, чтобы обработчик формы мог его идентифицировать.
value	Помещает начальный текст в поле для ввода, при желании пользователь может его стереть.
size	Отвечает за ширину поля формы, которая определяется числом символом моноширного шрифта. Простыми словами, ширина задается количеством близстоящих букв одинаковой ширины, стоящих по горизонтали.
placeholder	Определяет короткую подсказку-"заполнитель", описывающую ожидаемое для ввода значение. (То, что часто используется для форм на сайтах)
maxlength	Устанавливает максимальное число символов, которое может быть введено пользователем в текстовом поле. Когда это количество достигается при наборе, дальнейший ввод становится невозможным. Если этот атрибут опустить, то можно вводить строку длиннее самого поля. Также есть противоположный ему атрибут minlength .
required	Делает поле обязательным для заполнения перед отправкой формы. Принимает параметры <code>yes</code> и <code>no</code> , в зависимости от задачи формы.
readonly	Делает поле доступным только для чтения, пользователь не может изменять его значение.
disabled	Делает поле недоступным для ввода и отправки данных. (не требует указания значения)
autocomplete	Указывает, должна ли браузер автоматический заполнять поле на основе ранее введенных значений (<code>on</code> или <code>off</code>).

pattern	Задаёт регулярное выражение, которому должно соответствовать введённое значение, для проверки правильности формата.
tabindex	Определяет порядок перехода между полями при использовании клавиши Tab.
spellcheck	Указывает, нужно ли проверять орфографию введённого текста (true или false). По умолчанию принимает значение true, если задать значение false, то неправильно написанный текст не будет подчеркиваться волнистой линией.

Пример 1:

```
<form action="/search">
  <p><strong>Как вас зовут?</strong></p>
  <p><input maxlength="25" size="40" value="Александр"
name="text"></p>
</form>
```



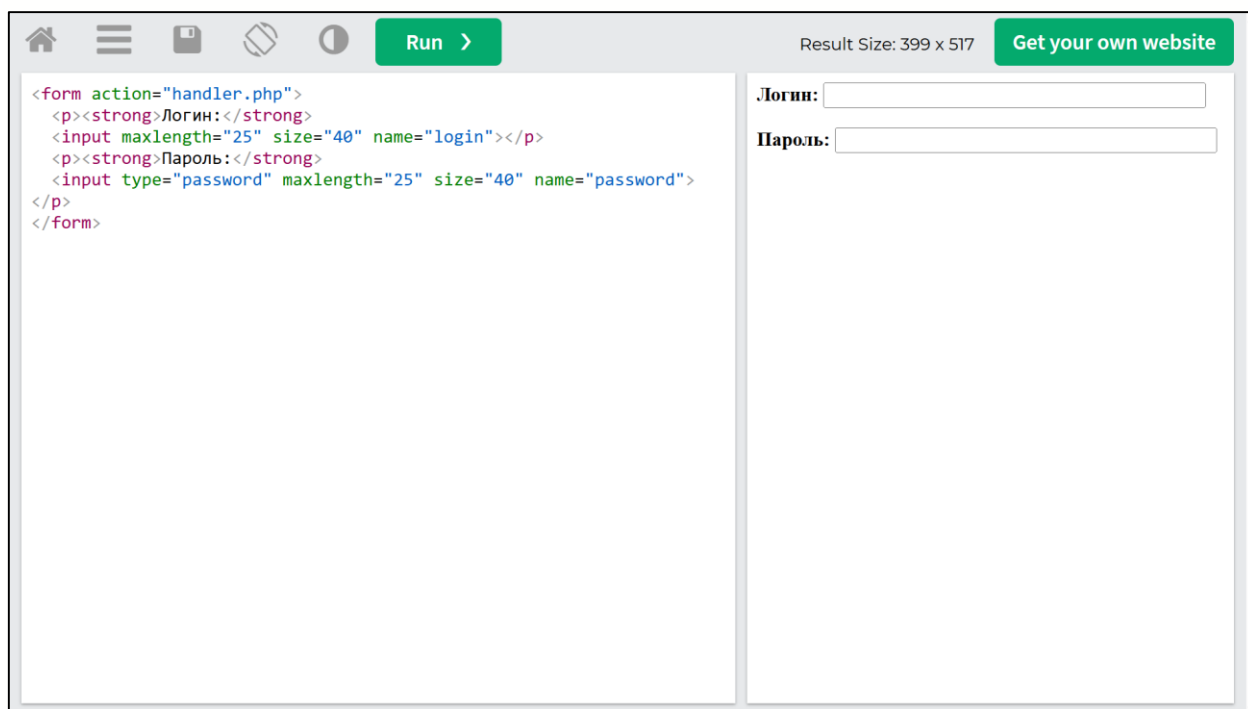
9.2 Поле для пароля

Поле для пароля — обычное текстовое поле, но отличается от него тем, что все вводимые символы отображаются звездочками, точками или другими знаками (это зависит от браузера). Поле предназначено для того, чтобы никто не подглядел вводимый пароль.

Чтобы создать поле для ввода пароля, нужно в атрибуте **type** указать значение **password**. Данный тип формы имеет те же атрибуты, что и обычное поле для ввода текста с клавиатуры.

Пример 1:

```
<form action="handler.php">
  <p><strong>Логин:</strong>
    <input maxlength="25" size="40" name="login">
  </p>
  <p><strong>Пароль:</strong>
    <input type="password" maxlength="25" size="40" name="password">
  </p>
</form>
```



Такой пароль, введенный с клавиатуры и преобразованный в специальные символы, уже не получится скопировать. А чтобы сделать временное отображение введенного символа, нужно использовать JavaScript.

9.3 Многострочный текст

Этот элемент формы предназначен для создания области, в которой можно вводить несколько строк текста. В таком текстовом поле допустимо делать переносы строк, они сохраняются при отправке данных на сервер (как при использовании тега `<pre>`).

Поле для многострочного текста незаменимо для добавления комментариев к статьям, написания сообщений форума, вставки и редактирования постов в блоге и во многих других случаях, когда одной строки текста явно недостаточно.

Форма оформляется стандартно, с использованием парного тега `<textarea>`. Данный вид формы также имеет и собственные атрибуты, которые представлены ниже.

Атрибут	Описание
name	Принимает имя поля, предназначенное для того, чтобы обработчик формы мог его идентифицировать.
placeholder	Определяет короткую подсказку-"заполнитель", описывающую ожидаемое для ввода значение. (То, что часто используется для форм на сайтах)
maxlength	Устанавливает максимальное число символов, которое может быть введено пользователем в текстовом поле. Когда это количество достигается при наборе, дальнейший ввод становится невозможным. Если этот атрибут опустить, то можно вводить строку длиннее самого поля. Также есть противоположный ему атрибут minlength .
required	Делает поле обязательным для заполнения перед отправкой формы. Принимает параметры yes и no , в зависимости от задачи формы.
readonly	Делает поле доступным только для чтения, пользователь не может изменять его значение.
spellcheck	Указывает, нужно ли проверять орфографию введенного текста (true или false). По умолчанию принимает значение true, если задать значение false, то неправильно написанный текст не будет подчеркиваться волнистой линией.

rows	Определяет количество видимых строк текста в области для ввода, фактически задает начальную высоту формы.
cols	Задаёт количество символов в одной строке, которое будет помещено в области для ввода, фактически задает начальную длину формы.
wrap	Определяет поведение переноса текста. Может принимать значения soft (на сервер придет текст в виде одной строчки, без каких-либо переносов) и hard (на сервер придет текст со всеми переносами, заданными пользователем). По умолчанию принимает значение soft.
autofocus	Автоматически устанавливает фокус на этом поле при загрузке страницы.

Пример 1:

```
<form action="handler.php">
  <p><b>Введите ваш отзыв:</b></p>
  <p><textarea name="comment" placeholder="text" spellcheck="false"
required="yes" wrap="hard"></textarea></p>
  <p><input type="submit"></p>
</form>
```

The screenshot shows a web browser window with a code editor on the left and a rendered web page on the right. The code editor displays the HTML code from the previous block. The rendered page shows a form with the title "Введите ваш отзыв:", a text area with the placeholder "text", and a submit button labeled "Отправить". The browser's address bar shows "Result Size: 305 x 661" and a button "Get your own website".

9.4 Кнопки

Кнопки являются одним из самых понятных и интуитивных элементов интерфейса. По их виду сразу становится понятно, что единственное действие, которое с ними можно производить — это нажимать на них. За счёт этой особенности кнопки часто применяются в формах, особенно при их отправке и очистке.

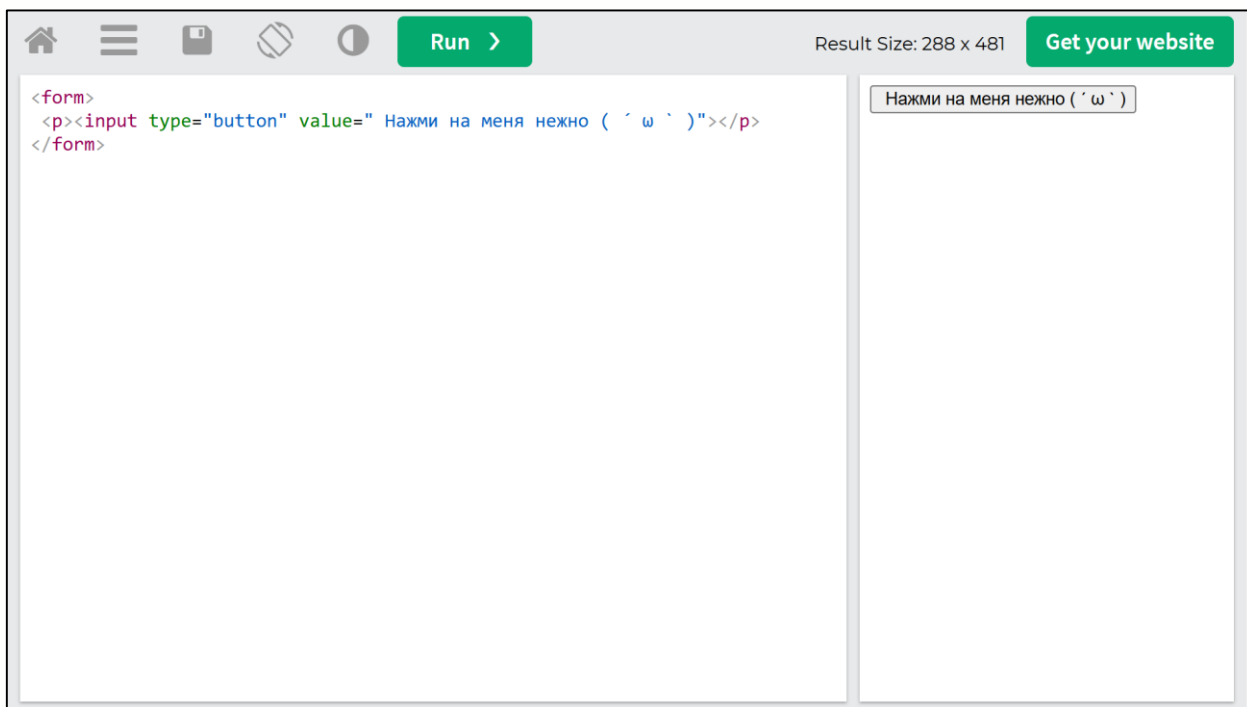
Кнопку на веб-странице можно создать двумя способами — с помощью тега `<input>` и тега `<button>` (более современный вариант).

Рассмотрим сначала добавление кнопки через использование `<input>`. Данный тег является непарным имеет атрибуты, описанные ниже.

Атрибут	Описание
name	Имя кнопки, предназначено для того, чтобы обработчик формы мог его идентифицировать.
value	Значение кнопки и одновременно надпись на ней.

Пример 1:

```
<form>
  <p><input type="button" value=" Нажми на меня нежно ( ´ω` )"></p>
</form>
```



Полезное:

- Пробелы в надписи на кнопке, в отличие от текста HTML, учитываются, поэтому можно ставить любое количество пробелов, которые в итоге влияют на ширину кнопки.

Второй способ создания кнопки основан на использовании тега **<button>**. Он по своему действию напоминает результат, получаемый с помощью тега **<input>**. Но в отличие от него предлагает **расширенные возможности** по созданию кнопок. Например, на подобной кнопке можно размещать любые элементы HTML включая изображения и таблицы.

Атрибуты перечислены в таблице выше, но в отличие от кнопки **<input>** атрибут **value** определяет только отправляемое на сервер значение, а не надпись на кнопке. Если требуется вывести на кнопке изображение, то тег **** добавляется между тегами **<button>**. Полный список доступный атрибутов представлен в таблице ниже.

Атрибут	Описание
name	Имя кнопки, предназначено для того, чтобы обработчик формы мог его идентифицировать.
value	Определяет только отправляемое на сервер значение, а не надпись на кнопке.
type	Значения: • button (по умолчанию) — обычная кнопка без специального действия. • submit — отправляет данные формы (аналог <code><input type="submit"></code>). • reset — сбрасывает поля формы к значениям по умолчанию.
formtarget	Указывает, где открыть ответ формы: _blank - Загружает страницу в новое окно браузера. _self - Загружает страницу в текущее окно. _parent - Загружает страницу во фрейм-родитель, если фреймов нет, то это значение работает как _self. _top - Отменяет все фреймы и загружает страницу в полном окне браузера, если фреймов нет, то это значение работает как _self.

formnovalidate	Отключает валидацию формы (например, проверку email/required). Работает только с type="submit"
formmethod	Переопределяет метод отправки формы (GET или POST). Работает только с type="submit".
formenctype	Указывает, как кодируются данные формы при отправке. Работает только с type="submit". Значения: • application/x-www-form-urlencoded (по умолчанию). • multipart/form-data (для файлов). • text/plain (простой текст).
formaction	Переопределяет атрибут action формы (куда отправляются данные). Работает только для type="submit".
form	Связывает кнопку с формой по её id, даже если кнопка находится вне <form>.
autofocus	Устанавливает фокус на кнопку при загрузке страницы. Полезно для главных действий (например, "Сохранить").
disabled	Блокирует кнопку: её нельзя нажать, она не отправляет данные формы. Не требует значения (достаточно указать атрибут).

Для отправки данных на сервер предназначена специальная кнопка **<submit>**. Её вид ничем не отличается от обычных кнопок, но при нажатии на нее происходит **выполнение серверной программы**, указанной атрибутом action тега <form>. Эта программа, называемая еще обработчиком формы, получает данные, введенные пользователем в полях формы, производит с ними необходимые манипуляции, после чего возвращает результат в виде HTML-документа. Что именно делает обработчик, зависит от автора сайта, например, подобная технология применяется при создании опросов, форумов, тестов и многих других вещей.

Синтаксис создания кнопки <submit> зависит от используемого тега <input> или <button>.

Пример 2:

```
<input type="submit" атрибуты>  
<button type="submit">Надпись на кнопке</button>
```

При нажатии на кнопку **<reset>** данные формы возвращаются в первоначальное значение. Как правило, эту кнопку применяют для **очистки** введенной в полях формы информации. Для больших форм от использования кнопки **<reset>** лучше вообще отказаться, чтобы по ошибке на нее не нажать, ведь тогда придется заполнять форму заново.

Синтаксис создания указанной кнопки прост и похож на другие кнопки.

Пример 3:

```
<input type="reset" атрибуты>  
<button type="reset">Надпись на кнопке</button>
```

Значение кнопки **<reset>** никогда не пересылается на сервер. Если надпись на кнопке опустить, иными словами, не задавать атрибут **value**, на кнопке по умолчанию будет добавлен текст «Очистить».

9.5 Переключатели

Переключатели (жарг. радиокнопки) используют, когда необходимо выбрать один единственный вариант из нескольких предложенных. Создаются следующим образом.

Полный список доступный атрибутов для переключателей (радиокнопок) представлен в таблице ниже.

Атрибут	Описание
name	Имя группы переключателей для идентификации поля. Поскольку переключатели являются групповыми элементами, то имя у всех элементов группы должно быть одинаковым.
value	Задаёт, какое значение будет отправлено на сервер. Здесь уже каждый элемент должен иметь свое уникальное значение, чтобы можно было идентифицировать, какой пункт был выбран пользователем.
checked	Предварительное выделение переключателя. По определению, набор переключателей может иметь только один выделенный пункт, поэтому добавление checked сразу к нескольким полям не даст особого результата. В любом случае, будет отмечен элемент, находящийся в коде HTML последним.
disabled	Делает радиокнопку недоступной для выбора.
form	Связывает радиокнопку с формой по её id (если кнопка вне <form>).
required	Требует, чтобы одна из кнопок в группе была выбрана перед отправкой.
autofocus	Устанавливает фокус на радиокнопку при загрузке страницы.
accesskey	Задаёт горячую клавишу для быстрого выбора.
tabindex	Определяет порядок перехода по Tab.
aria-label	Описывает назначение кнопки для скринридеров.

aria-labelledby	Связывает радиокнопку с элементом, который её описывает.
aria-checked	Указывает состояние выбора (true/false). Нужно для динамического управления.

Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Переключатели</title>
</head>

<body>
  <form action="handler.php">
    <p><b>Какое у вас состояние разума?</b></p>
    <p><input name="dzen" type="radio" value="nedzen"> Не дзен</p>
    <p><input name="dzen" type="radio" value="dzen"> Дзен</p>
    <p><input name="dzen" type="radio" value="pdzen" checked> Полный
дзен</p>
    <p><input type="submit" value="Выбрать"></p>
  </form>
</body>

</html>
```

The screenshot shows a web browser interface. On the left, the raw HTML code is displayed, matching the example provided. On the right, the rendered web page is shown. The page has a title bar with a 'Run' button and a 'Get your website' button. The main content area displays the form: 'Какое у вас состояние разума?' followed by three radio buttons. The first two are 'Не дзен' and 'Дзен'. The third is 'Полный дзен' and is selected (indicated by a blue dot). Below the radio buttons is a 'Выбрать' button.

Для создания радиокнопки, так же, как и чекбокса, используются два тега:

- `<input>` с указанием `type="radio"`. Обязательным атрибутом является `name`, значением которого является имя. Данное имя должно быть одинаковым у всей группы радиокнопок. Без этого атрибута будет возможно выбрать все значения сразу, так как браузер не будет видеть связи между ними
- `<label>`, в котором будет текст, связанный с нужной нам радиокнопкой

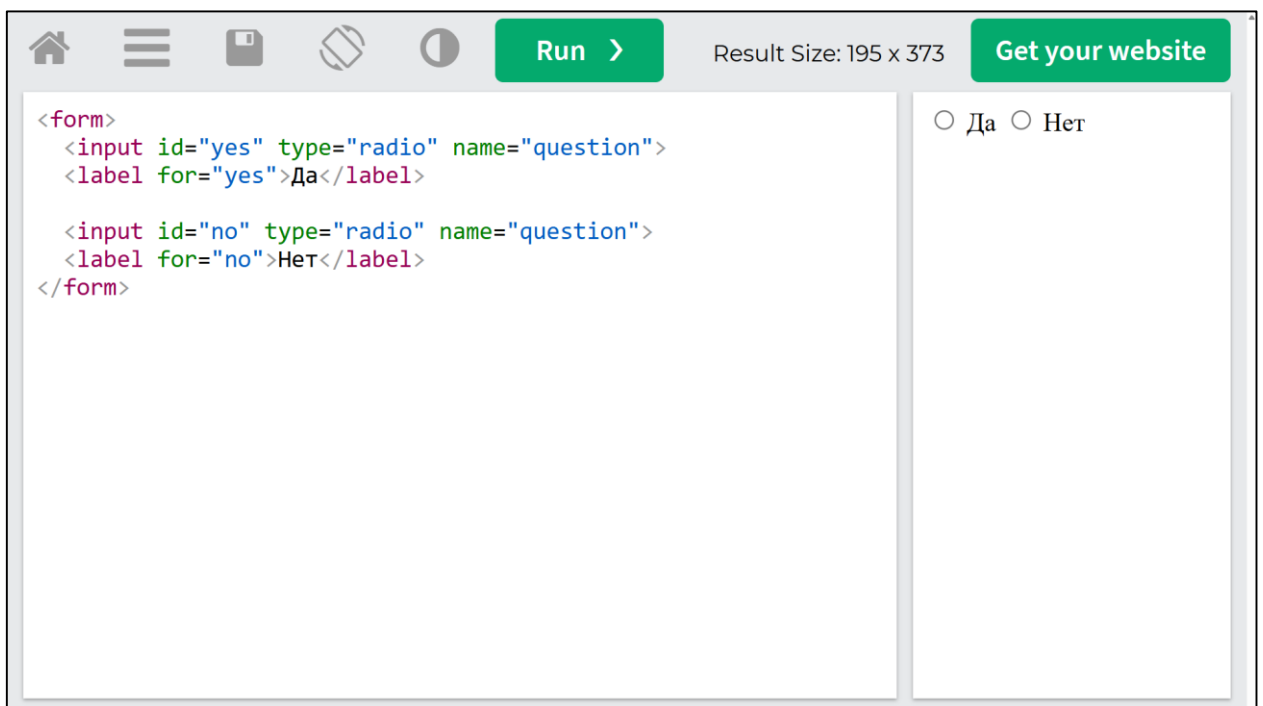
Связь `<input>` с `<label>` происходит уже по одному из двух знакомых нам сценариев:

- Связь по `id`. Для этого необходимо задать уникальный `id` для `<input>`, и связать `<label>` с радиокнопкой с помощью атрибута `for`.

Пример 2:

```
<form>
  <input id="yes" type="radio" name="question">
  <label for="yes">Да</label>

  <input id="no" type="radio" name="question">
  <label for="no">Нет</label>
</form>
```



- Вложить `<input>` внутрь тега `<label>`. При этом указание уникального `id` не требуется.

Пример 3:

```
<form>
  <label>
    <input type="radio" name="question">
    Да
  </label>
  <br>
  <label>
    <input type="radio" name="question">
    Нет
  </label>
</form>
```



Для того чтобы после отправки формы на сервер возможно было узнать, какой именно пункт был выбран, используется атрибут `value`, внутри которого находится значение, позволяющее определить радиокнопку.

Пример 4:

```
<form>
  <label>
    <input type="radio" name="question" value="yes">
    Да
  </label>
  <br>
  <label>
    <input type="radio" name="question" value="no">
    Нет
  </label>
</form>
```



9.6 Флажки

Флажки (жарг. чекбоксы) используют, когда необходимо выбрать любое количество вариантов из предложенного списка. Если требуется выбор лишь одного варианта, то для этого следует предпочесть переключатели (radiobutton). Флажок создается следующим образом.

Полный список доступный атрибутов для флажков (чекбоксов) аналогичен атрибутам [переключателей](#).

Пример 4:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Флажки</title>
</head>

<body>
  <form action="handler.php">
    <p>В каких годах произошли самые известные извержения вулкана
    Кракатау?</p>
    <p><input type="checkbox" name="a" value="1417"> 1417</p>
    <p><input type="checkbox" name="a" value="1680" checked> 1680</p>
    <p><input type="checkbox" name="a" value="1883" checked> 1883</p>
    <p><input type="checkbox" name="a" value="1934"> 1934</p>
    <p><input type="checkbox" name="a" value="2010"> 2010</p>
    <p><input type="submit" value="Отправить"></p>
  </form>
</body>

</html>
```



Run >

Result Size: 285 x 558

Get your website

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Флажки</title>
  </head>
  <body>
    <form action="handler.php">
      <p>В каких годах произошли самые известные извержения вулкана Кракатау?</p>
      <p><input type="checkbox" name="a" value="1417"> 1417</p>
      <p><input type="checkbox" name="a" value="1680" checked> 1680</p>
      <p><input type="checkbox" name="a" value="1883" checked> 1883</p>
      <p><input type="checkbox" name="a" value="1934"> 1934</p>
      <p><input type="checkbox" name="a" value="2010"> 2010</p>
      <p><input type="submit" value="Отправить"></p>
    </form>
  </body>
</html>
```

В каких годах произошли самые известные извержения вулкана Кракатау?

☐ 1417

☒ 1680

☒ 1883

☐ 1934

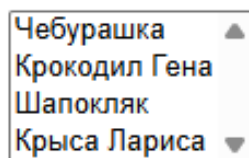
☐ 2010

Отправить

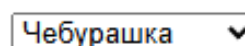
9.7 Поле со списком

Поле со списком, называемое еще ниспадающее меню, один из гибких и удобных элементов формы. В зависимости от настроек, в списке можно выбирать одно или несколько значений. Преимущество списка в его компактности и универсальности, список может занимать одну или несколько строк, в нём можно выбирать одно или несколько значений. Поле со списком создается следующим образом.

Тег `<select>` выступает контейнером для пунктов списка и определяет его вид, будет ли это раскрывающийся список или же список с одним или множественным выбором. Вид зависит от использования атрибута `size` тега `<select>`, который устанавливает высоту списка, ширина списка при этом определяется автоматически исходя из длины текста внутри `<option>`. Ниже представлен список множественного выбора, в котором пункты выделяются с помощью клавиши Ctrl и Shift и раскрывающийся список.



Список множественного выбора



Раскрывающийся список

Ниже представлены примеры кода данных списков. Пример 1 – список множественного выбора. Пример 2 – раскрывающийся список.

Пример 1:

```
<select multiple="multiple">
  <option>Чебурашка</option>
  <option>Крокодил Гена</option>
  <option>Шапокляк</option>
  <option>Крыса Лариса</option>
</select>
```

Пример 2:

```
<select size="1">
  <option>Чебурашка</option>
  <option>Крокодил Гена</option>
  <option>Шапокляк</option>
  <option>Крыса Лариса</option>
</select>
```

Если в коде списка со множественным выбором убрать атрибут `multiple`, то автоматически будет выставлен атрибут `size="1"` и список превратится в раскрывающийся (такой как представленный в примере 2).

Ниже представленная таблица атрибутов для `<select>`, используемых при создании списков.

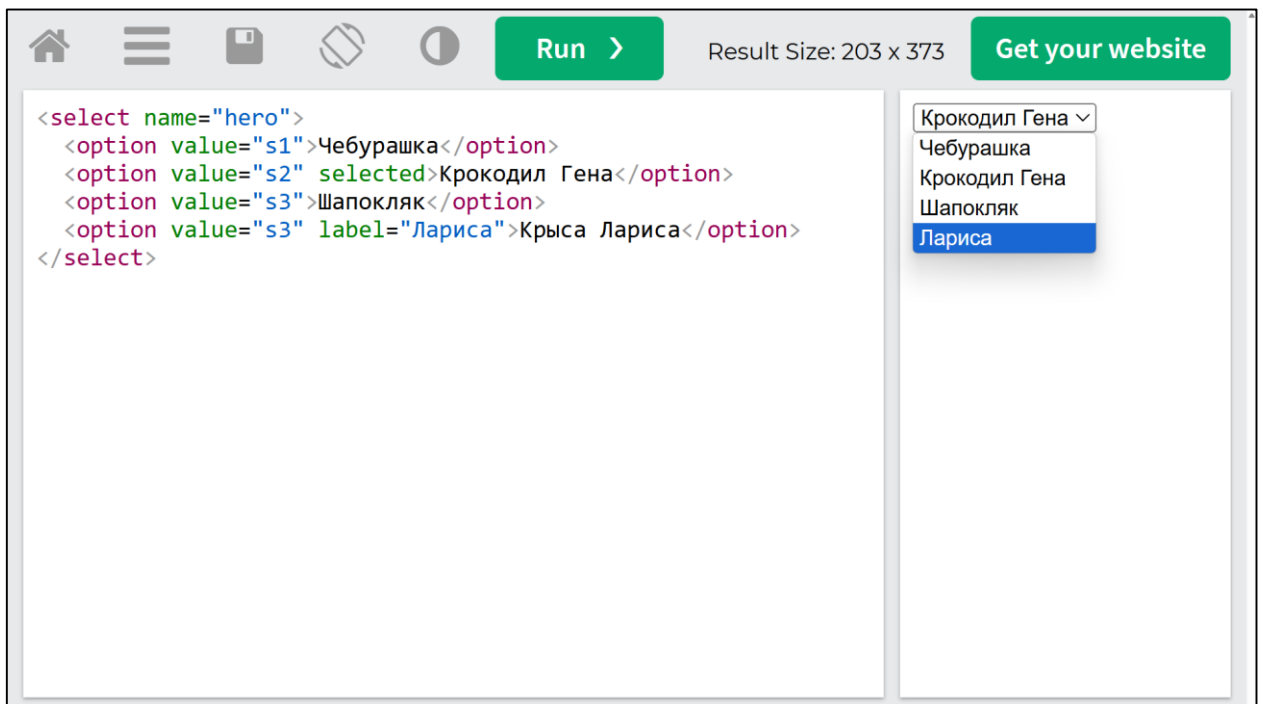
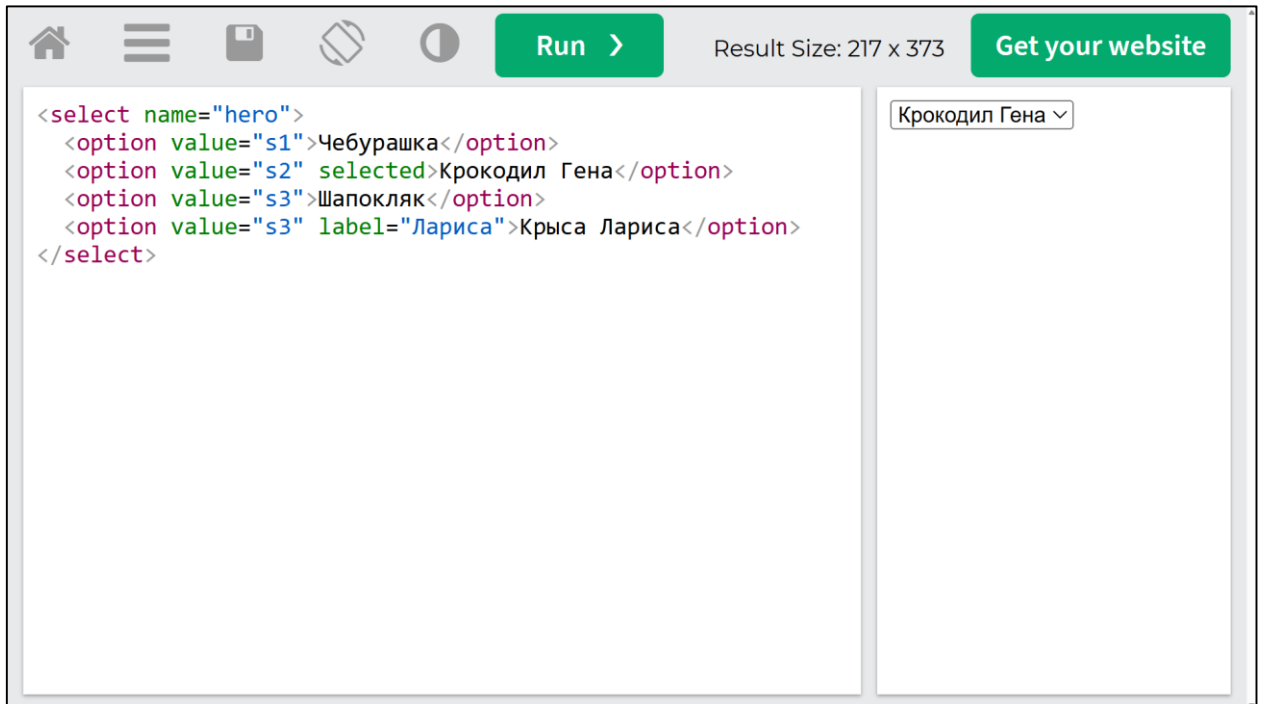
Атрибут	Описание
name	Имя списка, отправляемое на сервер.
multiple	Позволяет выбирать несколько вариантов (с зажатым Ctrl или Shift).
size	Количество видимых строк списка.
disabled	Блокирует выбор элемента списка.
form	Связывает <code><select></code> с формой по её id.
required	Обязателен для выбора перед отправкой.
autofocus	Фокус на списке при загрузке.

Помимо своего каркаса, в виде парного тега `<select>` список также состоит из элементов записанных внутри парного тега `<option>`. Ниже представлена таблица с атрибутами для тега элементов списка

Атрибут	Описание
value	Значение, отправляемое на сервер.
selected	Делает текущий пункт списка выделенным. Если у тега <code><select></code> добавлен атрибут <code>multiple</code> , то можно выделять более одного пункта. Выбрано по умолчанию.
disabled	Не позволяет выбрать элемент.
label	Альтернативный текст (для <code><optgroup></code>).

Пример 3:

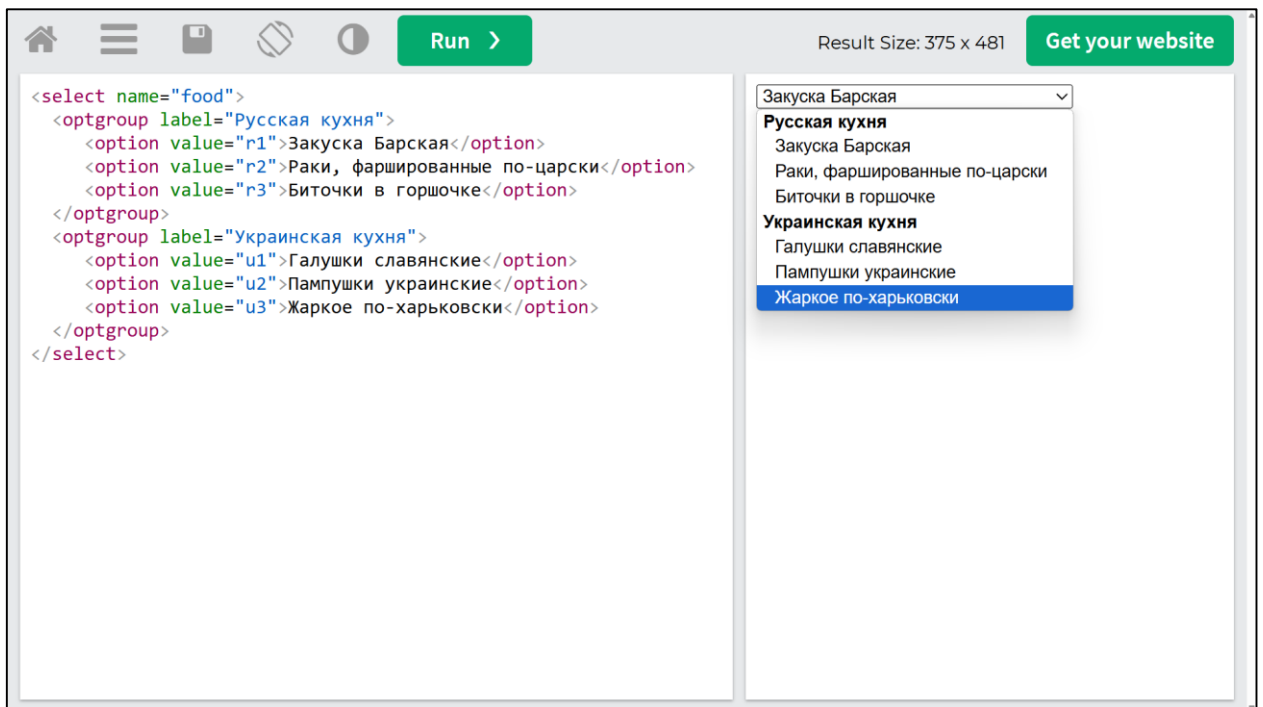
```
<select name="hero">
  <option value="s1">Чебурашка</option>
  <option value="s2" selected>Крокодил Гена</option>
  <option value="s3">Шапокляк</option>
  <option value="s3" label="Лариса">Крыса Лариса</option>
</select>
```



Списки в HTML также поддерживают группирование элементов. Это нужно, чтобы обеспечить наглядность списка и удобство работы с ним. Для этой цели применяется тег `<optgroup>`. Он представляет собой контейнер, внутри которого располагаются теги `<option>` объединённые в одну группу. Особенностью тега `<optgroup>` является то, что он не выделяется как обычный элемент списка, акцентируется с помощью жирного начертания, а все элементы, входящие в этот контейнер, смещаются вправо от своего исходного положения. Для добавления заголовка группы используется атрибут `label`.

Пример 4:

```
<select name="food">
  <optgroup label="Русская кухня">
    <option value="r1">Закуска Барская</option>
    <option value="r2">Раки, фаршированные по-царски</option>
    <option value="r3">Биточки в горшочке</option>
  </optgroup>
  <optgroup label="Украинская кухня">
    <option value="u1">Галушки славянские</option>
    <option value="u2">Пампушки украинские</option>
    <option value="u3">Жаркое по-харьковски</option>
  </optgroup>
</select>
```



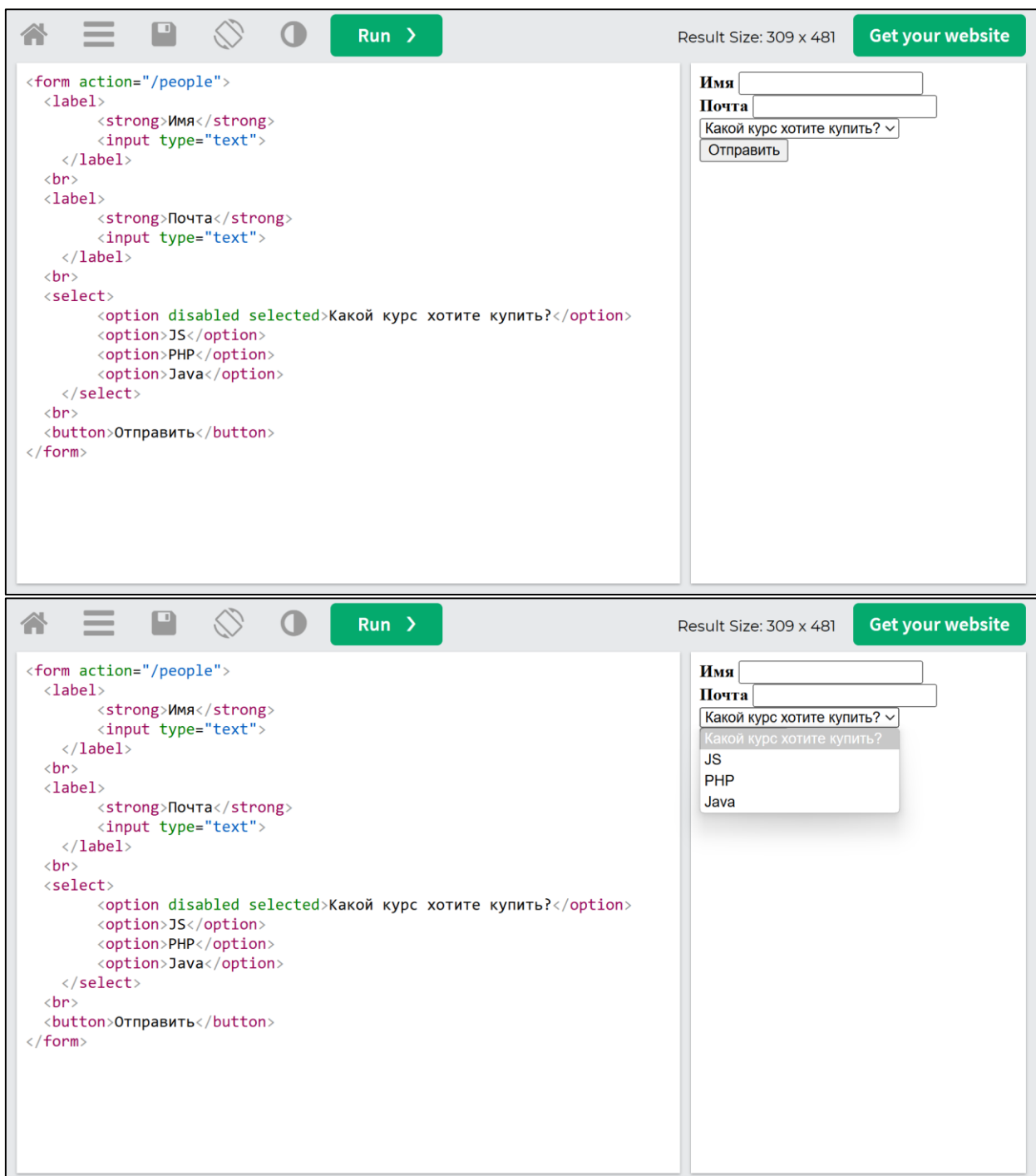
Ниже приведена таблица с атрибутами для парного тега <optgroup>, предназначенного для объединения схожих элементов в соответствующие им группы.

Атрибут	Описание
label	Название группы (его использование обязательно).
disabled	Делает всю группу недоступной для выбора.

Хотел бы дополнить главу тем, что нашел совершенно случайно. Думаю, многие знакомы с различными онлайн магазинами и знают, что товар можно сортировать по категориям, для более удобной работы-покупок. Конкретно меня интересовало как проверить похожую шутку как с обычным полем, когда мы писали атрибут placeholder, но только для раскрывающегося списка, при условии, что такая подсказка не пропадала бы при выборе какого-либо элемента этого самого списка. Следующий пример будет показывать реализацию данной вещи, которая безусловна является полезной при работе с написанием сайта.

Пример 5:

```
<form action="/people">
  <label>
    <strong>Имя</strong>
    <input type="text">
  </label>
  <br>
  <label>
    <strong>Почта</strong>
    <input type="text">
  </label>
  <br>
  <select>
    <option disabled selected>Какой курс хотите купить?</option>
    <option>JS</option>
    <option>PHP</option>
    <option>Java</option>
  </select>
  <br>
  <button>Отправить</button>
</form>
```



Как можно видеть из примера данная вещь реализуется путем блокировки элемента, которым она и является, а, чтобы она отображалась в закрытом списке первой нужно поставить атрибут `selected`.

Как небольшое уточнение оставлю то, что поле для почты в дальнейшем будет записываться по-другому, она сменит свой тип с `text` на `email`, но подробнее об этом в одной из следующих глав данного раздела.

9.8 Скрытое поле

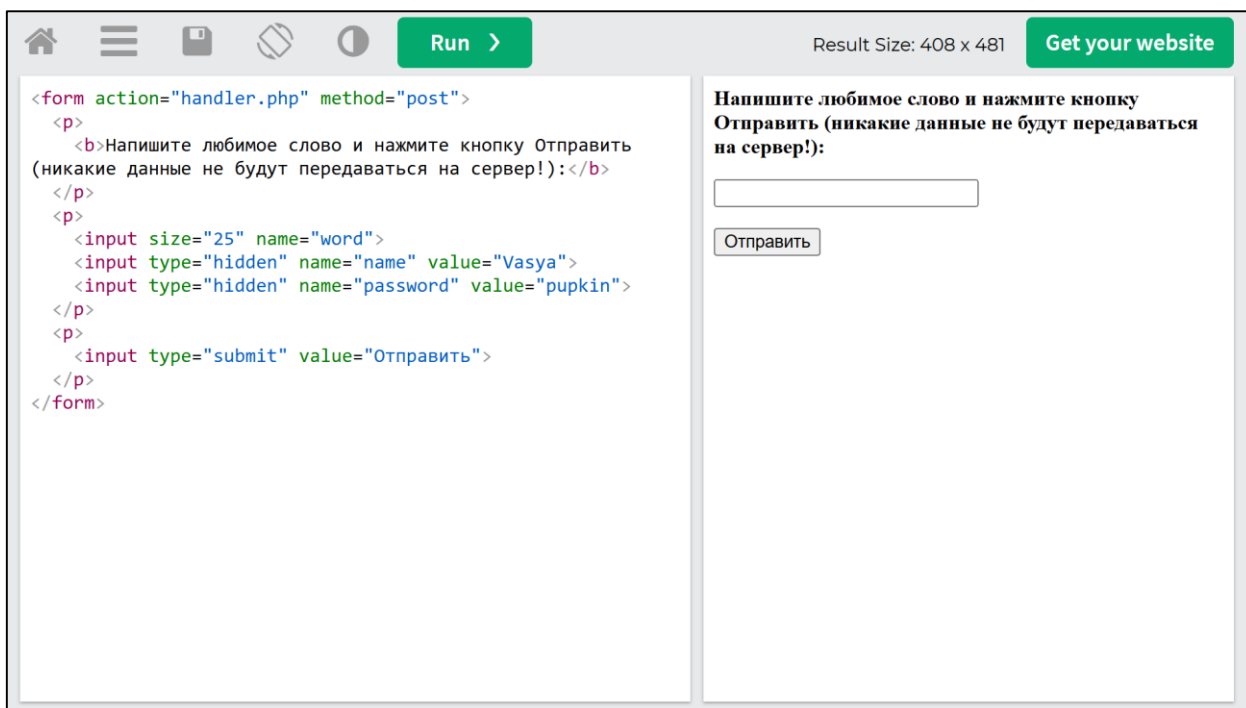
Часто возникает ситуация, когда требуется передать в форме некоторые промежуточные данные, которые не должны изменяться пользователем. Более того, такие данные не должны показываться пользователю, поскольку носят технический характер и обычно служат для передачи некоторой информации от страницы к странице. Для этой цели применяется скрытое поле, оно не отображается на странице и прячет своё содержимое от пользователя. Посетитель не может в него ничего внести или напечатать.

Ниже приведена таблица с обязательными атрибутами для реализации скрытого поля.

Атрибут	Описание
name	Имя поля для его идентификации обработчиком формы.
value	Значение поля, определяющее, какая информация будет отправлена на сервер.

Пример 1:

```
<form action="handler.php" method="post">
  <p>
    <b>Напишите любимое слово и нажмите кнопку Отправить (никакие
данные не будут передаваться на сервер!)</b>
  </p>
  <p>
    <input size="25" name="word">
    <input type="hidden" name="name" value="Vasya">
    <input type="hidden" name="password" value="pupkin">
  </p>
  <p>
    <input type="submit" value="Отправить">
  </p>
</form>
```



В данном примере показано создание двух скрытых полей, одно из них носит имя name и получает значение Vasya, а второе именуется password со значением pupkin. В результате отправки формы обработчику, указанному в атрибуте action, программа может легко прочесть эти данные и интерпретировать их по усмотрению разработчика.

Скрытые поля - важный инструмент для передачи служебной информации между клиентом и сервером без отображения в интерфейсе.

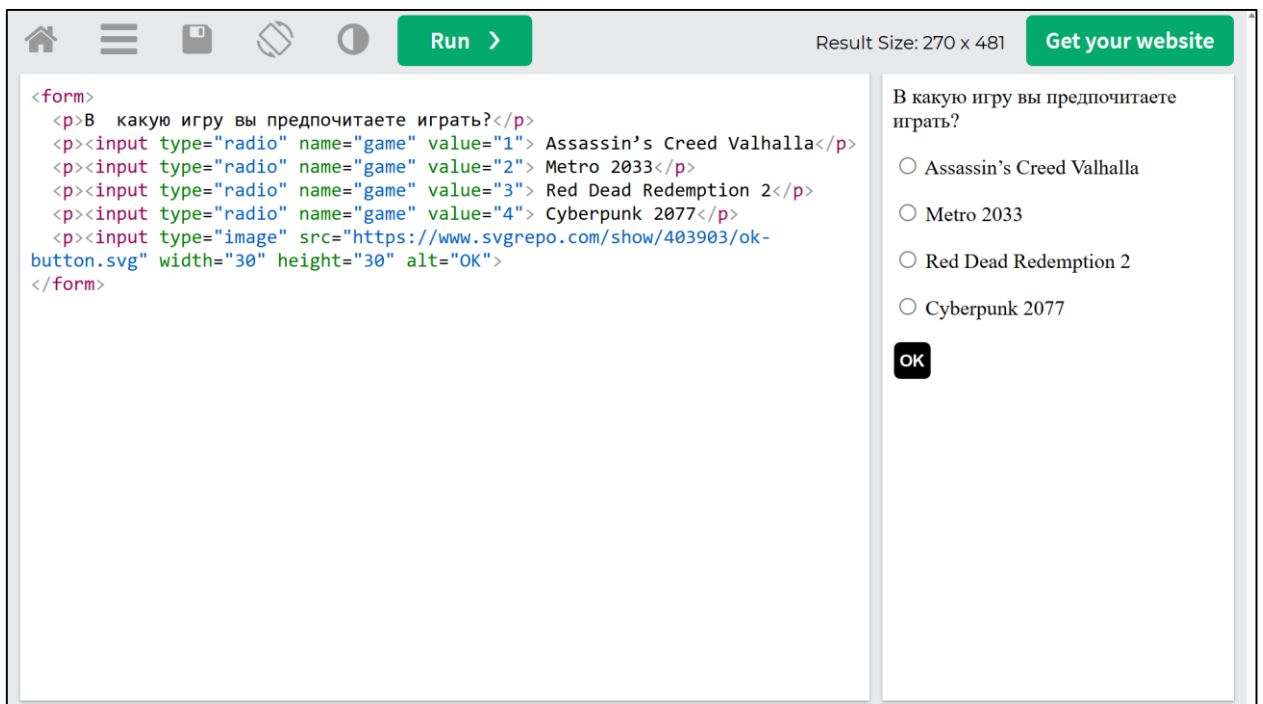
9.9 Поле с изображением

Для отправки формы на сервер применяется кнопка Submit, которая имеет слабые средства по настройке своего вида. Чтобы не ограничивать дизайн и расширить возможности по оформлению форм, используйте поле с изображением, аналогичное по своему действию кнопке Submit. При нажатии на рисунок данные формы отправляются на сервер и обрабатываются программой, заданной атрибутом action тега <form>.

Здесь URL это адрес изображения в формате JPEG, PNG или GIF, alt указывает альтернативный текст, который виден при отключении картинок в браузере. Вообще, это поле похоже на добавление в код изображения и работает подобно элементу .

Пример 1:

```
<form>
  <p>В какую игру вы предпочитаете играть?</p>
  <p><input type="radio" name="game" value="1"> Assassin's Creed
Valhalla</p>
  <p><input type="radio" name="game" value="2"> Metro 2033</p>
  <p><input type="radio" name="game" value="3"> Red Dead Redemption
2</p>
  <p><input type="radio" name="game" value="4"> Cyberpunk 2077</p>
  <p><input type="image" src="https://www.svgrepo.com/show/403903/ok-
button.svg" width="30" height="30" alt="OK">
</form>
```



В отличие от кнопки Submit на сервер также отправляются координаты точки щелчка по изображению в виде x и y. Отсчёт координат ведётся от

левого верхнего угла картинки и указывается в пикселах. Если добавить к полю с изображением уникальное имя через атрибут name, например, ok, тогда координаты передаются в виде ok.x и ok.y, где впереди через точку стоит имя поля.

9.10 Загрузка файлов

Для того чтобы можно было загружать на сервер один или несколько файлов, в форме применяется специальное поле. В браузерах Firefox, IE и Opera такой элемент отображается как текстовое поле, рядом с которым располагается кнопка с надписью «Обзор...» (рис. 1). В Safari и Chrome доступна только кнопка «Выберите файл» (рис. 2).

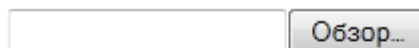


Рис. 1. Вид поля для загрузки файла в Firefox

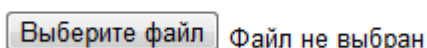


Рис. 2. Загрузка файлов в Chrome

При нажатии на кнопку открывается окно для выбора файла, где можно указать, какой файл пользователь желает использовать.

Таблица, содержащая атрибуты для формирования тега отправки файла представлена ниже.

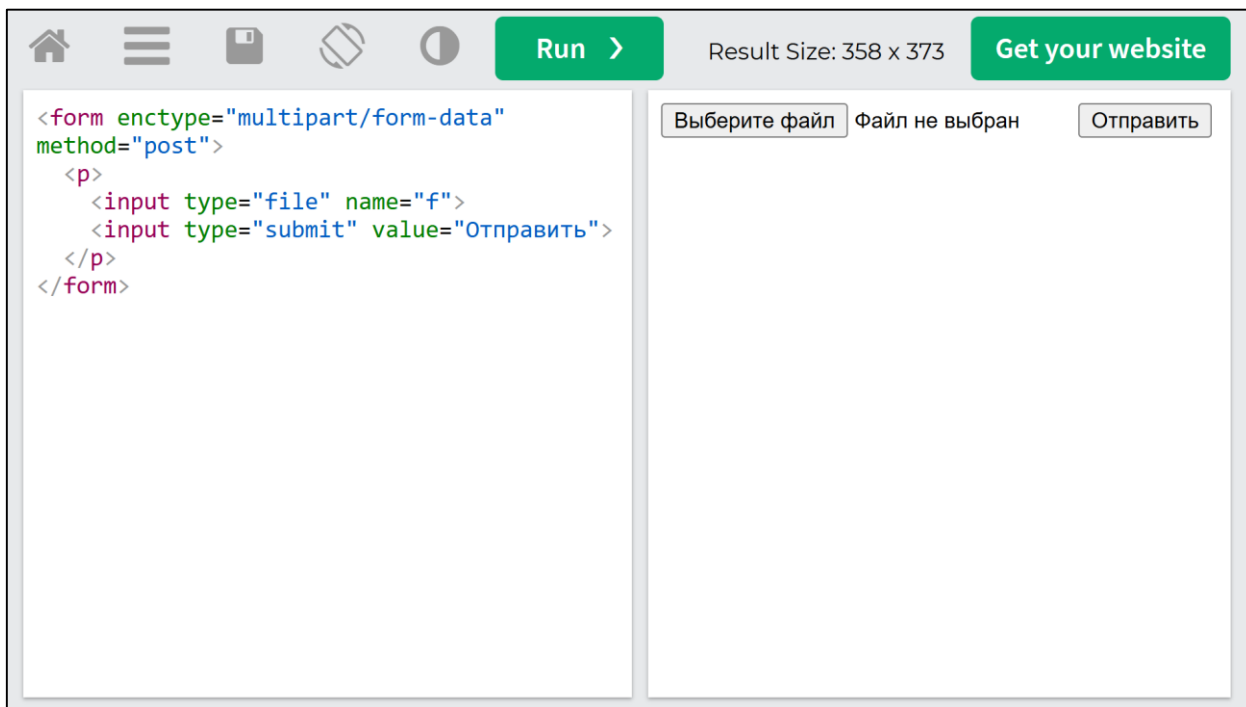
Атрибут	Описание
name	Имя поля, используется для его идентификации обработчиком формы.
multiple	Позволяет выбирать и загружать сразу несколько файлов.
size	Ширина текстового поля, которое определяется числом символов моноширинного шрифта.
accept	Устанавливает фильтр на типы файлов, которые вы можете отправить через поле загрузки файлов.
method	Принимает значение post.
enctype	Принимает значение multipart/form-data.

Прежде, чем использовать данное поле, в форме необходимо сделать следующее:

- задать метод отправки данных POST (method="post");
- установить у атрибута enctype значение multipart/form-data.

Пример 1:

```
<form enctype="multipart/form-data" method="post">
  <p>
    <input type="file" name="f">
    <input type="submit" value="Отправить">
  </p>
</form>
```



Хотя можно установить ширину поля через атрибут **size**, в действительности ширина никак не влияет на результат работы формы. В браузерах Safari и Chrome этот атрибут вообще никакого воздействия не оказывает.

Атрибут **multiple** более важен, он позволяет не ограничиваться одним файлом для выбора, а указать их сразу несколько для одновременной загрузки.

Если атрибут **accept** не указывать, тогда добавляются и загружаются файлы любого типа. Наличие **accept** позволяет ограничить выбор файла, что особенно важно, когда требуется загрузить только изображение или видео. В качестве значения выступает MIME-тип, несколько значений разделяются между собой запятой. Также можно использовать следующие ключевые слова:

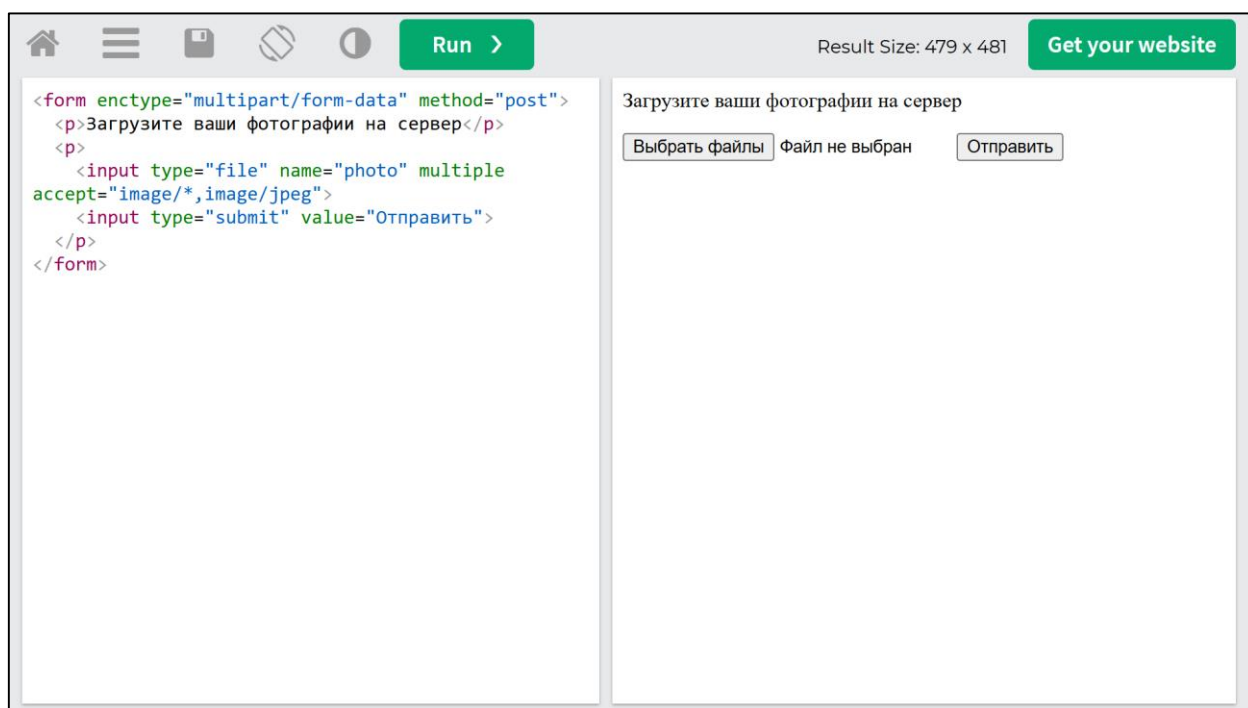
- audio/* — выбор музыкальных файлов любого типа;
- image/* — графические файлы;
- video/* — видеофайлы.

В таблице ниже представлены некоторые допустимые значения атрибута ассерт.

Значение	Описание
image/jpeg	Только файлы в формате JPEG.
image/jpeg,image/png	Только файлы в формате JPEG и PNG.
image/*	Любые графические файлы.
image/*,video/*	Любые графические и видеофайлы.

Пример 2:

```
<form enctype="multipart/form-data" method="post">
  <p>Загрузите ваши фотографии на сервер</p>
  <p>
    <input type="file" name="photo" multiple
accept="image/*,image/jpeg">
    <input type="submit" value="Отправить">
  </p>
</form>
```



Не все браузеры поддерживают новые атрибуты. IE полностью игнорирует multiple и ассерт, Safari не поддерживает ассерт, а Firefox не работает с MIME-типом, только с ключевыми словами. Поэтому в примере выше специально для Firefox установлено значение image/*,image/jpeg. Также

учтите странную ошибку в Опере, она не допускает пробелы после запятой внутри ассерт.

9.11 Адрес электронной почты

В формах часто требуется ввод адреса электронной почты, для чего обычно применяется однострочное текстовое поле. Однако в отличие от него специальное поле для ввода адреса почты позволяет проверять корректность записи введенного адреса.

Атрибуты по большей части совпадают с текстовым полем и приведены в таблице ниже.

Атрибут	Описание
name	Имя поля, предназначено для того, чтобы обработчик формы мог его идентифицировать.
value	Начальный почтовый адрес отображаемый в поле.
size	Ширина поля, которая определяется числом символов моноширинного шрифта. Иными словами, ширина задается количеством близстоящих букв одинаковой ширины по горизонтали.
multiple	Позволяет указывать несколько адресов через запятую.
maxlength	Устанавливает максимальное число символов, которое может быть введено пользователем в поле. Когда это количество достигается при наборе, дальнейший ввод становится невозможным. Если этот атрибут опустить, то можно вводить строку длинее самого поля.

По сравнению со значением `text`, добавлен атрибут `multiple`, который позволяет вводить сразу несколько почтовых адресов.

В примере 1 показано применение значения `email` для создания формы авторизации на сайте, где в качестве логина указывается адрес электронной почты.

Пример 1:

```
<form action="handler.php">
  <p>Вход на сайт</p>
  <p>Email: <input type="email" name="login"></p>
  <p>Пароль: <input type="password" name="pass"></p>
  <p><input type="submit" value="Вход"></p>
</form>
```



Run >

Result Size: 275 x 373

Get your website

```
<form action="handler.php">
  <p>Вход на сайт</p>
  <p>Email: <input type="email" name="login"></p>
  <p>Пароль: <input type="password" name="pass"></p>
  <p><input type="submit" value="Вход"></p>
</form>
```

Вход на сайт

Email:

Пароль:

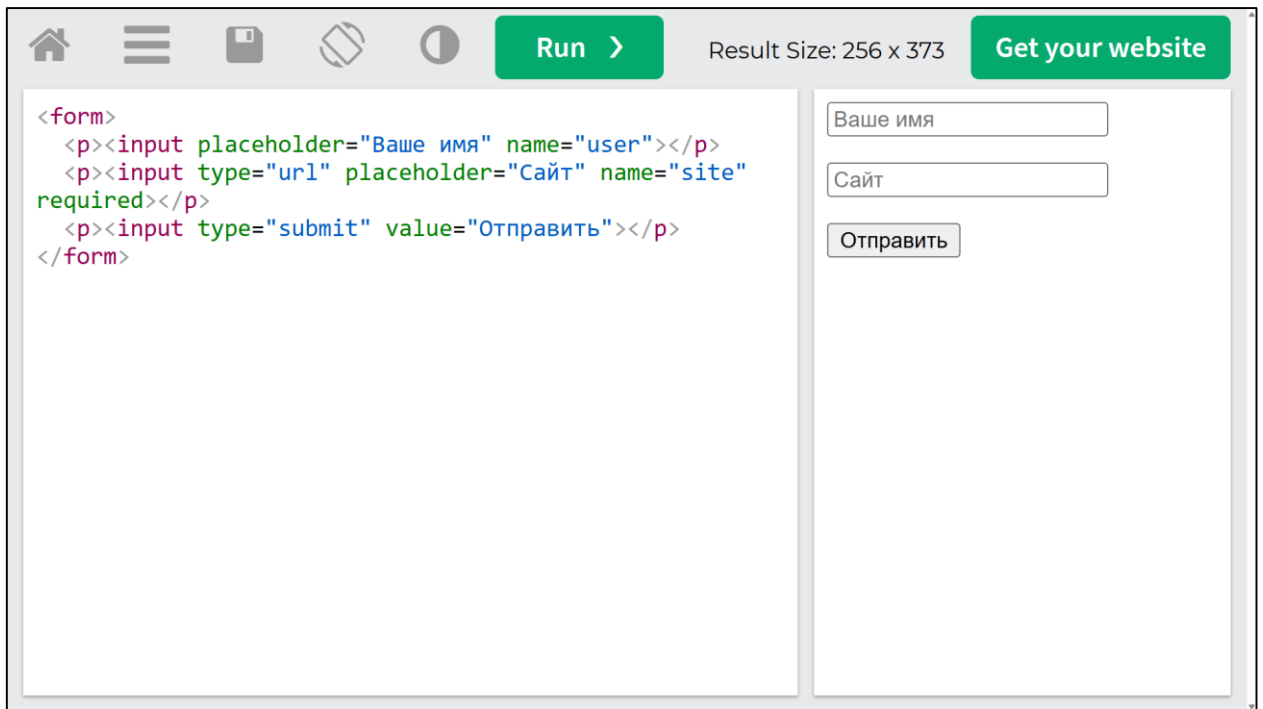
9.12 Веб-адрес

Для ввода адресов сайтов или, как их ещё называют веб-адресов, предназначено значение `url` тега `<input>`, которое делает проверку на правильность ввода данных. Каждый веб-адрес должен начинаться с протокола (`http://`, `https://`, `ftp://`), больше ограничений нет — адрес может быть набран латинскими символами, кириллицей, содержать точку или наоборот, писаться без неё. Браузер Opera не требует даже наличие протокола, подставляя «`http://`» перед текстом автоматически в случае его отсутствия.

Атрибуты совпадают с текстовым полем (`<input type="text">`).

Пример 1:

```
<form>
  <p><input placeholder="Ваше имя" name="user"></p>
  <p><input type="url" placeholder="Сайт" name="site" required></p>
  <p><input type="submit" value="Отправить"></p>
</form>
```



The screenshot shows a web browser window with a toolbar at the top containing icons for home, menu, save, print, and a 'Run' button. To the right of the toolbar, it says 'Result Size: 256 x 373' and 'Get your website'. The main content area displays the rendered HTML form. It consists of three input fields stacked vertically. The first field has the placeholder text 'Ваше имя'. The second field has the placeholder text 'Сайт' and is highlighted with a green border, indicating it is required. The third field is a submit button with the text 'Отправить'.

В данном примере при нажатии на кнопку «Отправить» браузер будет выводить сообщение, что необходимо правильно ввести URL. Вид и текст сообщения зависит от браузера.

Opera не выводит предупреждений при вводе неверного веб-адреса, исправляя его автоматически.

По своему виду поле для веб-адреса не отличается от текстового, за исключением браузера Opera, который несколько увеличивает ширину поля для веб-адреса.

9.13 Выбор цвета

Для выбора шестнадцатеричного значения цвета в формы HTML5 включено специальное поле, которое позволяет указать желаемый цвет.

Атрибут `value` необходим для установки исходного цвета и может быть опущен, name используется для идентификации получаемого значения.

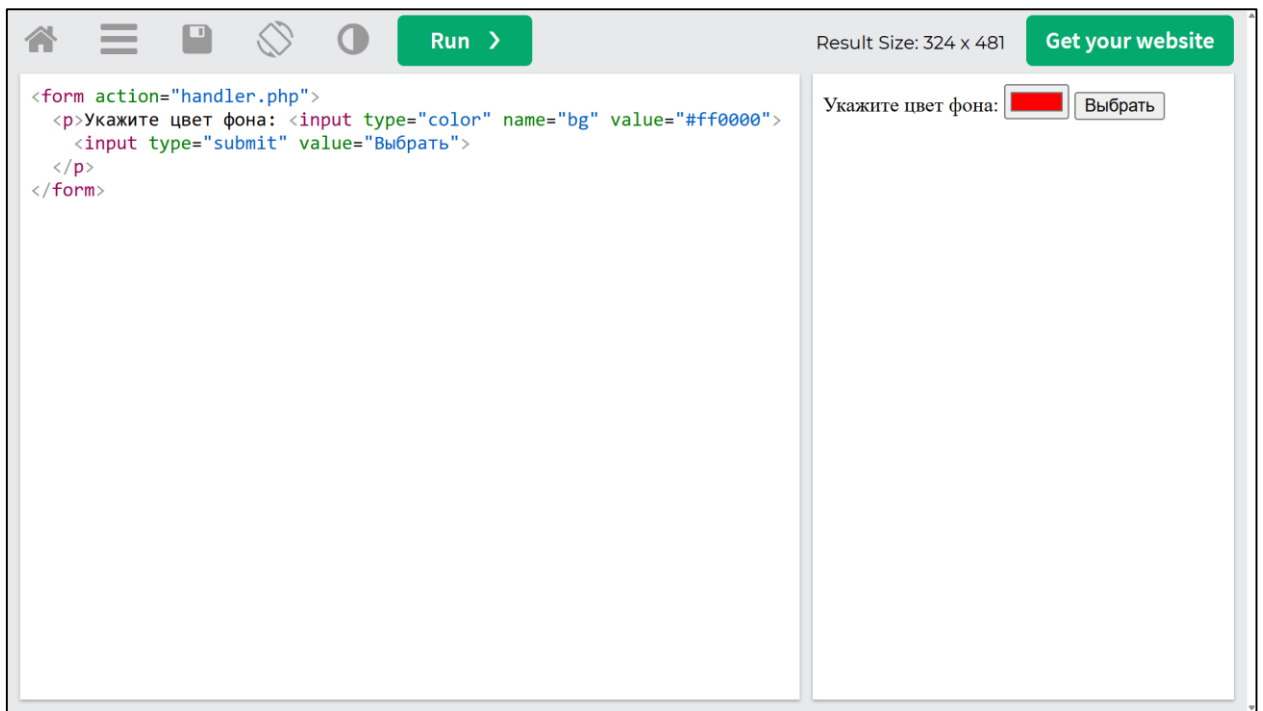
Вид поля для выбора цвета возложен на браузеры и может различаться по своему оформлению. В действительности же пока только Opera и Chrome корректно работает с выбором цвета, остальные браузеры покажут стандартное текстовое поле.

Атрибуты используемые при выборе цвета представлены в таблице ниже.

Атрибут	Описание
name	Имя поля, предназначено для того, чтобы обработчик формы мог его идентифицировать.
value	начальное значение (hex-формат, например #ffffff).
id	идентификатор для связи с <label> или JavaScript.
list	связь с <datalist> пресетов цветов.
disabled	запрещает изменение (но можно выбрать).
required	обязательное поле.
aria-*	атрибуты доступности.

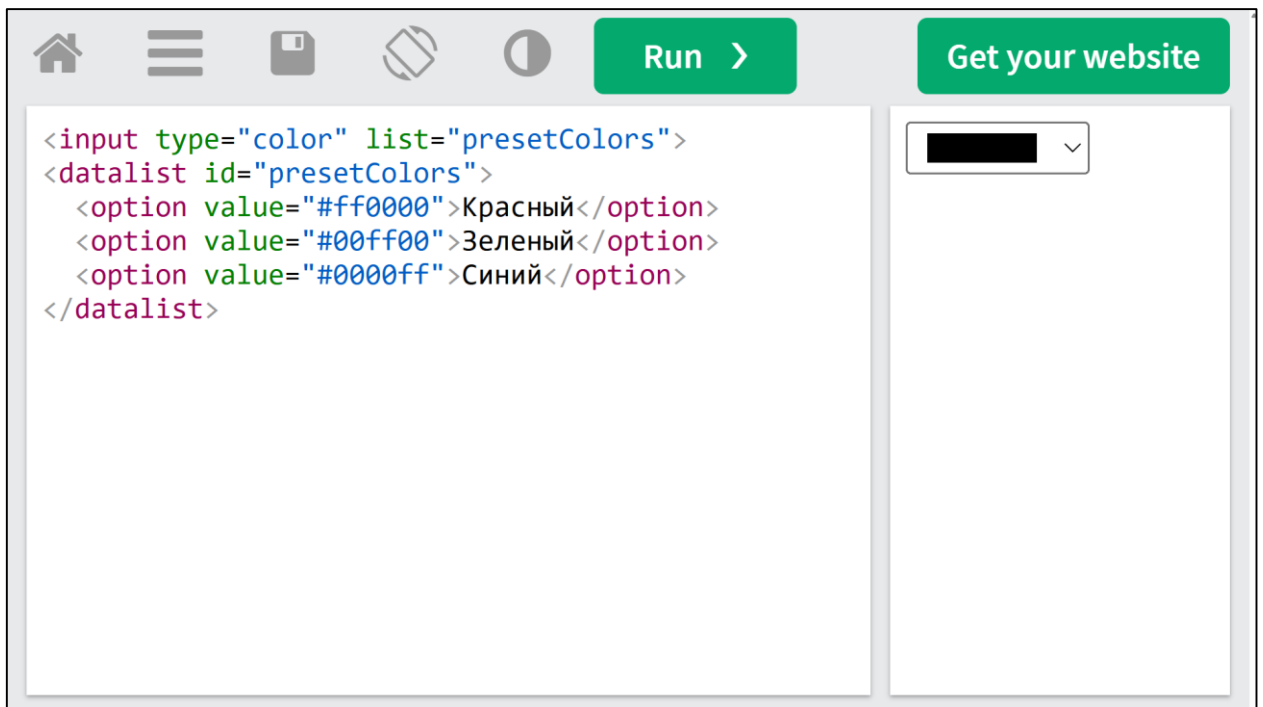
Пример 1:

```
<form action="handler.php">
  <p>Укажите цвет фона: <input type="color" name="bg" value="#ff0000">
    <input type="submit" value="Выбрать">
  </p>
</form>
```



Пример 2:

```
<input type="color" list="presetColors">
<datalist id="presetColors">
  <option value="#ff0000">Красный</option>
  <option value="#00ff00">Зеленый</option>
  <option value="#0000ff">Синий</option>
</datalist>
```



9.14 Ввод чисел

Для ввода чисел предназначено специальное поле, которое допускает ограничения по нижней и верхней границе, а также устанавливает шаг приращения. Само поле для ввода чисел похоже на обычное текстовое поле, но со стрелками, которые позволяют увеличивать и уменьшать значение.

Атрибуты указаны в таблице ниже.

Атрибут	Описание
name	Имя поля, предназначено для того, чтобы обработчик формы мог его идентифицировать.
required	Обязательное поле.
value	Начальное число, которое выводится в поле.
size	Ширина поля.
min	Минимальное значение.
max	Максимальное значение.
step	Шаг приращения числа. Может быть как целым (2), так и дробным (0.2).

Для ограничения введённого числа предназначены атрибуты `min` и `max`, они могут принимать отрицательное и положительное значение. При достижении верхнего или нижнего порога стрелки в поле в зависимости от браузера блокируются или не дают никакого эффекта (пример 1). Несмотря на такие запреты, в любом случае в поле самостоятельно можно вводить любые значения, включая текст. Атрибуты `min` и `max` работают только при использовании стрелок в поле.

Пример 1:

```
<form action="handler.php">
  <p>Введите число от 1 до 10:</p>
  <p><input type="number" size="3" name="num" min="1" max="10"
step="2" value="1"></p>
</form>
```



Run >

Result Size: 197 x 373

Get your website

```
<form action="handler.php">
  <p>Введите число от 1 до 10:</p>
  <p><input type="number" size="3" name="num" min="1" max="10"
step="2" value="1"></p>
</form>
```

Введите число от 1 до 10:

Если значение min превышает max, то атрибут min игнорируется.

9.15 Ползунок

Ползунок предназначен для ввода чисел в указанном диапазоне, но в отличие от поля `<input type="number">` имеет другой интерфейс и применяется в тех случаях, когда не особенно важно указывать точное значение.

Здесь `min` — минимальное число в диапазоне (по умолчанию 0), `max` — максимальное число (по умолчанию 100), `step` — шаг изменения чисел (по умолчанию 1), `value` — текущее значение. По умолчанию `value` вычисляется по формуле:

$$value = \frac{max+min}{2}$$

Если значение `max` меньше, чем значение `min`, то `value` равно `min`.

Атрибуты не являются обязательными, их можно опустить, в таком случае они принимают значения по умолчанию.

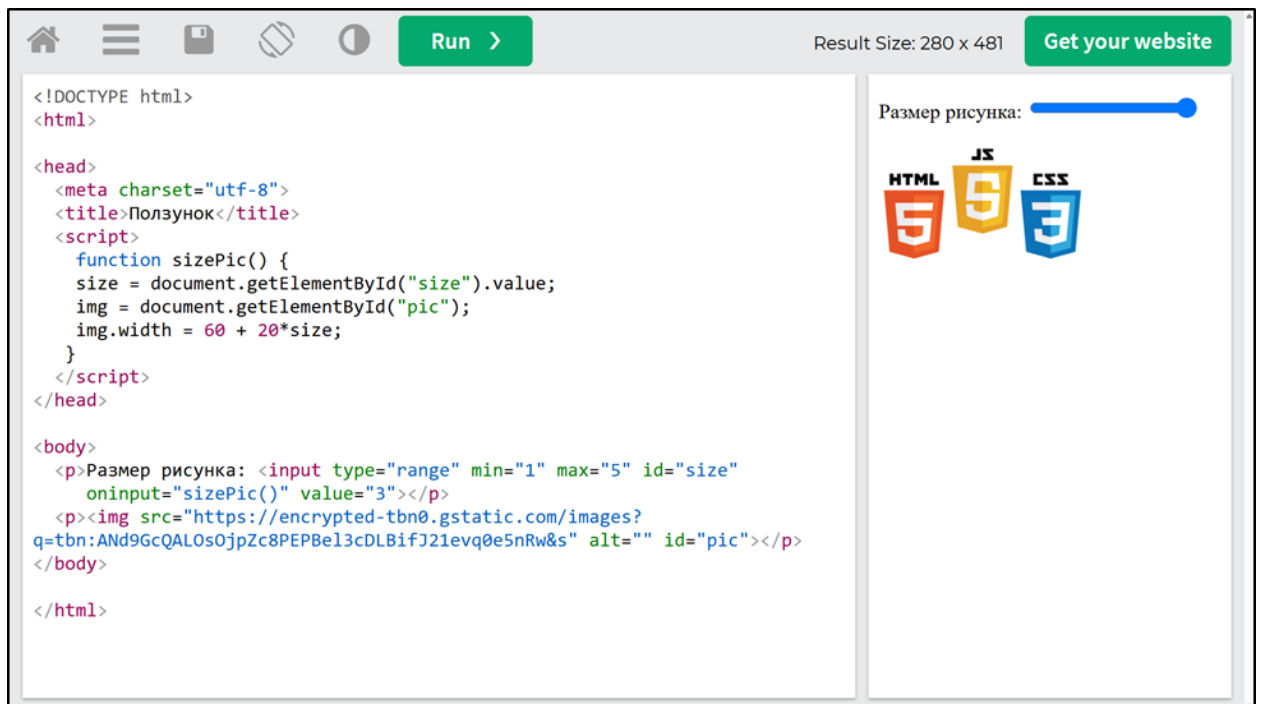
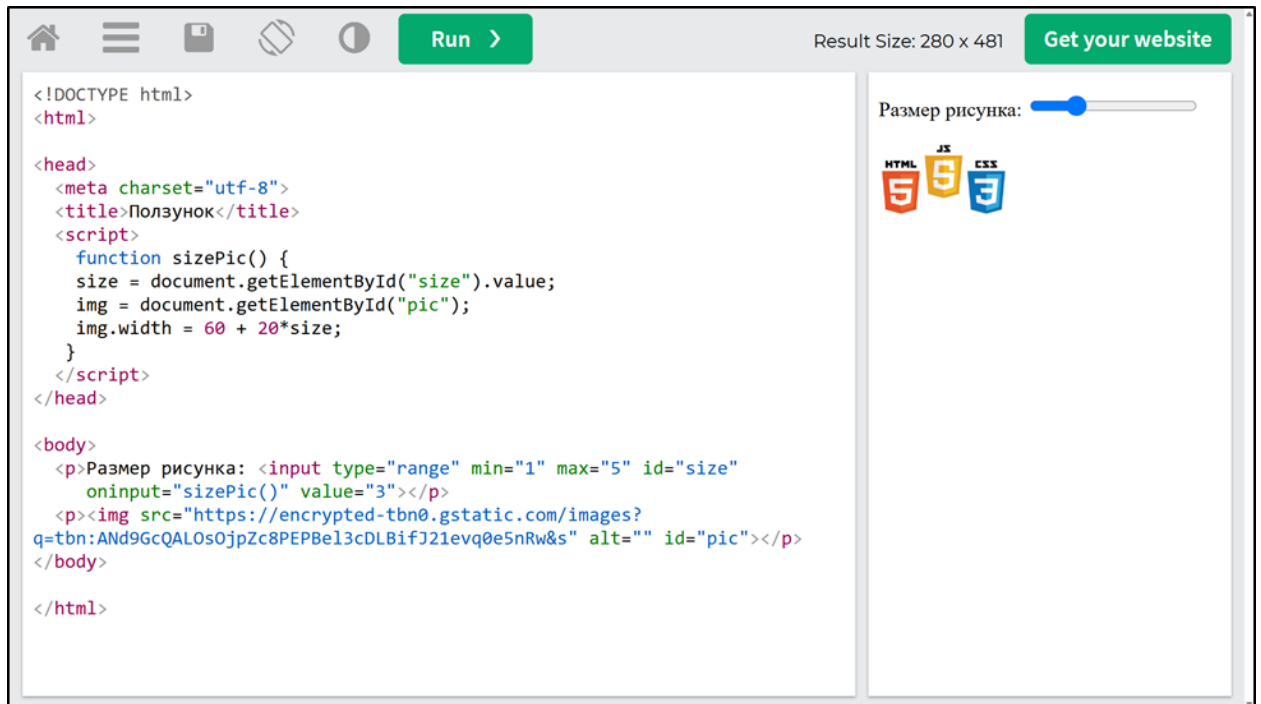
Независимо от минимального и максимального числа ширина ползунка остаётся одинаковой.

Сами ползунки редко применяются в «чистом» виде, поскольку не обеспечивают необходимую обратную связь с пользователем, а вот в сочетании с JavaScript это становится мощным и удобным элементом интерфейса. В примере 1 с помощью ползунка изменяется размер изображения, такая возможность часто используется в различных фотогалереях.

Пример 1:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Ползунок</title>
  <script>
    function sizePic() {
      size = document.getElementById("size").value;
      img = document.getElementById("pic");
      img.width = 60 + 20*size;
    }
  </script>
</head>
<body>
  <p>Размер рисунка: <input type="range" min="1" max="5" id="size"
    oninput="sizePic()" value="3"></p>
  <p>
</p>
```

```
</body>
</html>
```



В данном примере при управлении ползунком срабатывает событие `oninput`, которое вызывает функцию `sizePic`. Эта функция изменяет размер изображения в зависимости от установленного пользователем значения ползунка. Тем самым ширина картинки при желании уменьшается или наоборот, увеличивается.

9.16 Дата и время

Выбор даты применяется на сайтах, торгующих авиа и железнодорожными билетами, ведь посетителя интересует заказ билета на определённый день. Календари также применяются в блогах, где записи систематизируются по дате, и сайтах, связанных с разными событиями, например, спортивными.

На сервер данные передаются в формате ГГГГ-ММ-ДД, например, 22.12.2014, а вид календаря может различаться в зависимости от браузера. Полностью поддерживает календарь пока только Opera, выводя виджет для выбора любой даты (рис. 1).

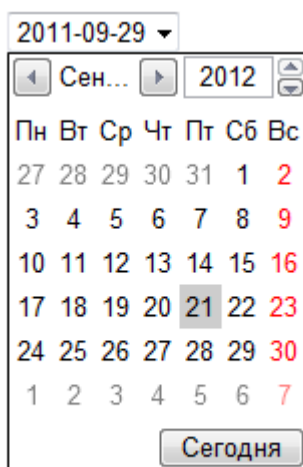


Рис. 1. Календарь в браузере Opera

Браузер Chrome также поддерживает календарь, но делает это весьма скупо (рис. 2). По сути вы только можете прокручивать дату или вводить её как текст.

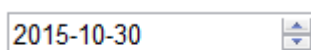


Рис. 2. Календарь в Chrome

Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Календарь</title>
</head>

<body>
  <form>
    <p>Выберите дату: <input type="date" name="calendar">
      <input type="submit" value="Отправить">
    </p>
```

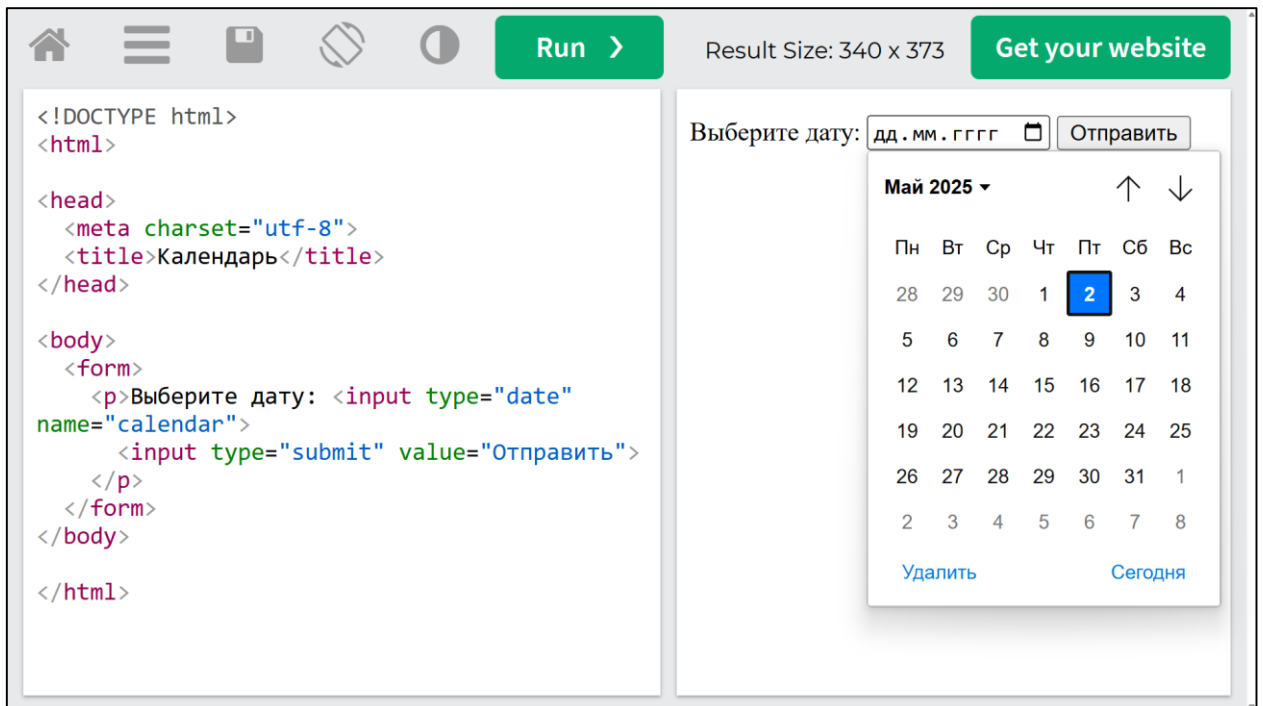
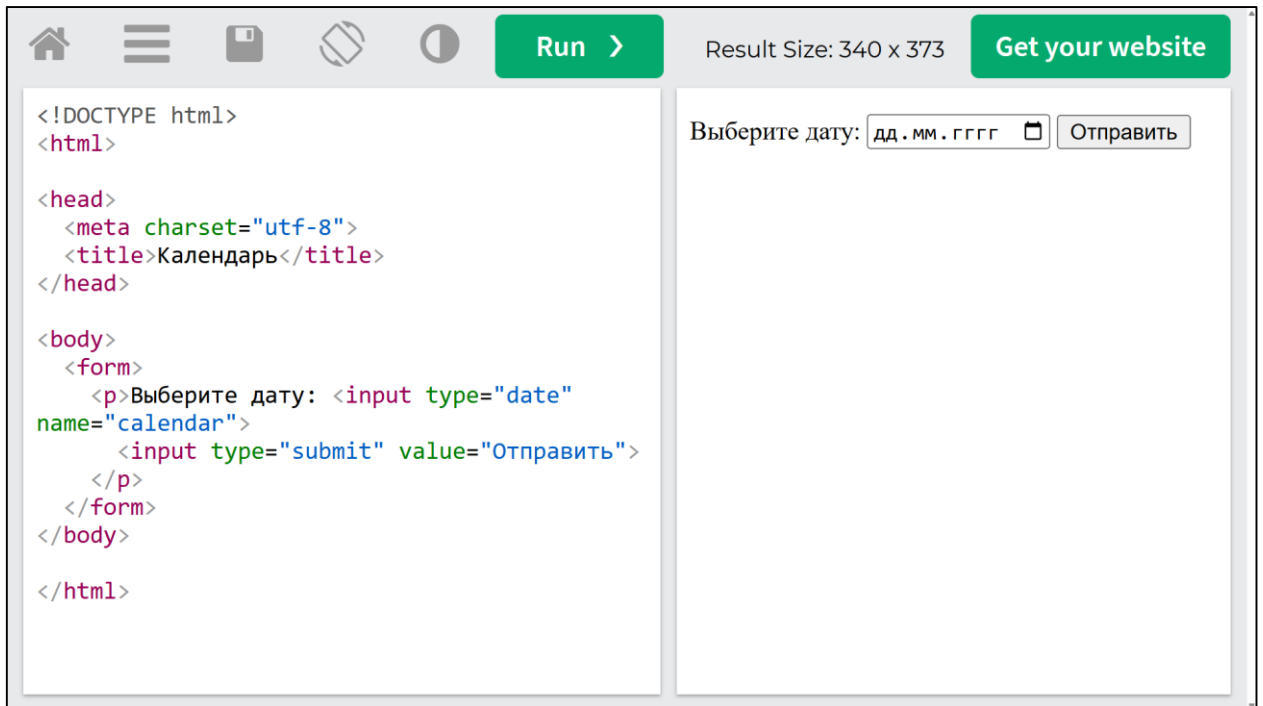


```

    </form>
</body>

</html>

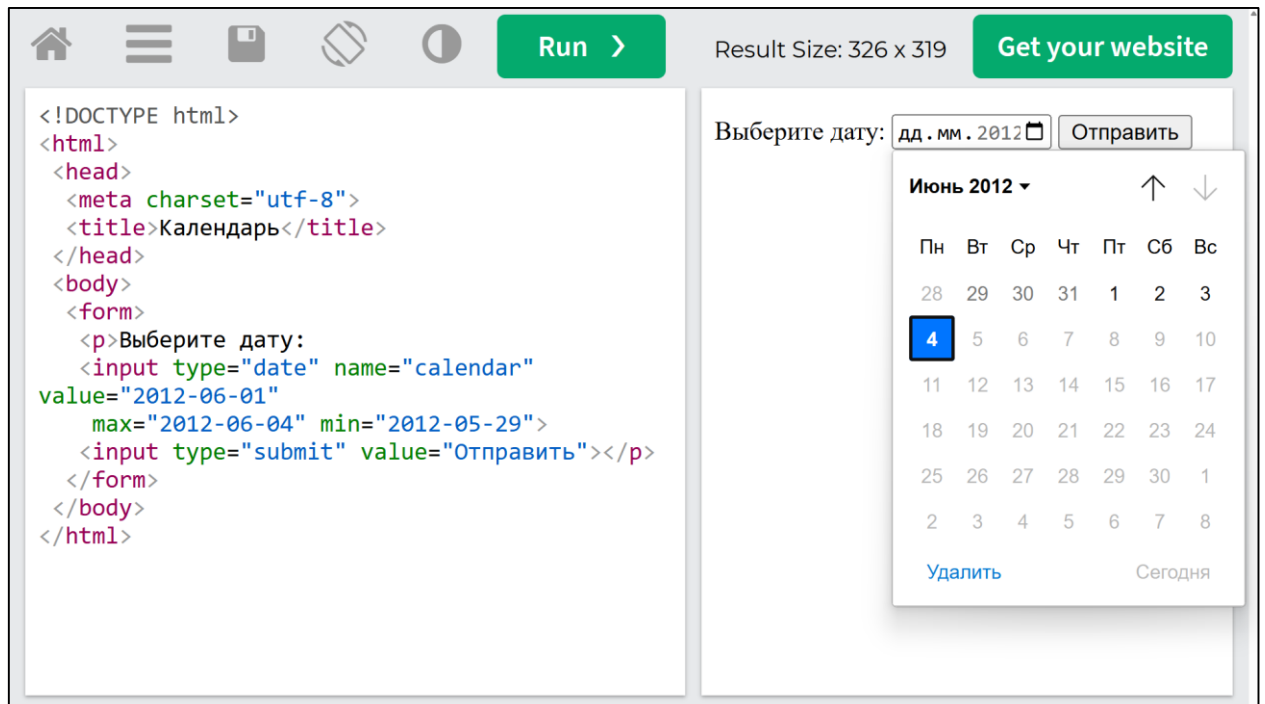
```



Допустимо ограничить ввод даты заданным значением через атрибуты `min` и `max`, они соответственно указывают нижнюю и верхнюю дату. Так, если вам требуется сузить диапазон ввода до ± 3 дней от даты 01.06.2012, то код запишется, как показано в примере 2.

Пример 2:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Календарь</title>
</head>
<body>
  <form>
    <p>Выберите дату:
      <input type="date" name="calendar" value="2012-06-01"
        max="2012-06-04" min="2012-05-29">
      <input type="submit" value="Отправить">
    </p>
  </form>
</body>
</html>
```



Текущая дата, заданная через атрибут value подсвечивается фоном, неактивные дни, которые нельзя выбрать — серым цветом (рис. 3).

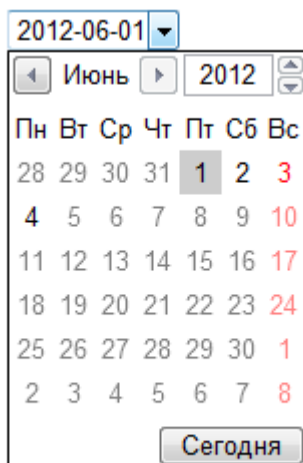


Рис. 3. Календарь с диапазоном ввода

В конце хотелось бы сказать, что для формы календаря характерны общие для всех типов атрибуты, они приведены в таблице ниже. Используя их можно сделать базовую настройку данной формы.

Атрибут	Описание
name	Имя поля, предназначено для того, чтобы обработчик формы мог его идентифицировать.
value	Начальное число, которое выводится в поле.
required	Обязательное поле.
readonly	Поле, доступное только для чтения.
disabled	Запрещает взаимодействие с полем.
min	Минимальное значение.
max	Максимальное значение.
id	Для связи.

Также календари имеют и характерные для них собственные атрибуты, такие как type:

Атрибут	Описание
type="date"	Дата (год-месяц-день).
type="datetime-local"	Дата и время.
type="time"	Время.
type="month"	Месяц и год.
type="week"	Неделя года.

Тип	Пример значения	Формат
date	2023-12-31	YYYY-MM-DD
datetime-local	2023-12-31T23:59	YYYY-MM-DDThh:mm
time	14:30	hh:mm
month	2023-12	YYYY-MM
week	2023-W52	YYYY-Www

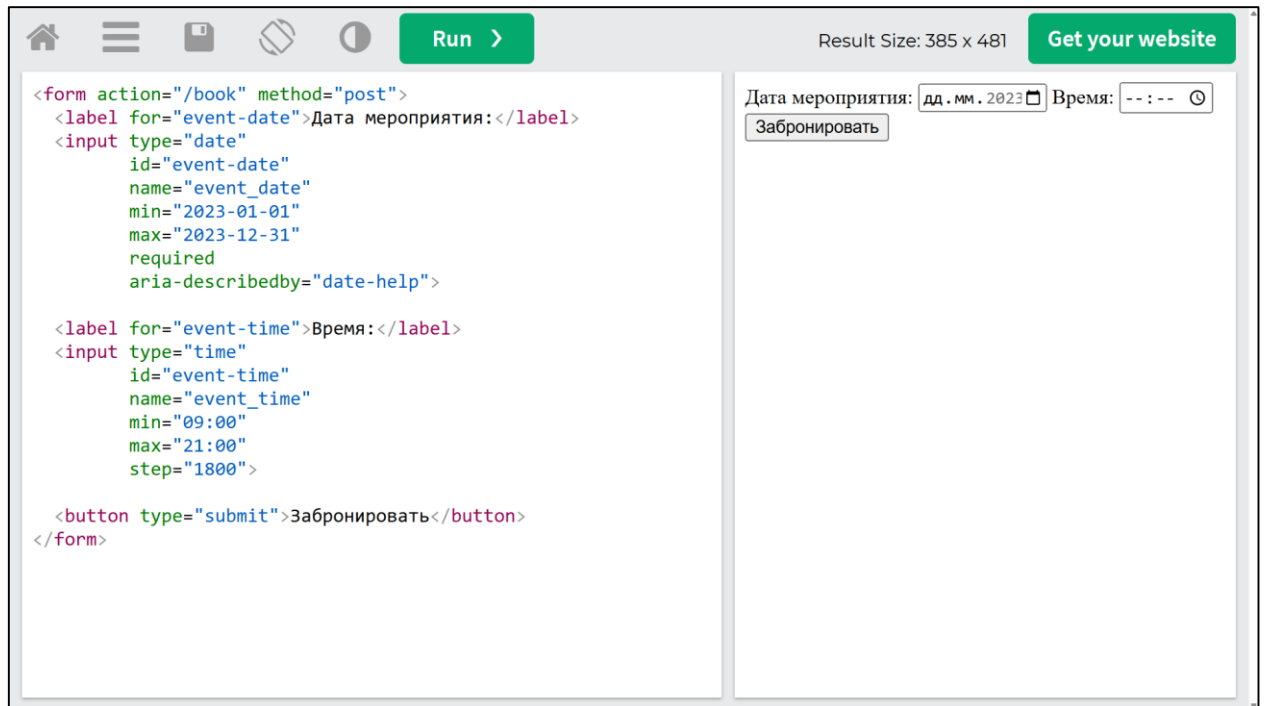
Пример 3:

```
<form action="/book" method="post">
  <label for="event-date">Дата мероприятия:</label>
  <input type="date"
    id="event-date"
    name="event_date"
    min="2023-01-01"
    max="2023-12-31"
    required
    aria-describedby="date-help">

  <label for="event-time">Время:</label>
  <input type="time"
    id="event-time"
    name="event_time"
    min="09:00"
    max="21:00"
```

`step="1800">`

`<button type="submit">Забронировать</button>`
`</form>`



Result Size: 385 x 481 [Get your website](#)

Дата мероприятия: Время:

```
<form action="/book" method="post">
  <label for="event-date">Дата мероприятия:</label>
  <input type="date"
    id="event-date"
    name="event_date"
    min="2023-01-01"
    max="2023-12-31"
    required
    aria-describedby="date-help">

  <label for="event-time">Время:</label>
  <input type="time"
    id="event-time"
    name="event_time"
    min="09:00"
    max="21:00"
    step="1800">

  <button type="submit">Забронировать</button>
</form>
```

9.17 Поле для поиска

На сайтах часто востребован поиск по ключевым словам, для ввода которых используется текстовое поле. Это очень полезная функция, ведь всегда удобнее найти информацию по знакомому слову, чем искать нужное, перемещаясь по кнопкам.

Разница между текстовым полем и полем для поиска состоит в стилистическом оформлении. На платформах, где поисковая форма имеет свой собственный уникальный вид, легко придать полю аналогичное оформление. В большинстве браузеров поле для поиска по своему виду никак не отличается от текстового поля, за исключением Chrome, Safari и IE10, которые добавляют небольшой крестик в правой части для быстрой очистки введённого текста (рис. 1).

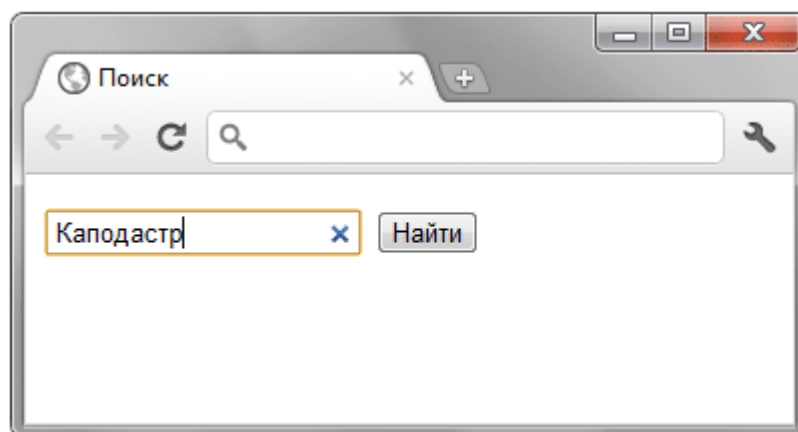
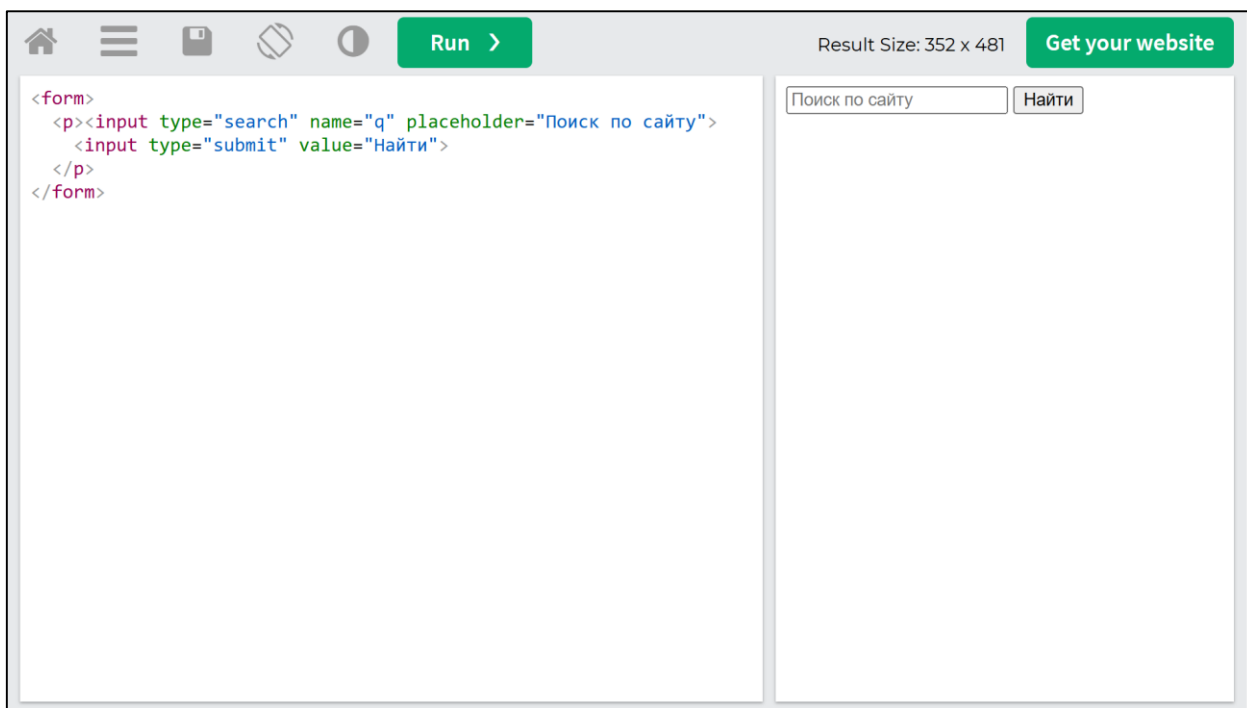


Рис. 1. Вид поля для поиска в Chrome

Пример 1:

```
<form>
  <p><input type="search" name="q" placeholder="Поиск по сайту">
    <input type="submit" value="Найти">
  </p>
</form>
```



Так как формы для поиска и для ввода текста очень похожи, то и атрибуты они принимают одинаковые.

9.18 Поле для телефона

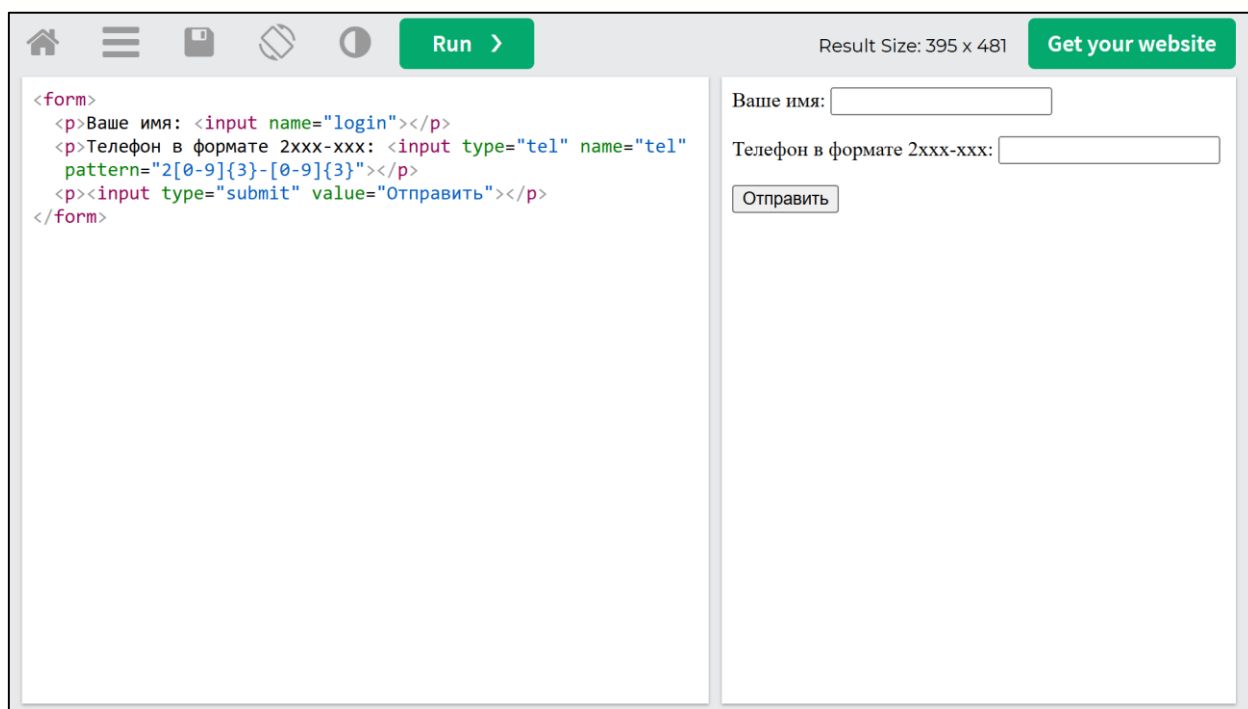
Чтобы указать телефонный номер используйте для этого специальное поле. По своему виду и работе оно совпадает с текстовым полем.

В отличие от веб-адреса или адреса электронной почты, поле для телефона не проверяется на правильность синтаксиса. Это связано с тем, что телефонные номера в разных местах мира могут иметь самые разнообразные формы написания, включать цифры, буквы и другие символы. Смысл применения этого поля в настройке шаблона ввода телефона, который используется в определённом городе или стране, а также в некоторых расширенных возможностях по управлению стилями.

Создание поля для телефона показано в примере 1. Требуется ввести телефон в указанном формате, иначе браузер выведет сообщение об ошибке и не станет отправлять форму, пока поле не будет заполнено корректно.

Пример 1:

```
<form>
  <p>Ваше имя: <input name="login"></p>
  <p>Телефон в формате 2xxx-xxx: <input type="tel" name="tel"
    pattern="2[0-9]{3}-[0-9]{3}"></p>
  <p><input type="submit" value="Отправить"></p>
</form>
```



The screenshot shows a web browser window with a toolbar at the top. The main content area displays the rendered HTML form. On the left, the source code is visible, showing the HTML tags for the form. On the right, the rendered form is shown, consisting of three parts: a text input field labeled 'Ваше имя:', a text input field labeled 'Телефон в формате 2xxx-xxx:' with a 'tel' type and a specific pattern, and a submit button labeled 'Отправить'.

Браузеры IE и Safari не поддерживают этот вид поля и соответственно не делают проверку на шаблон телефона.

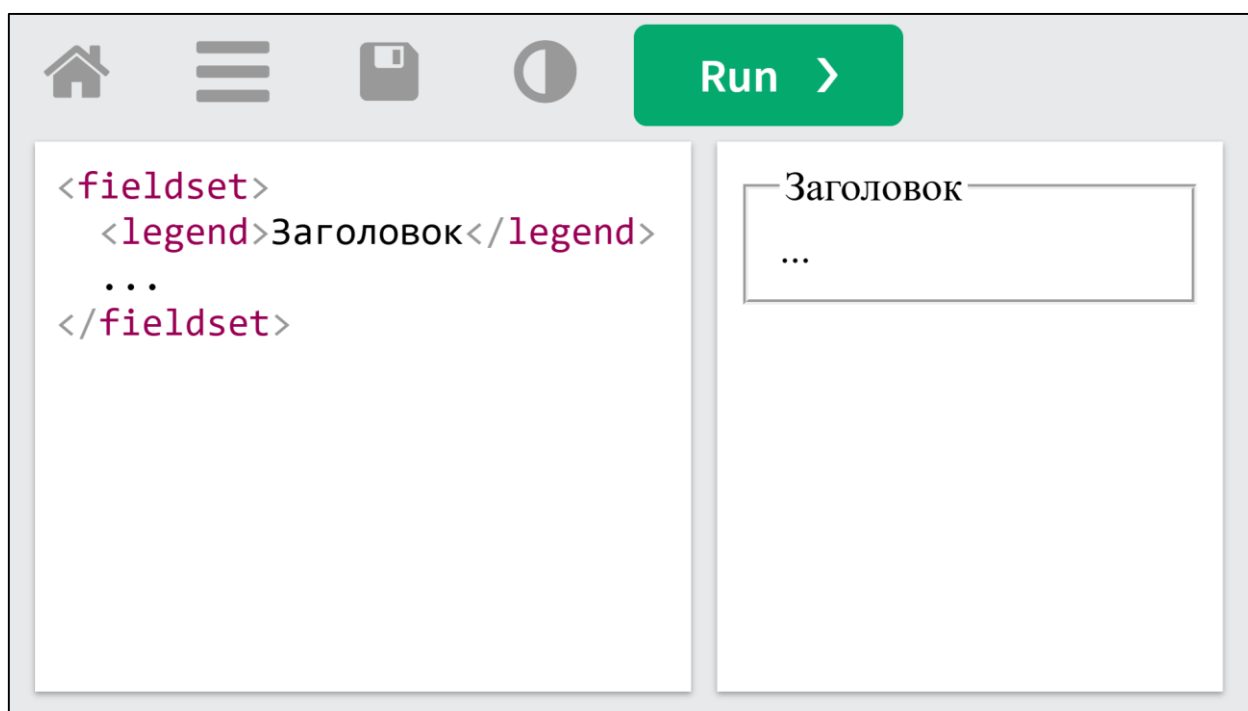
Атрибуты полностью совпадают с текстовым полем.

9.19 Группирование элементов форм

При создании сложной формы можно группировать некоторые элементы форм между собой, такое группирование удобно для пользователя и позволяет визуально отделить один логический блок от другого. Для этой цели применяется элемент `<fieldset>`, который создаёт рамку в форме с заголовком или без него. Структура кода следующая.

Пример 1:

```
<fieldset>
  <legend>Заголовок</legend>
  ...
</fieldset>
```



Элемент `<legend>` не обязателен, но если присутствует, должен идти сразу же после тега `<fieldset>`. Другие теги или текст перед `<legend>` недопустимы. Внутри `<legend>` можно использовать теги форматирования вроде `<b>`, `<i>`, `<sup>`, `<sub>` и др.

В примере 2 показано использование группирование на практике.

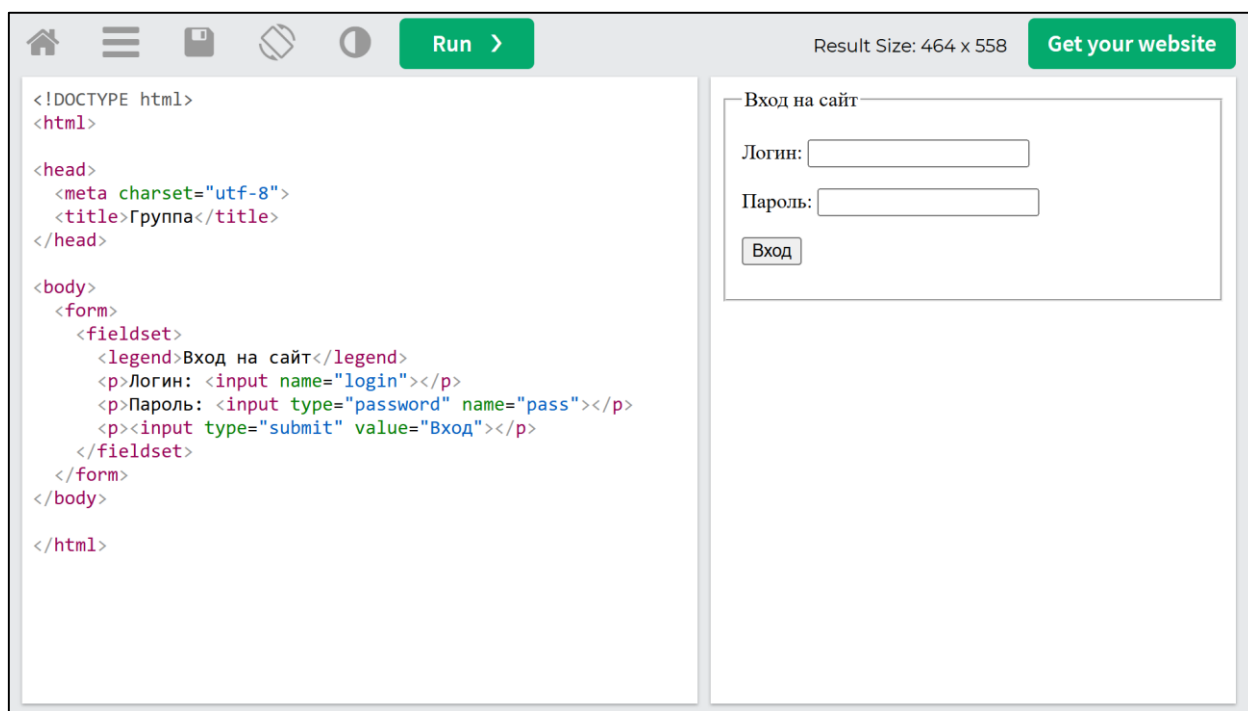
Пример 2:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Группа</title>
</head>

<body>
  <form>
    <fieldset>
      <legend>Вход на сайт</legend>
      <p>Логин: <input name="login"></p>
      <p>Пароль: <input type="password" name="pass"></p>
      <p><input type="submit" value="Вход"></p>
    </fieldset>
  </form>
</body>

</html>
```



Вид рамки в браузерах в целом одинаков — заголовок располагается в левом верхнем углу, встраиваясь прямо в рамку. Однако есть и мелкие различия, к примеру, в IE рамка имеет скруглённые углы, а текст плотнее прилегает к границе.

9.20 Базовые атрибуты форм

Несмотря на то, что в параграфах были приведены таблицы с подробным списком и описанием для каждого атрибута, находящегося в нем, есть смысл более подробно рассмотреть некоторые из них. Поэтому в следующих главах этого раздела будут подробно разобраны самые популярные из таких атрибутов, которые несомненно пригодятся в построении своего собственного сайта.

9.20.1 Переход между полями с помощью табуляции

При достаточно большом количестве полей формы, которые необходимо заполнить, переходить между ними с помощью курсора мыши становится неудобно. При этом требуется навести курсор на соответствующее поле, нажать кнопку мыши, и только после этого вводить нужное значение. Как альтернатива, используется клавиша Tab, которая позволяет быстро переключать фокус с одного поля на другое. Атрибут **tabindex** определяет последовательность перехода между полями при нажатии на Tab.

Важно:

- Фокусом называется активность поля, иными словами, поле доступно для того, чтобы в него вводили информацию или использовали какое другое действие. Например, кнопки, флажки, переключатели можно активизировать с помощью пробела.

Следующие элементы формы могут иметь атрибут **tabindex**: `<button>`, `<input>`, `<select>`, `<textarea>`. В качестве значения принимается число, которое задаёт шаг перехода. Так, номер 1 означает, что это поле первое получит фокус, номер 2 будет идти следующим и т.д. В примере 1 показано применение **tabindex** когда поля формы размещаются в ячейках таблицы. Если значение **tabindex** не указано, то по умолчанию переход по элементам формы происходит так, как они расположены в коде HTML, т.е. сверху вниз.

Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Порядок полей в форме</title>
  <style>
    fieldset {
      width: 200px;
      /* Ширина */
    }
  </style>
</head>

<body>
  <table border="1">
    <tr>
      <td><input type="text" tabindex="1" value="Имя" />
      <td><input type="text" tabindex="2" value="Фамилия" />
    </tr>
    <tr>
      <td><input type="text" tabindex="3" value="Адрес" />
      <td><input type="text" tabindex="4" value="Почта" />
    </tr>
  </table>
</body>
</html>
```

```

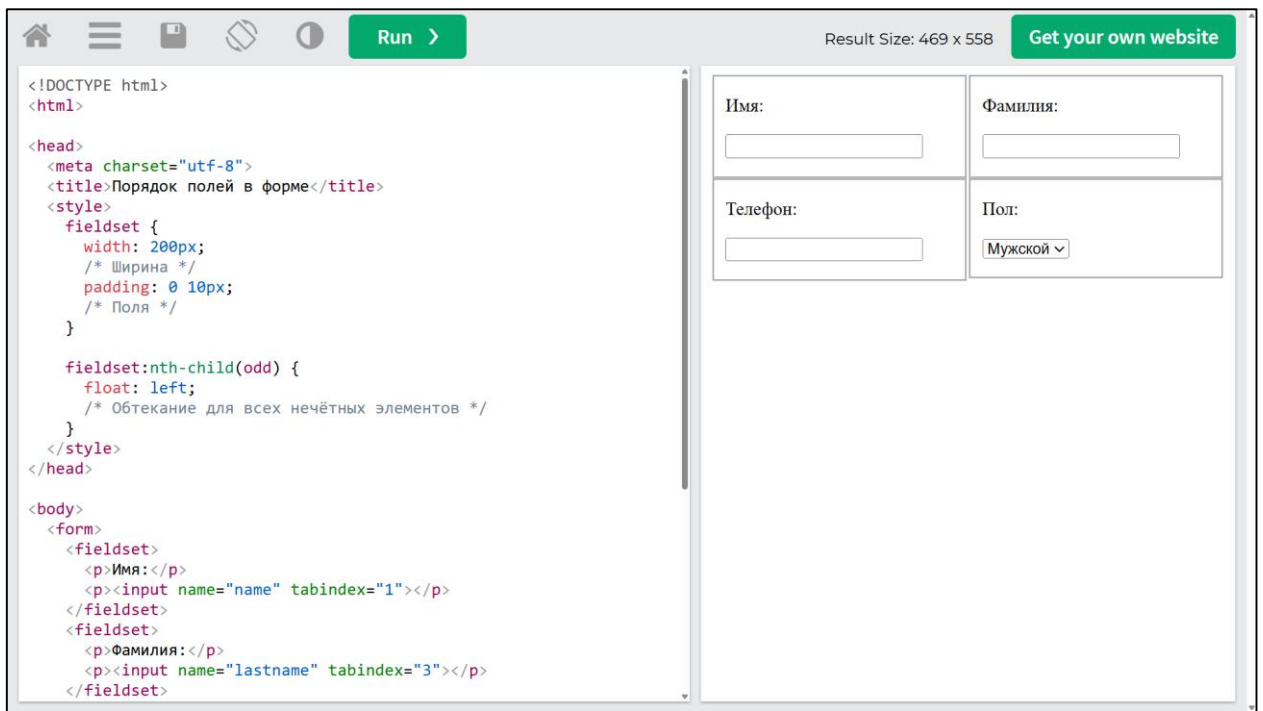
padding: 0 10px;
/* Поля */
}

fieldset:nth-child(odd) {
float: left;
/* Обтекание для всех нечётных элементов */
}
</style>
</head>

<body>
<form>
<fieldset>
<p>Имя:</p>
<p><input name="name" tabindex="1"></p>
</fieldset>
<fieldset>
<p>Фамилия:</p>
<p><input name="lastname" tabindex="3"></p>
</fieldset>
<fieldset>
<p>Телефон:</p>
<p><input name="tel" type="tel" tabindex="2"></p>
</fieldset>
<fieldset>
<p>Пол:</p>
<p><select name="gender" tabindex="4">
<option selected>Мужской</option>
<option>Женский</option>
</select></p>
</fieldset>
</form>
</body>

</html>

```



Обратите внимание, что с помощью табуляции фокус вначале получает поле, где надо ввести имя, а затем поле для ввода телефона.

9.20.2 Блокирование элементов форм

Блокирование элемента не позволяет вообще производить с ним каких-либо действий, в том числе выделять содержимое текстового поля, изменять его или активизировать. К тому же такие поля не отправляются на сервер.

Важно:

- Некоторые браузеры позволяют выделять и копировать содержимое заблокированного текстового поля, но все остальные действия недоступны.

На рис. 1 представлены разные элементы форм в заблокированном состоянии.

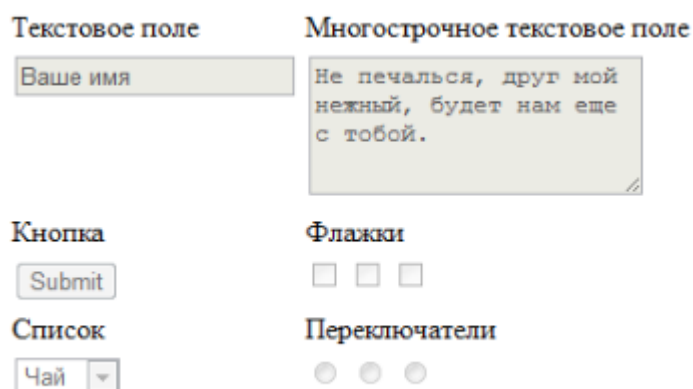


Рис. 1. Вид заблокированных элементов в Chrome

Для блокирования элемента формы используется атрибут **disabled**. Добавление этого атрибута разрешает отображать элемент формы, но не позволяет изменять его.

Блокирование элементов форм обычно используется для того, чтобы динамически с помощью скриптов изменять значение поля. Пользователь не должен в подобном случае иметь доступ к полю, поэтому оно блокируется. В примере 1 показано применение скриптов для изменения блокировки кнопки.

Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Блокировка поля</title>
  <script>
    function agreeForm(f) {
      // Если поставлен флажок, снимаем блокирование кнопки
      if (f.agree.checked) f.submit.disabled = 0
      // В противном случае вновь блокируем кнопку
    }
  </script>
</head>
<body>
  <div>
    <input type="checkbox" value="agree" /> Я согласен
    <input type="button" value="Submit" />
  </div>
</body>
</html>
```

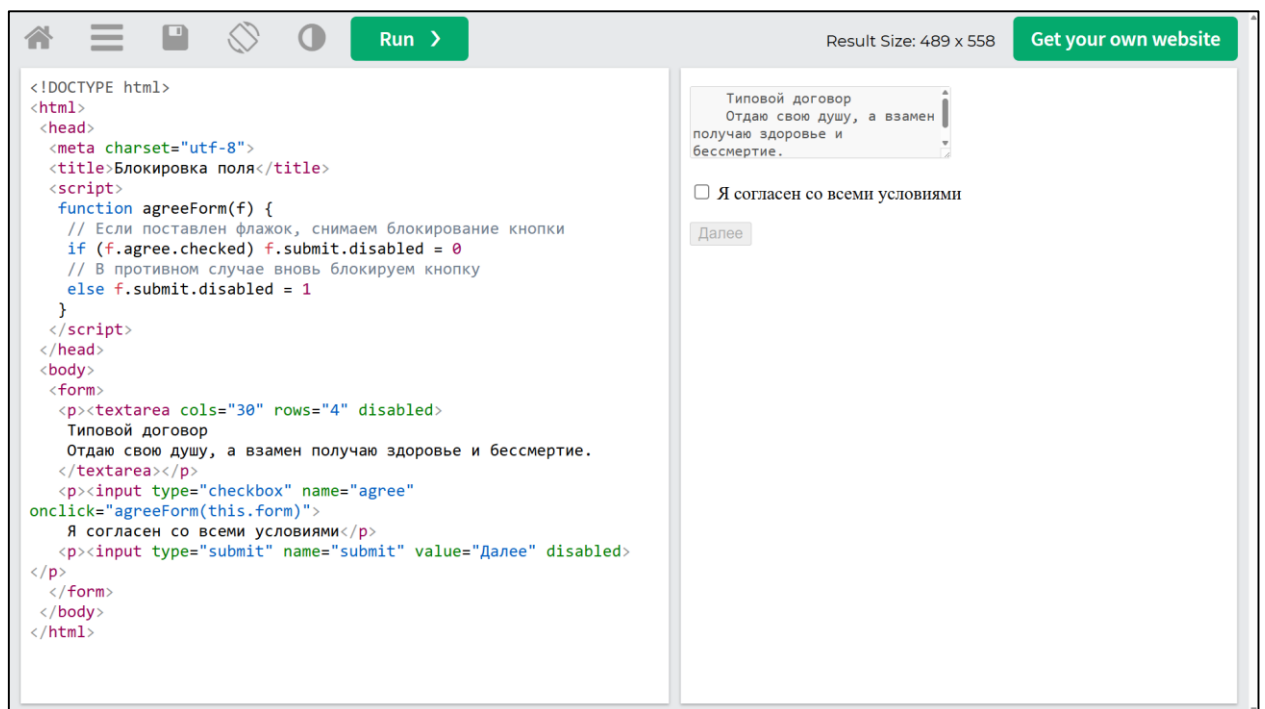
```

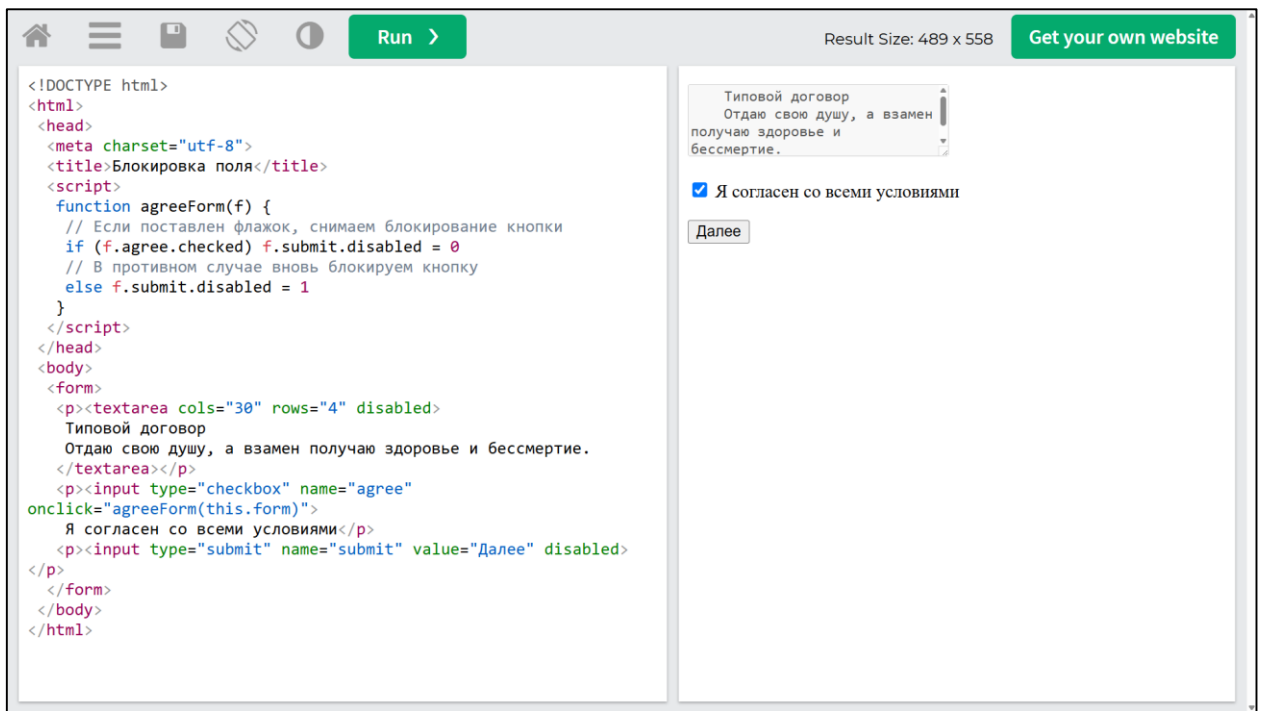
        else f.submit.disabled = 1
    }
</script>
</head>

<body>
    <form>
        <p><textarea cols="30" rows="4" disabled>
            Типовой договор
            Отдаю свою душу, а взамен получаю здоровье и бессмертие.
        </textarea></p>
        <p><input type="checkbox" name="agree"
onclick="agreeForm(this.form)">
            Я согласен со всеми условиями</p>
        <p><input type="submit" name="submit" value="Далее" disabled></p>
    </form>
</body>

</html>

```





В данном примере применяется блокирование кнопки, но оно снимается, как только пользователь поставит флажок возле текста «Я согласен со всеми условиями».

9.20.3 Поле только для чтения

Поля формы можно не только блокировать, но и переводить их в режим только для чтения. В этом случае доступ к ним сохраняется, но изменять значения заданные по умолчанию нельзя. Разумеется, речь идёт только о полях, где требуется вводить текст. Выделять и копировать текст можно, но изменить не получится.

Для установки режима «только для чтения» используется атрибут **readonly**, он добавляется к тегу `<input>` или `<textarea>`. На вид элемента формы это никак не влияет, но как было уже замечено, модифицировать значение поля не удастся.

Ниже представлен пример реализации поля доступного только для чтения, рекомендуется ознакомиться с ним наглядно в собственной среде разработки или в онлайн редакторе.

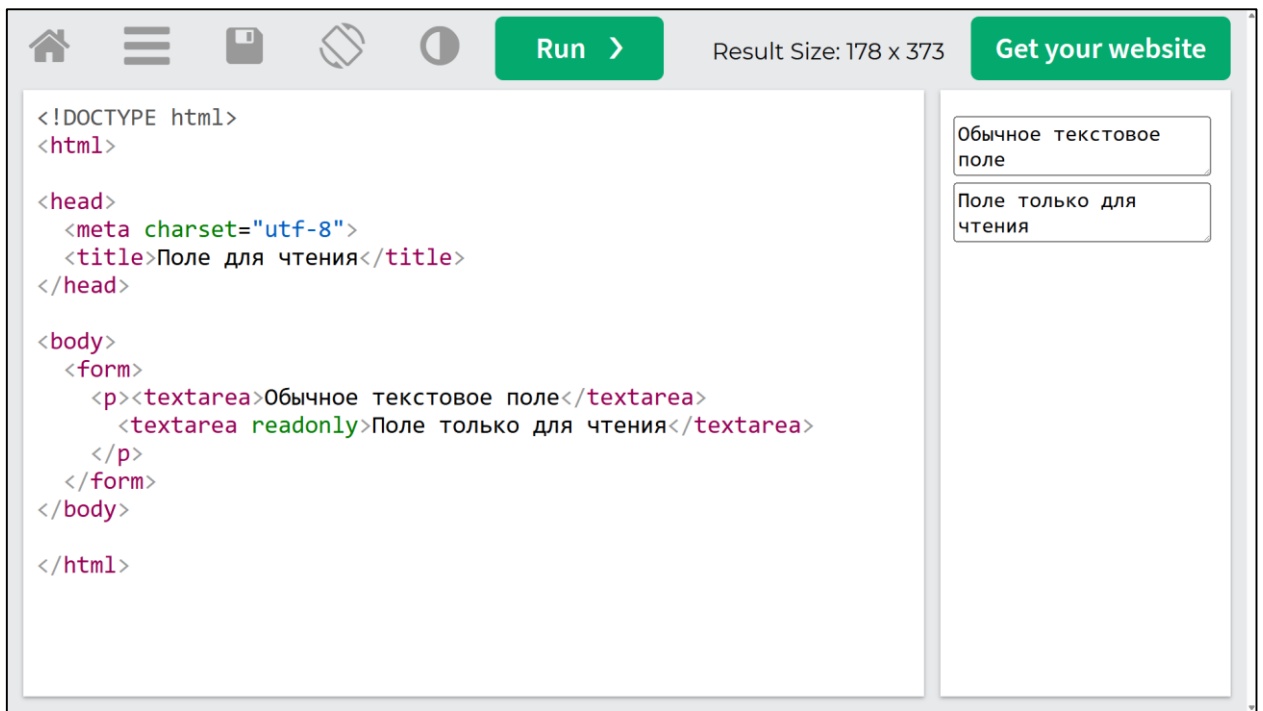
Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Поле для чтения</title>
</head>

<body>
  <form>
    <p><textarea>Обычное текстовое поле</textarea>
      <textarea readonly>Поле только для чтения</textarea>
    </p>
  </form>
</body>

</html>
```



9.20.4 Автофокус

Фокус — это активность элемента формы, позволяющая производить с ним какие-то действия. Для текстового поля можно вводить текст, для списка выбирать пункт с помощью клавиатуры и др. Автофокус — это автоматически установленный фокус поля формы. К примеру, при открытии google.ru вы можете сразу набирать текст в строке поиска без лишних манипуляций с мышью и клавиатурой.

Автофокус создаётся с помощью атрибута **autofocus**, который можно добавлять к тегам: `<button>`, `<input>`, `<keygen>`, `<select>`, `<textarea>`.

Важно:

- На странице должен быть только один элемент с автофокусом.

В примере 1 показано создание формы авторизации с автофокусом.

Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Автофокус</title>
</head>

<body>
  <form>
    <fieldset>
      <legend>Вход на сайт</legend>
      <p><input name="login" placeholder="Логин" autofocus></p>
      <p><input type="password" name="pass" placeholder="Пароль"></p>
      <p><input type="submit" value="Вход"></p>
    </fieldset>
  </form>
</body>

</html>
```



Run >

Result Size: 323 x 481

Get your website

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Автофокус</title>
</head>

<body>
  <form>
    <fieldset>
      <legend>Вход на сайт</legend>
      <p><input name="login" placeholder="Логин" autofocus></p>
      <p><input type="password" name="pass" placeholder="Пароль"></p>
      <p><input type="submit" value="Вход"></p>
    </fieldset>
  </form>
</body>

</html>
```

Вход на сайт

Логин

Пароль

Вход

9.20.5 Подсказывающий текст

В дизайне часто требуется вставить пояснение к текстовому полю, но не всегда для этого имеется место. Решением в таком случае является добавление подсказывающего текста непосредственно внутрь поля, при получении фокуса исходный текст должен пропадать. Это делается с помощью атрибута **placeholder**, значением которого служит любой текст.

Подсказка делается для полей:

- `<input type="text">`;
- `<input type="password">`;
- `<input type="search">`;
- `<input type="email">`;
- `<input type="tel">`;
- `<input type="url">`;
- `<textarea>`.

Подсказывающий текст отображается серым цветом, исчезает при получении фокуса и снова появляется, если фокус теряется. Если в поле с подсказкой ввести любой текст, то подсказка больше не появится. Также её не будет при наличии атрибута `value` с непустым значением.

[В примере](#) из прошлой главы показано добавление подсказки к полям формы для создания авторизации.

9.20.6 Шаблон для ввода данных

Текстовые поля никак не ограничивают ввод данных, хотя часто возникает необходимость задать для них параметры ввода, например, вводить только буквы, цифры или текст в определённом формате. Такие ограничения позволяют снизить ошибки пользователя и добиться от него ввода данных в нужном виде. Сам шаблон устанавливается для тега `<input>` с помощью атрибута **pattern** значением которого выступают [регулярные выражения](#).

Выражение	Описание
<code>\d [0-9]</code>	Одна цифра от 0 до 9.
<code>\D [^0-9]</code>	Любой символ кроме цифры.
<code>\s</code>	Пробел.
<code>[A-Z]</code>	Только заглавная латинская буква.
<code>[A-Za-z]</code>	Только латинская буква в любом регистре.
<code>[А-Яа-яЁё]</code>	Только русская буква в любом регистре.
<code>[A-Za-zA-Яа-яЁё]</code>	Любая буква русского и латинского алфавита.
<code>[0-9]{3}</code>	Три цифры.
<code>[A-Za-z]{6,}</code>	Не менее шести латинских букв.
<code>[0-9]{,3}</code>	Не более трёх цифр.
<code>[0-9]{5,10}</code>	От пяти до десяти цифр.
<code>^[a-zA-Z]+\$</code>	Любое слово на латинице.
<code>^[А-Яа-яЁё\s]+\$</code>	Любое слово на русском включая пробелы.
<code>^[0-9]+\$</code>	Любое число.

В примере 1 показано, как сделать ввод IP-адреса.

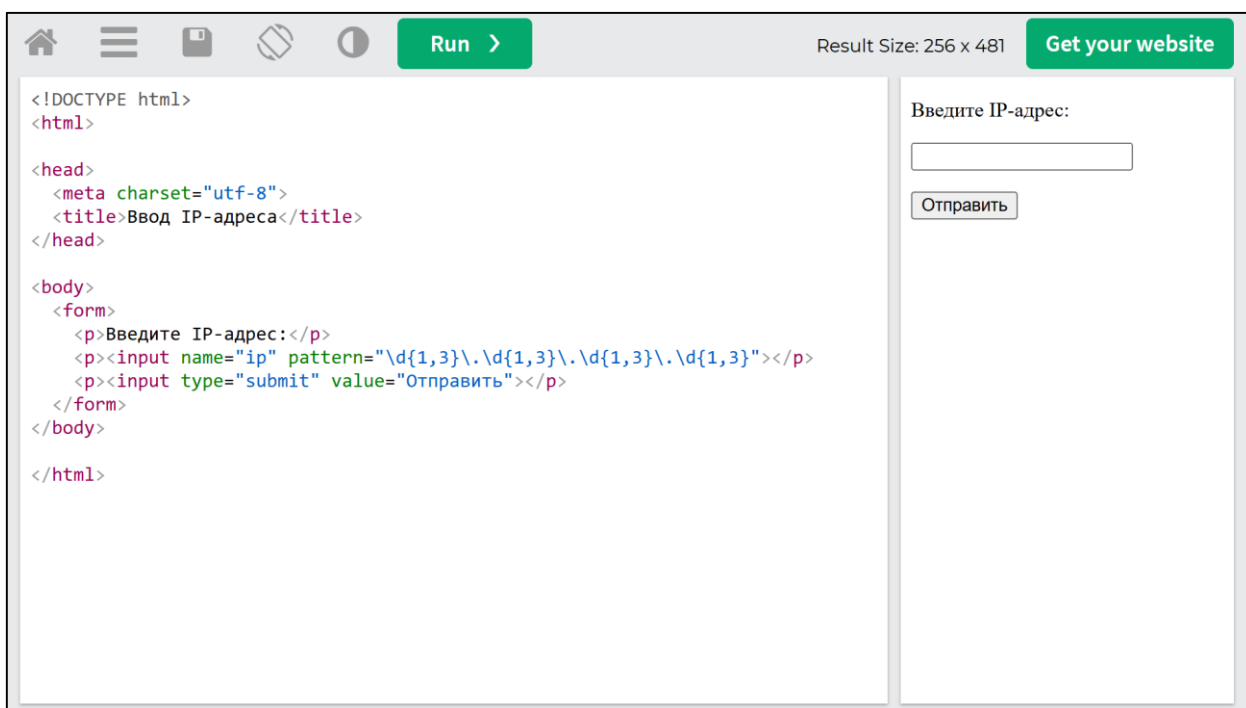
Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Ввод IP-адреса</title>
</head>

<body>
  <form>
    <p>Введите IP-адрес:</p>
    <p><input name="ip"
pattern="\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"></p>
    <p><input type="submit" value="Отправить"></p>
  </form>
</body>

</html>
```



При вводе значения, не соответствующего шаблону будет выводиться сообщение об ошибке. Вид и содержание этого сообщения зависит от браузера, так, в Chrome оно будет иметь вид, как показано на рисунке ниже.



Run >

Result Size: 256 x 481

Get your website

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Ввод IP-адреса</title>
</head>

<body>
  <form>
    <p>Введите IP-адрес:</p>
    <p><input name="ip" pattern="\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"></p>
    <p><input type="submit" value="Отправить"></p>
  </form>
</body>

</html>
```

Введите IP-адрес:

ахахахаха

 Введите данные в указанном формате.

9.21 Атрибуты для защита от дурака

«Защитой от дурака» называется комплекс мер по пресечению ввода неправильной информации в форме. Например, если в поле требуется ввести положительное число от 0 до 10, то следует проверить, чтобы пользователь не ввёл текст или число, которое не лежит в указанном диапазоне, т.е. число не должно быть меньше нуля и больше десяти.

Почему происходит ввод неправильной информации? Это в основном совершается по трём причинам.

- 1) Пользователь ошибся случайно, например, невнимательно прочитал, что ему требуется указать.
- 2) На веб-странице неоднозначно просят ввести данные, поэтому пользователю приходится гадать и делать предположение, что же в действительности от него хотят. При этом не всегда происходит совпадение мнений разработчика и пользователя.
- 3) Есть ряд людей, которые воспринимают инструкции как вызов и стараются поступить наоборот. Такие пользователи рассуждают примерно так: «Ага, меня просят ввести число. А что будет, если я укажу буквы?». После чего задают явно неправильную информацию и смотрят, к чему это приведёт.

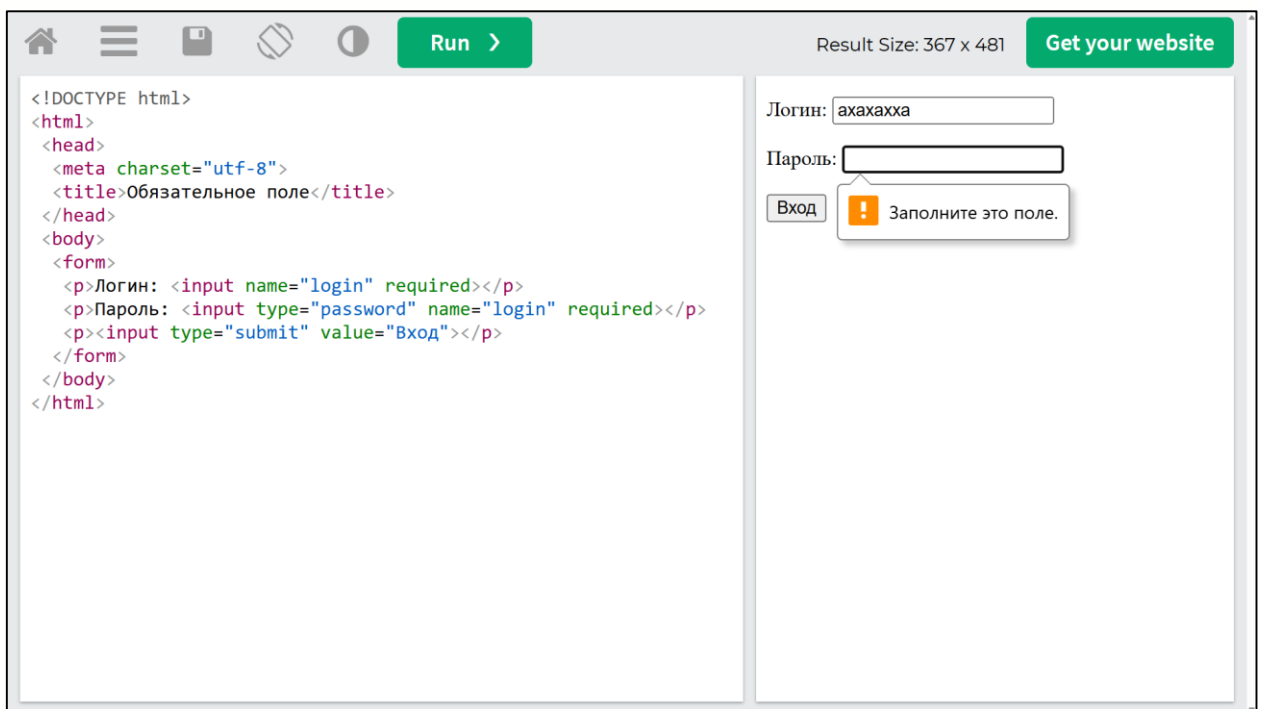
Следует понимать, что точные и правильные формулировки хотя и снижают вероятность возникновения ошибок, но никак не спасают от них. Только технические средства на стороне сервера позволяют получить требуемый результат и избежать ввода неправильной информации. Тем не менее, ревизия или, как её ещё называют, валидация на стороне клиента позволяет быстро проверить данные, вводимые пользователем, на корректность, без отправки формы на сервер. Таким образом экономится время пользователя и снижается нагрузка на сервер. С позиции юзабилити (это показатель того, насколько легко и удобно пользователю взаимодействовать с интерфейсом сайта. С английского *usability* так и переводится — удобство использования) тоже имеются плюсы — пользователь сразу получает сообщение о том, какую информацию он указал неверно и может исправить свою ошибку.

9.21.1 Обязательное поле

Некоторые поля формы должны быть обязательно заполнены перед их отправкой на сервер. Это, к примеру, относится к форме регистрации, где требуется ввести логин и пароль. Для указания обязательных полей используется атрибут **required**, как показано в примере 1.

Пример 1:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Обязательное поле</title>
  </head>
  <body>
    <form>
      <p>Логин: <input name="login" required></p>
      <p>Пароль: <input type="password" name="login" required></p>
      <p><input type="submit" value="Вход"></p>
    </form>
  </body>
</html>
```



Обязательные поля должны быть заполнены перед отправкой формы, иначе форма на сервер не отправится и браузер выдаст об этом предупреждение. Вид сообщения зависит от браузера. Пример реакции Chrome на незаполненное поле формы представлен выше.

9.21.2 Корректность данных

Исходно имеется два поля, в котором вводимые пользователем данные проверяются автоматически. Это веб-адрес и адрес электронной почты. Браузер Chrome также проверяет на корректность поле с календарными данными, но только потому, что у него не предусмотрен интерфейс выбора календаря щелчком мыши. Для этих элементов характерны следующие правила.

- Веб-адрес (<input type="url">) должен содержать протокол (http://, https://, ftp://).
- Адрес электронной почты (<input type="email">) должен содержать буквы или цифры до символа @, после него, затем точку и домен первого уровня.

У браузеров несколько различается политика по проверке данных пользователя. К примеру, Opera подставляет протокол http:// перед введённым текстом автоматически, тогда как другие браузеры ждут его от пользователя. Chrome и Opera требуют, чтобы в почтовом адресе была точка, для Firefox она не обязательна.

В примере 1 показана форма с обязательными полями, в которой два поля проверяется браузером.

Пример 1:

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Корректность данных</title>
</head>

<body>
  <form>
    <p>Заполните форму (все поля обязательны)</p>
    <p>Имя: <input name="name" required></p>
    <p>Email: <input type="email" name="email" required></p>
    <p>Сайт: <input type="url" name="site" required></p>
    <p><input type="submit" value="Отправить"></p>
  </form>
</body>

</html>
```



Run >

Result Size: 367 x 481

Get your website

```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Корректность данных</title>
</head>

<body>
  <form>
    <p>Заполните форму (все поля обязательны)</p>
    <p>Имя: <input name="name" required></p>
    <p>Email: <input type="email" name="email" required></p>
    <p>Сайт: <input type="url" name="site" required></p>
    <p><input type="submit" value="Отправить"></p>
  </form>
</body>

</html>
```

Заполните форму (все поля обязательны)

Имя:

Email:

 Адрес электронной почты должен содержать символ "@". В адресе "ахахаха" отсутствует символ "@".

9.21.3 Шаблон ввода

Некоторые данные нельзя отнести к одному из видов элементов формы, поэтому для них приходится использовать текстовое поле. При этом их ввод происходит по определённому стандарту. Так, IP-адрес содержит четыре числа разделённых точкой (192.168.0.1), почтовый индекс России ограничен шестью цифрами (124007), телефон содержит код города и конкретное количество цифр, часто разделяемых дефисом (391 555-341-42) и др. Браузеру необходимо указать шаблон ввода, чтобы он согласно нему, проверял вводимые пользователем данные. Для этого используется атрибут **pattern**, а его значением выступает регулярное выражение. Некоторые типовые значения перечислены в таблице ниже.

Шаблон	Описание
<code>^[a-zA-Z]+\$</code>	Любые латинские буквы.
<code>^[0-9]+\$</code>	Любое количество цифр.
<code>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}</code>	IP-адрес.
<code>[0-9]{6}</code>	Почтовый индекс.
<code>\d+(\,\d{2})?</code>	Цена в формате 1,34 (разделитель запятая).
<code>\d+(\.\d{2})?</code>	Цена в формате 2.10 (разделитель точка).

В примере 1 просят ввести шестнадцатеричное значение цвета (#ffcc00) и, если оно не лежит в этом диапазоне, браузер выводит сообщение об ошибке.

Пример 1:

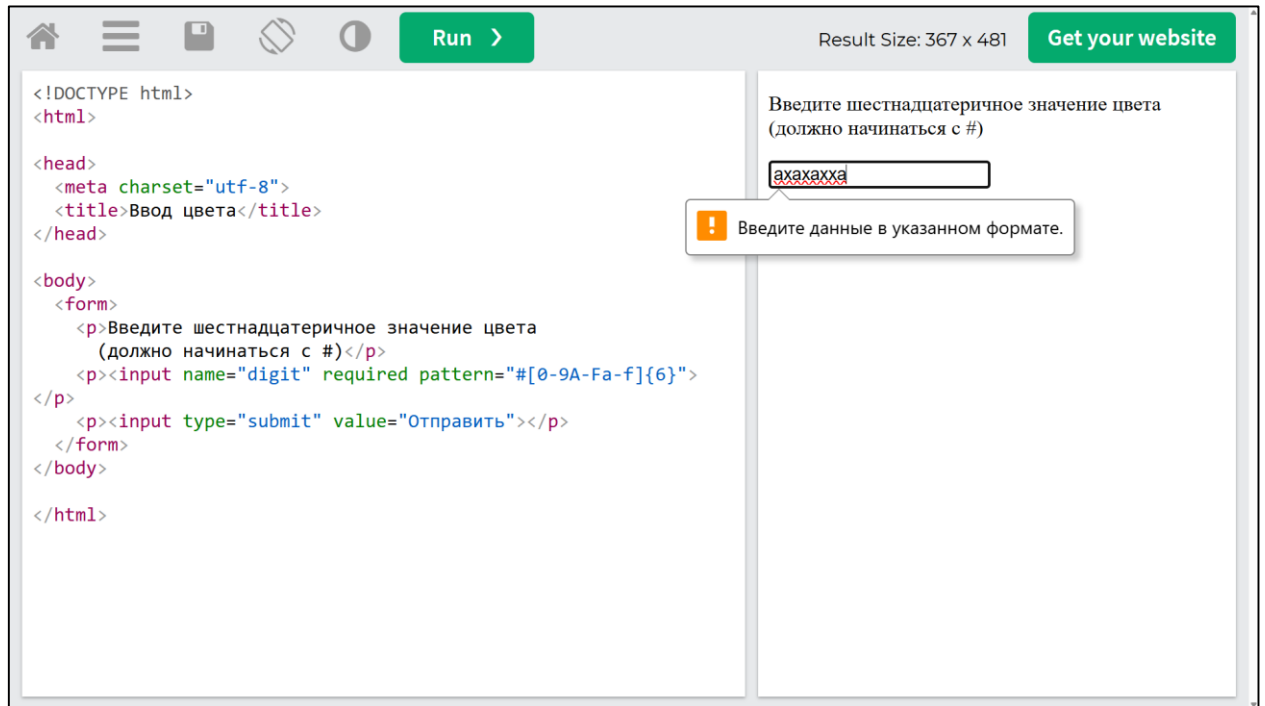
```
<!DOCTYPE html>
<html>

<head>
  <meta charset="utf-8">
  <title>Ввод цвета</title>
</head>

<body>
  <form>
    <p>Введите шестнадцатеричное значение цвета
      (должно начинаться с #)</p>
    <p><input name="digit" required pattern="#[0-9A-Fa-f]{6}"></p>
    <p><input type="submit" value="Отправить"></p>
```

```
</form>
</body>

</html>
```

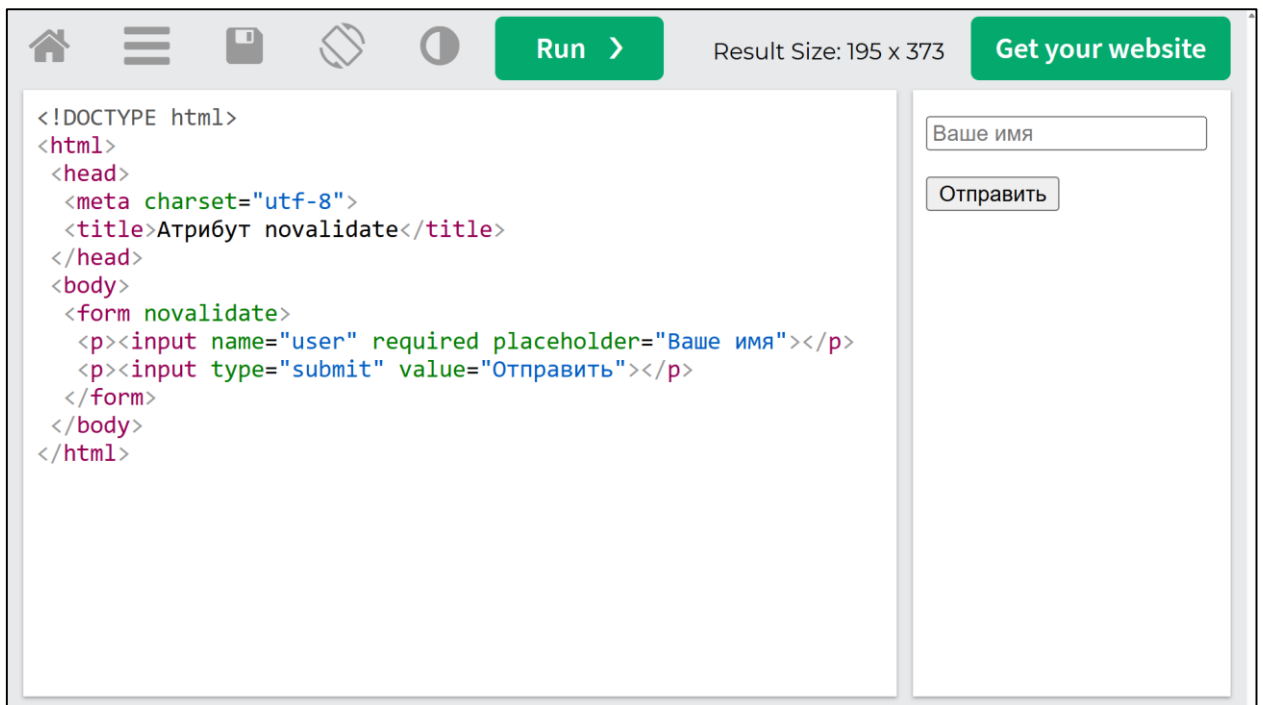


9.21.4 Отмена валидации

Валидация не всегда требуется для формы, к примеру, разработчик пожелает использовать универсальное решение на JavaScript и дублирующая проверка браузером ему уже ни к чему. В подобных случаях необходимо отключить встроенную валидацию. Для этого применяется атрибут **novalidate** тега `<form>`. В примере 4 показано использование этого атрибута.

Пример 1:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Атрибут novalidate</title>
  </head>
  <body>
    <form novalidate>
      <p><input name="user" required placeholder="Ваше имя"></p>
      <p><input type="submit" value="Отправить"></p>
    </form>
  </body>
</html>
```



Для аналогичной цели применяется и атрибут **formnovalidate**, который добавляется к кнопке для отправки формы, в данном случае к тегу `<input type="submit">`. В этом случае форма из примера 4 будет иметь следующий вид.

Пример 2:

```
<form>  
  <p><input name="user" required placeholder="Ваше имя"></p>  
  <p><input type="submit" value="Отправить" formnovalidate></p>  
</form>
```



Заключение

Если в ходе прочтения документа у вас появились предложения по переформулировки какой-либо его части, ее дополнении или вы нашли ошибки, то просьба сообщить об этом. Заранее спасибо!

Почта для связи: lundenmn@mail.ru

Основная работа окончена: 04.05.2025